



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校

参赛队号

队员姓名	1.
	2.
	3.

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题目：通用神经网络处理器的多核切图与调度优化

摘要：

面向具有多条计算与搬运管道的通用神经网络处理器，多核切图必须同时权衡计算并行、跨子图通信、同步等待与共享存储竞争。本文将算子依赖图划分为子图，联合确定核心映射和核上顺序，以赛题评估程序给出的总完成时间为主目标，以额外数据搬运量为同时时间下的次目标。三个问题分别对应跨子图均经 DDR 的独立 Task、允许同核复用的跨核同步 Task，以及新增共享 FIFO 只读 L2 的场景；我们在共同决策空间上逐层加入运行机制。

对问题一，建立保持依赖无环的有向图划分与多核列表调度模型，从完整分量、局部切分和共享输入关系构造结构不同的初解，再对关键区域做有界联合调整。100 张图的逐图平均加速比在 1 至 5 核下分别为 1.000、1.835、2.546、3.161、3.666；五核总完成时间为 73170501 周期。对问题二，模型显式约束跨核 COPY 完成后的 500 周期释放，并按同核驻留与四条管道的事件执行评价。在同一批图上，五核逐图平均加速比为 4.160，总完成时间为 65769876 周期；相对本工程上一轮完整方案，400 个多核配置中 13 组改善、387 组持平。

对问题三，进一步跟踪共享 L2 的 FIFO 队列，把命中搬运与 DDR 搬运分配到各自共享带宽池，并区分固定调度方案的硬件效应与重新调度收益。五核逐图平均加速比达到 4.240，总完成时间为 64562282 周期；但 500 组逻辑额外搬运合计增加 4418288 字节，且仍有 4 组比无 L2 对照更慢。三个场景均完成 100 图、1 至 5 核的逐组官方评价与结果复核。结果说明，缓存命中或搬运减少不能单独代表执行时间收益，方案优劣应回到完整依赖和资源事件下的最终完成时间判断。

关键词：有向无环图划分；多处理器调度；离散事件仿真；共享缓存

1 问题重述

1.1 背景与任务

通用神经网络处理器以多个 AI 核执行计算图。每核具有矩阵计算、向量计算与数据搬运等独立执行通道，但核内存储有限，多个核心共享核外 DDR 带宽。原始计算图为带张量依赖的有向无环图；切分粒度决定可同时运行的工作量，也改变输入重复读取、跨子图数据搬运和缓存溢出。题目要求输出每个非 COPY_IN/COPY_OUT 算子的子图编号，以及各核心上的子图顺序，再由给定评估程序确定总执行周期和额外搬运量^[1]。

三个问题的决策形式相近，运行语义逐步改变。问题一的场景 A 规定一个子图就是一个 Task，子图边界上的数据总经 DDR 中转；即使两个 Task 在同一核上连续执行，也不保留私有缓存。问题二的场景 B 把同核各子图纳入一个 Task，允许核内驻留，但跨核数据需等待同步释放。问题三在场景 B 上增加全核共享的只读 FIFO L2，读请求可由 DDR 或 L2 服务。因而，问题二和三不能直接沿用场景 A 的通信成本公式，也不能把问题三的缓存命中率当作独立可选变量。

1.2 所需回答与指标

对每张图和每个核数 $k = 1, \dots, 5$ ，本文构造合法的切图、核心分配与核上顺序。首要评价量为官方仿真得到的完成时间 $T_{i,k}$ (cycles)，同时报告逻辑额外数据搬运量 $D_{i,k}$ (bytes)。设 $T_{i,1}^0$ 是题目规定的整图单核核内启发式结果，则前两问的逐图加速比与平均加速比定义为

$$S_{i,k} = \frac{T_{i,1}^0}{T_{i,k}}, \quad \bar{S}_k = \frac{1}{100} \sum_{i=1}^{100} S_{i,k}. \quad (1)$$

这是一百个比值的算术平均，不能换成总单核周期除以总多核周期。问题三另比较相同图、相同核数下的无 L2 与有 L2 周期，单核缓存收益由真实仿真决定，不人为设为 1。

1.3 研究范围

全部实验使用题目给定的 100 张计算图和固定硬件参数。本文不改变图的算子、张量或官方核内启发式；搜索仅改变题目允许的划分、映射和子图顺序。求解器运行秒数与被评估方案的执行周期是不同量，前者反映求解效率，后者是模型的主要目标。各问的逐例结果列于附录，所有整体结论均以同一百图为统计总体。

2 模型假设与符号说明

2.1 假设及适用依据

假设输入图与官方固定配置正确给出算子、张量、周期、依赖和硬件参数；算子不能被拆成更细的可调度原子，张量大小与算子周期在一次评价内确定。忽略 CPU 主控本身未由官方评价器计入的时间，不把网络传输、温度或动态频率引入模型。每个方案需满足题目规定的覆盖、无环、核内顺序、L1/UB 容量及资源占用约束。若容量压力触发溢出，新增 COPY 由原始核内启发式确定，不能由外层算法任意删除。

使用的 100 张图由原附件逐字节无损提取，没有删除异常图、归一化周期或调整张量大小。导入时对全部图执行官方结构校验，并保存源文件与每份模型输入的 SHA-256；所有问题读取同一处理目录。由此，三问结果的差异来自场景语义及合法方案，而非改写数据。前期挑选小图和反例用于调试与同预算对照；最终汇总只以完整百图为准，不把开发样本当独立盲测集。

2.2 固定硬件量

L1 容量为 524 288 bytes，UB 容量为 131 072 bytes，共享 DDR 总带宽为 60 bytes/cycle。场景 A 的跨核任务等待为 1000 cycles，同核连续 Task 间隔为 100 cycles；场景 B 的跨核 COPY 释放延迟为 500 cycles。问题三的共享只读 L2 容量为 1 048 576 bytes，带宽为 250 bytes/cycle。以上均是主实验的固定配置，灵敏度扫描仅用于解释机制，不替换正式结果。

2.3 统一符号

表 1 主要符号

符号	含义
$G = (V, E), V_c$	原计算 DAG 及其中的非 COPY 算子集合
$\pi : V_c \rightarrow \{1, \dots, q\}$	操作至子图的划分; q 为子图数
$m : \{1, \dots, q\} \rightarrow \{1, \dots, k\}$	子图至核心的映射; σ_c 为核心 c 上的合法顺序
$p = (\pi, m, \sigma)$	一份完整调度方案; $\mathcal{P}_A, \mathcal{P}_B$ 为对应场景的合法方案集
$T_i^A(p), T_i^B(p), T_i^{L2}(p)$	第 i 图在三种语义下的官方完成周期
$D_i(p)$	相对原始图 COPY 总量的逻辑额外搬运字节数
$Q(t), C, b_D, b_L$	时刻 t 的共享 FIFO 队列、L2 容量、DDR 与 L2 带宽
h_e, s_e, a_e, f_e	COPY 事件 e 的命中标志、字节数、发射和完成时刻

3 问题分析与总体路线

3.1 关键权衡

合并强依赖算子可以减少边界 COPY、重复读取和任务释放等待，但过度聚合会压缩多核并行空间，并可能让核内启发式在 L1/UB 容量压力下插入更多换入换出。过度切分则使共享 DDR 竞争、同步及启动间隔增加。因此，通信字节数、任务数或各核算子数都不是总完成时间的充分代理。若 Cube 与 Vector 工作量不均，按算子个数平衡也会让其中一条 Pipe 长期空闲。论文以合法方案的官方 Makespan 为最终准则，将成本估计只用于有限候选的生成和排序。

三问共同采用“结构候选—依赖调度—官方评价—有界改进”的框架。先识别计算依赖和共享输入，构造完整连通分量、主导分量有界拆分等不同粒度的合法初解；再按后续关键工作量和最早可行完成时间安排核心与空隙；最后用少量局部划分、迁移和顺序调整寻找更短的总时间。场景 B 的候选必须重新按同核驻留、跨核 COPY 释放和容量规则评价；共享 L2 的候选还需经过 FIFO 事件仿真。算法保留已知可行方案，在限定评价机会内比较，不进行分组、核心和排列的组合穷举。

3.2 三种运行语义的关系

表 2 概括后两问相对前一问的改变。相同的输出字段不意味着相同的代价函数：尤其在场景 B 中，一个核心内的多个子图仍有先后次序，但其通信可利用驻留，跨核释放以具体数据搬运事件为准。问题三缓存只改变合资格读取的服务路径，不能绕过算子依赖、同步和四条 Pipe 的占用。

表 2 三问运行语义与增量机制

内容	问题一：场景 A	问题二：场景 B	问题三：B 加共享 L2
Task 与驻留	一子图一 Task; 边界不驻留	同核子图同属一 Task; 可驻留	保持 B 的 Task 与私有存储
跨核数据	边界经共享 DDR; 按 A 释放	COPY 完成后另有同步释放	同 B, 不因命中取消同步
读取服务	DDR	DDR	COPY_IN 可命中共享 FIFO L2
主要决策	划分、映射、核上序	同左, 重估驻留和生命周期	同左, 重估缓存事件与双带宽池

3.3 结构画像：为什么需要多种初解

在建立具体求解器之前，可用操作数、弱连通分量数、最大分量规模与最长拓扑深度描述图的结构，但不能仅凭这些静态量断言最终周期。表 3 列出本工程早期用于开发与确认的六张图的结构快照；它们不是由最终性能反推而后选出的“六张有利图”，也不是一百图的全体统计。可切计算操作数与总操作数不同，因为原始 COPY 等节点不由外层分配子图。最大分量占比 χ 按式 (53) 用可切操作数计算。第 48 图近乎单一深长依赖分量，第 18、33 图却有大量小分量；对这两类图使用相同的“每块固定算子数”会在一类图上切碎关键链，在另一类图上错失天然独立块。因此分量完整性、主导分量有界切分和拓扑窗口必须作为不同初解来源并存。

表 3 六张结构诊断图的静态画像；不是全百图的分布统计

图号	总操作	可切计算操作	弱连通分量	最大分量	最大拓扑深度
12	1304	972	50	98	18
48	1900	1728	1	1728	368
50	4781	4555	23	4011	185
18	3285	2727	126	59	27
33	4069	3465	95	64	33
67	8996	8804	71	124	123

具体计算最大分量占比：第 48 图 $\chi = 1728/1728 = 1$ ；第 50 图 $\chi = 4011/4555 \approx 0.8806$ 。完整分量直接分核，至少有一个核心需要承担主导分量的大部分工作，难以充分利用其余核心。相反，第 18 图 $\chi = 59/2727 \approx 0.0216$ ，第 33 图 $\chi = 64/3465 \approx 0.0185$ ，分量整体分核已经提供很多可调的独立工作块；对它们过早细切单个小分量，很可能增加 Task 或跨核边界而无明显并行收益。第 67 图操作多，却有 71 个分量、最大分量仅 124 个可切操作，提示“图很大”不等于“需要把某一个分量切成极小块”。这说明主机运行耗时与 NPU 最优划分尺度是不同问题：第 67 图因为节点多、官方核内展开复杂而求解贵，仍可先尝试以完整分量为单位构造高质量初解。

这些结构量也并非独立决定瓶颈。最大拓扑深度是按操作层最长链的节点数，忽略每个操作的周期、Pipe 类别与张量字节；两条深度相同的链可以有完全不同的计算时长和 DDR 压力。弱连通分量不计外部输入共享，因此不同分量即使没有中间依赖，也可能共同消费同一输入张量，分给不同 Task 后重复读取。最大分量占比则只用算子数量，没有权衡某些算子的高周期或大张量。正因如此，表 3 只用于初解门控和结构解释，不能替代官方评价或作为全局最优的预言变量。

3.4 静态结构与五核实测的交叉核对

为检验结构判断有没有和实际结果完全脱节，表 4 把同六张图在五核下的两种场景实测并列。第 48 图在 A 中只取得约 1.1245 倍，在 B 中提高到约 2.1863 倍；第 50 图从约 1.5062 倍到约 2.6254 倍。它们的主导分量和长链为“完整分量单核”带来明显限制，场景 B 允许同核驻留并重构候选后表现更好。不过这个成对结果同时改变了 Task 规则、同步、正式候选集，不能把差值单独归因于某一个因素。第 18、33 图无主导分量，A 的五核表现已接近五倍；B 中绝对周期反而略高。第 67 图 A/B 五核周期也几乎持平。这些反例说明后续场景的机制更丰富，不意味着每张图都严格比前一问快。

对第 48 图，仅看“最大分量占比为一”还不足以决定把链切在何处；若切点位于共享张量的高扇出处，会产生大量边界 COPY。可沿关键输出的祖先链选择候选切点，估计切开后双计算 Pipe 的并行余量，再查看跨切点张量集合和释放点。对第 18、33 图，完整分量数量已远超过五核，最重要的问题转为把分量按两 Pipe 负载放到核心、以及控制共享输入重复读取；继续细化每个很小的分量大概率只是增加候选准备成本。对第 67 图，五核周期很大却已有超过四倍的相对加速，若以赛题逐图均值为目标，它贡献与任一小图相同；

表 4 结构诊断图的五核实测对照：两场景使用各自自己选方案

图号	A 周期	A 逐图加速比	B 周期	B 逐图加速比
12	13406	4.4076	12519	4.7199
48	197613	1.1245	101641	2.1863
50	99382	1.5062	57017	2.6254
18	203806	4.9281	205917	4.8775
33	353592	5.0176	361176	4.9123
67	13455316	4.7442	13461153	4.7422

若以总周期为目标，它的绝对改善又非常重要。结构分型因此同时服务于哪类初解有价值 and 主机搜索时间投向哪里两项不同决策。

3.5 从三问差异推出统一而不混淆的求解路线

共同变量 $p = (\pi, m, \sigma)$ 允许三个问题共享合法性框架和结构候选，但不允许共用一个不变的通信权重矩阵。场景 A 的同核跨 Task 张量也必须经 DDR，故主要边界是子图；场景 B 的同核多个子图在同一 Task 中可驻留，故主要边界转成核心；场景 C 又使相同 B 方案中的某些读入由 L2 而非 DDR 服务，并引入全局 FIFO 历史。若按“通信成本从 A 到 C 逐渐减小”简单描述，会漏掉 L2 竞争与非单调反例。严谨的递进关系是三个评价算子

$$\mathfrak{E}_A \circ \mathfrak{R} \circ \mathfrak{G}_A, \quad \mathfrak{E}_B \circ \mathfrak{R} \circ \mathfrak{G}_B, \quad \mathfrak{E}_C \circ \mathfrak{R} \circ \mathfrak{G}_B, \quad (2)$$

其中 $\mathfrak{E}_C$ 比 $\mathfrak{E}_B$ 多共享 L2 队列和第二服务池，而 $\mathfrak{G}_A$ 与 $\mathfrak{G}_B$ 采用不同 Task 与 COPY 规则。这个表达把可复用的图结构候选与必须重评的物理成本分开，解释了为何问题二以问题一方案作种子、却全部重放正式评价；问题三以问题二方案作保底、再做固定方案消融与少量缓存事件候选。

算法最终回答的是每个配置的具体方案，而非仅给出理论表达式。对一张图，结构初解多样化是为了避免落入单一分块尺度；关键工作量和空隙插入是为了让已存在的计算资源不被低优先级任务占住；有限官方评价是为了让 DDR 竞争、核内溢出与 FIFO 路由这些非线性因素进入最终接受判断。若后续证明某个解析代理完全复刻官方仿真，它在计算成本上就应按正式评价记账；在本论文中代理始终被限定为有误差的排序工具。统一框架的优势不是宣称一个权重公式适用于三问，而是在每问按相应事件语义重新评估的同时保持候选结构、合法性和证据口径的一致。

3.6 共同的图表示与决策空间

原始输入是一张有向无环的算子-张量图。令 $\mathcal{O}$ 为操作节点， $\mathcal{X}$ 为张量节点；张量 x 的生产操作为 $p(x)$ ，消费操作集合为 $C(x)$ ，大小为 b_x 字节。原始的 COPY_IN 与 COPY_OUT 是既有数据路径，不属于外层切图决策；令 $\mathcal{O}_c$ 为允许分配子图的其余操作。为讨论依赖，在操作层构造投影弧集合

$$E_c = \{(u, v) : \exists x \in \mathcal{X}, p(x) = u, v \in C(x), u, v \in \mathcal{O}_c\}. \quad (3)$$

投影只便于分析，不丢弃张量的身份、字节数及一对多消费关系。若一个输入张量同时供给三个算子，三条投影边仍共享同一个逻辑张量；通信是否发生三次必须由具体场景的 Task 和缓存规则决定。将 $(p(x), C(x))$ 看成有向超边，正是为了避免把共享输入的读入次数误等同于普通割边条数。

对于核数 k ，方案表示为 $p = (\pi, m, \sigma)$ 。其中 $\pi : \mathcal{O}_c \rightarrow \{1, \dots, q\}$ 为非空子图划分， $m : \{1, \dots, q\} \rightarrow \{1, \dots, k\}$ 为核心映射， σ_c 是核心 c 上各子图的排列。若 $\pi(u) \neq \pi(v)$ 且 $(u, v) \in E_c$ ，则商图有弧 $\pi(u) \rightarrow \pi(v)$ 。核心上的顺序还诱导相邻子图串行弧。令

$$H(p) = (\{1, \dots, q\}, E_\pi \cup E_\sigma), \quad E_\pi = \{(\pi(u), \pi(v)) : (u, v) \in E_c, \pi(u) \neq \pi(v)\}. \quad (4)$$

必要合法条件是 $H(p)$ 无环。证明并不只依赖原图无环：即使 E_π 无环，错误的核上顺序也

可能产生 $A \rightarrow B$ 的数据依赖和 $B \rightarrow A$ 的串行顺序，形成互等。对每个候选重新拓扑检查 $H(p)$ ，才能保证至少存在满足先后关系的顺序扩展。图中的任意子图必须非空，且每个可切算子仅出现一次：

$$V_r = \pi^{-1}(r) \neq \emptyset, \quad \bigcup_{r=1}^q V_r = \mathcal{O}_c, \quad V_r \cap V_s = \emptyset \quad (r \neq s). \quad (5)$$

本文不把“每块固定包含若干算子”当作可行性条件；固定块长仅是生成一种候选的启发式参数。任务数、映射和排列均随图结构变化。

这种联合决策的规模解释了为什么必须限制搜索。忽略依赖后， $n = |\mathcal{O}_c|$ 个带标签算子划成 q 个非空无标签子图已有第二类 Stirling 数 $S(n, q)$ 种，满足

$$S(n+1, q) = S(n, q-1) + qS(n, q). \quad (6)$$

前一项让新算子独立成组，后一项把它加入已有 q 组之一。再为 q 个子图选核心，最多有 k^q 个映射；固定每核的子图集合后，还存在各核排列。依赖和内存约束会删去部分组合，但并不会使“枚举全部划分、核心、次序”在百图规模上可行。因此结构候选只覆盖有依据的局部，官方评价机会在同一预算下分配，而不是对指数空间作均匀遍历。

3.7 从固定工作量到事件完成时间

若把每个子图预先赋予一个固定时长，再做普通多机排程，模型会漏掉两层内生变化。其一，外层划分改变核内张量的生存区间，官方核内启发式可能在容量压力下插入换出与换入；于是同一子图放在不同上下文中，展开后的 COPY 与 Pipe 序都可能不同。其二，多个核心的搬运共同占用 DDR，实际传输时长取决于同时在途操作的集合。令 e 为展开后的执行事件， s_e, f_e 为起止时刻， $d_e = f_e - s_e$ 为其实现时长。对固定时长的计算操作，有 $d_e = c_e$ ；对搬运事件， d_e 由带宽过程产生。发射必须等待它的所有前驱、同一 Pipe 的上一事件和外部释放：

$$s_e \geq \max \left\{ \max_{a \in \text{pred}(e)} f_a, f_{\text{prevPipe}(e)}, r_e^{\text{ext}} \right\}. \quad (7)$$

上式是依赖约束，尚不是一个可直接把 d_e 相加的显式日程；传输与计算可在不同 Pipe 上重叠。将所有实际事件和资源先后关系构成扩展事件图 $\mathcal{H}(p)$ 后，在固定一次仿真已经给出的搬运时长条件下，完成时间等于源到汇的最长事件路径：

$$T(p) = \max_{\gamma \in \Gamma(\mathcal{H}(p))} \left(\sum_{e \in \gamma} d_e + \sum_{(a,e) \in \gamma} \ell_{ae} \right). \quad (8)$$

这里 ℓ_{ae} 只放路径上真正额外发生的释放等待。不同任务的 DDR 忙碌、计算和同步可能同时发生，不能把全局“DDR 忙碌总时长”“Pipe 空闲总时长”“任务释放等待总时长”直接相加当作可节省周期。诊断必须追问某段等待是否落在最终输出的控制路径上。式 (8) 是事后时间线的解释式；因为 d_e 随竞争状态变化，它不是外层优化的线性目标。

共享 DDR 可以按连续时间的服务工作量描述。令 $A_D^+(t)$ 为时刻 t 尚有正剩余工作量的 DDR 搬运集合，事件 e 发射时的独占参考工作量为 $w_e(s_e) = \lceil b_e/B_D \rceil$ ，其中 $B_D = 60$ bytes/cycle。在活跃集合不变的开区间内，公平共享的连续部分满足

$$\frac{dw_e(t)}{dt} = -\frac{1}{|A_D^+(t)|}, \quad e \in A_D^+(t). \quad (9)$$

到达或完成事件发生时先结算过去时间段的剩余工作，再重算下一完成事件；最后按官方整数周期与同刻事件次序执行。式 (9) 解释了为什么只对某个 Task 加上 $b_e/60$ 惩罚不准确：新传输会拖慢同时在途的其他传输，而与其无依赖的 Cube/Vector 计算可能继续。一个候选改变启动时刻后，竞争集合也随之改变，局部估价只能为官方评价筛选候选。

3.8 可证明下界与诊断的边界

令 W_M 、 W_V 为原图中矩阵和向量固定周期操作的总工作量， L_{op} 为只计这些操作本身周期的最长依赖链。无论切图怎样安排，在 k 核、每核各一条同类 **Pipe** 的条件下，都有

$$T_{i,k} \geq L_0(k) := \max \left\{ \frac{W_M}{k}, \frac{W_V}{k}, L_{\text{op}} \right\}. \quad (10)$$

工作量项由抽屉原理推出：若所有核心的矩阵 **Pipe** 占用都小于 W_M/k ，总占用便小于必须执行的 W_M ，矛盾；向量 **Pipe** 同理。最长链项由逐条算子依赖递推，链上计算不能并行。原图固定周期操作不因划分消失，所以式 (10) 对三个问题均适用。它忽略了存储、启动和同步，因而可能很松，但与候选方案无关，可以安全作为统一理论参照。

对已固定的方案 p ，设实际事件展开中 DDR 总搬运字节为 $B_D(p)$ 。带宽容量再给出一个更强但方案依赖的事后界：

$$T(p) \geq \frac{B_D(p)}{60}, \quad L(p) = \max \left\{ L_0(k), \frac{B_D(p)}{60} \right\} \leq T(p). \quad (11)$$

因为 DDR 在一周期内合计最多搬运 60 字节，全部字节即便连续传输也要至少上述时间。 $B_D(p)$ 包含原始 **COPY**、因划分新增的 **COPY** 和核内溢出；它随 p 改变。因此 $T(p) - L(p)$ 只能描述这个候选相对自身宽松界的松弛，不能据此宣称它距所有方案的最优值还有多少。即使差距很小，也可能有另一方案同时显著减少 B_D 和 T 。对于问题三，还须把 DDR 与 L2 分成两个独立服务池，不能把两条带宽简单相加。

3.9 两种全量指标的严格区别

对图 i ，题目指定整图核内启发式给出固定单核参考时间 $T_{i,1}^0$ ，所交多核方案的时间为 $T_{i,k}$ 。赛题规定的“平均加速比”是

$$\bar{S}_k = \frac{1}{N} \sum_{i=1}^N \frac{T_{i,1}^0}{T_{i,k}}, \quad N = 100. \quad (12)$$

若关心这一批图合计要占用多少周期，也可报告总周期下降率

$$\rho_k = 1 - \frac{\sum_i T_{i,k}}{\sum_i T_{i,1}^0}. \quad (13)$$

二者不是同一统计量。令 $S_{i,k} = T_{i,1}^0/T_{i,k}$ ，则总周期比可改写为

$$\frac{\sum_i T_{i,1}^0}{\sum_i T_{i,k}} = \frac{\sum_i T_{i,k} S_{i,k}}{\sum_i T_{i,k}} = \sum_i \omega_{i,k} S_{i,k}, \quad \omega_{i,k} = \frac{T_{i,k}}{\sum_j T_{j,k}}. \quad (14)$$

所以总周期比是以多核耗时为权重的加速比，而式 (12) 是每图等权。两个数值可以同时陈述，但不能互换作题目要求的折线图纵轴。若某些大图总周期改善很多、而大量小图改善一般，总周期下降可以明显，平均加速比却不一定同幅增长。后续结果表同时列出平均加速比、百图总周期和额外搬运，正是为了避免单一指标遮蔽这层差异。

3.10 从操作依赖到真实可执行任务的两级展开

外层变量 $p = (\pi, m, \sigma)$ 只有子图标识、核归属及同核顺序，尚不是可直接执行的周期时间线。它必须经过两个确定性映射：首先是场景规则 $\mathfrak{G}_s$ ，依据场景 $s \in \{A, B, C\}$ 把跨子图张量改写成必要的 **COPY**、同步和 **Task** 释放；随后由核内启发式 $\mathfrak{R}$ 为各 **Task** 建立 **Pipe** 顺序与容量安全的换入换出；最后事件仿真器 $\mathfrak{E}_s$ 给出性能。可把评估写成复合函数

$$(T_s(p), D_s(p), \mathcal{L}_s(p)) = \mathfrak{E}_s(\mathfrak{R}(\mathfrak{G}_s(G, p))), \quad (15)$$

其中 $\mathcal{L}_s(p)$ 为时间线与资源账本。提出一个新划分时， $\mathfrak{G}_s$ 的输出可能有不同数量的 **COPY**， $\mathfrak{R}$ 的换出决定亦可能变化；因此 $T_s(p)$ 不是一个关于子图边界的线性函数。场景 C 在相同

图展开后又增加状态相关的 FIFO 路由，故事件映射也随历史轨迹变化。这一复合结构是模型与算法分工的根本：解析式负责刻画合法性、建立下界和排序候选；完整事件评估负责判断候选的真实目标值。

对固定 Task r ，定义其操作事件集合 E_r ，每个事件 e 有一个所属 Pipe、前驱集合及可能的张量写入或释放动作。如果把全部事件的持续时间直接相加，得到的是串行上界式直觉，并非真实时长。例如 Cube 上的事件 a 与 Vector 上的事件 b 若无相互依赖，时间区间可以相交，Task 长度至少为 $\max\{d_a, d_b\}$ ，不必达到 $d_a + d_b$ ；若 b 依赖一个正在 DDR 上读取的张量，它只能等待对应读入结束，而另一个无关的 Cube 仍可继续。因而当 DDR 拥塞增加一段 δ 周期时，Task 的完成时间增量满足的只有 $0 \leq \Delta T \leq \delta$ 这一类在固定事件图和单处扰动下的条件性关系；若延迟改变事件顺序，连这些条件也须重新检查。不能把每条搬运的增量一概加到整个 Task 长度。

3.11 共享带宽的状态方程与整数周期边界

为说明仿真为何必须事件驱动，设 DDR 搬运 e 的剩余独占工作量为 $w_e(t) \geq 0$ ，发射事件发生于 a_e ，初值 $w_e(a_e) = \lceil b_e/B_D \rceil$ 。在无到达和无完成的区间 $[t, t + \Delta)$ ，若活跃集合大小为 n_D ，则

$$w_e(t + \Delta) = \max\{0, w_e(t) - \Delta/n_D\}, \quad e \in A_D(t). \quad (16)$$

所能安全推进的下一个完成间隔为 $\Delta_D = n_D \min_{e \in A_D(t)} w_e(t)$ ；外部新任务、核内 Pipe 事件可能更早发射，因此实际推进量应取 $\min\{\Delta_D, \Delta_{\text{next arrival}}, \Delta_{\text{next compute}}\}$ 。事件发生后重新计算活跃集合及其服务份额。官方整数周期语义包含取整和同刻事件处理顺序，式 (16) 是解释该机制的连续服务形式；没有复刻同刻顺序的自写代理，不能声称和官方完全等价。该表述也说明“每条边按 $b_e/60$ 加一项”有两类误差：其一，独占时间忽略已有并发搬运在新流加入后变慢；其二，持续时间不能直接转换成最终输出的增量，因为一部分拥塞藏在其它 Pipe 的计算后面。

若对方案 p 的 DDR 事件集合记为 $E_D(p)$ ，其总独占工作量定义为

$$W_D(p) = \sum_{e \in E_D(p)} \left\lceil \frac{b_e}{B_D} \right\rceil. \quad (17)$$

连续公平服务在所有在途传输之间重新分配这份工作。若每条传输的整数周期取整后必须服务至少 $\lceil b_e/B_D \rceil$ 个“单流等效周期”，便有 $T(p) \geq W_D(p)$ ；但是当官方把取整置于不同层级时，以字节总量给出的 $T(p) \geq B_D(p)/B_D$ 才是更稳妥的容量界。本文在理论式中采用后者，并把独占工作量只当作局部代理输入。这样区分可避免把对官方舍入细节的假设写成不加条件的严格下界。

3.12 关键路径归因与重叠时间的去重

给定已经评估的时间线，取最后完成的输出事件 z ，沿使其开始时刻达到最大值的约束边逆行追溯。若 z 的前一事件完成于 t_1 、其跨核数据直到 t_2 才释放、其 Pipe 直到 t_3 才空闲，则其发射为 $\max\{t_1, t_2, t_3\}$ ，真正使其等待的只有达到最大值的约束；另外两个可能同时存在，却不是该次发射的边际原因。多个约束同时达到最大值时存在并列关键边，不能强制把同一等待周期完全分给某一个原因。为避免重叠时间多计，可为每段 $[u, v)$ 定义同时满足的状态指标：DDR 忙碌、相关任务未释放、目标 Pipe 空闲。三类指标会重叠，求和可能大于 $v - u$ 。只有在逐事件逆向追溯后，把关键链上唯一或并列的激活约束计入候选诊断，才能解释改善机会。

这一方法也限定了“减少跨核边”和“减少额外搬运”两类结构指标的用途。若一条边处于非关键分支，即使删去其 1000 周期释放规则，也可能不改变最终完成时间；若某个共享输入读入的流量完全与长时计算重叠，减少其 DDR 字节也可能使 D 降低而 T 不动。反之，减少关键输出前的一个很小搬运，可能使后续多个 Task 提前释放，最终节省远高于该条搬运的独占时间。局部候选选择因此考虑边是否与关键输出的祖先集合相交、是否与 DDR 拥塞窗口相交、改变后是否可能触发另一条路径成为关键路径，而不使用单一的全局

边权阈值决定接受。

设当前已实现的关键路径长度为 T_1 ，第二长路径为 T_2 ，且候选只缩短第一条路径 δ ，不改变其它路径和资源竞争。则新完成时间为 $T' = \max\{T_1 - \delta, T_2\}$ ，边际改善为

$$T_1 - T' = \min\{\delta, T_1 - T_2\}. \quad (18)$$

这是固定路径集合下的条件性推导，说明单个瓶颈的收益会在第二路径接管时饱和。真实方案改变会重构路径，式 (18) 仅用作排序和解释，不作为官方评价结果。它也解释为何反复围绕一条旧关键链做开环搜索，后期容易出现大量完全持平的候选；继续增大区域未必有效，可能只增加评价机会成本。

3.13 候选排序的可证范围与公平预算

一个候选池 $\mathcal{C} = \{p_1, \dots, p_M\}$ 中，代理给出 $\hat{T}_j$ ，官方评价给出 T_j 。若代理只能评价所有 M 个、官方最多评价 $B < M$ 个，选择器的实际损失取决于前 B 个覆盖了多少真优候选，而不是代理均方误差本身。令 $j^* = \arg \min_j T_j$ ，代理按升序排列索引为 $\hat{j}_1, \dots, \hat{j}_M$ ，可定义候选池内选择遗憾

$$R_B = \min_{1 \leq b \leq B} T_{\hat{j}_b} - T_{j^*} \geq 0. \quad (19)$$

当 $R_B = 0$ 时，前 B 覆盖了该池的官方最优；但这不表示全局最优已经找到。若同一指紋的方案重复占用多个名额，名义预算 B 与真正不同的 B_{unique} 不同，必须按唯一完整方案数登记。冻结同一核内准备与成本状态后，才能把多个候选的代理名次和官方名次相配对，分析误差来自 Task 释放、Pipe 路径还是共享 DDR；若在候选间随意重算准备状态，相关系数和遗憾的解释基础也消失。

公平对照使用相同的图、核数、代码版本、随机种子、官方评价次数和候选保底。若新增一个机制挤掉原来有效的候选，它的表观收益包含预算替代效应；若只给新机制额外评价机会，则结果又混入预算扩张效应。定义 $\mathcal{B}$ 为固定官方评价机会，基础搜索分配 B_0 ，结构候选分配 B_1 ，局部修复分配 B_2 ，需满足 $B_0 + B_1 + B_2 \leq \mathcal{B}$ 。在某一邻域连续无效时可以把尚未使用的 B_1 或 B_2 转给另一个，但必须保留基础搜索下限，且在日志记录实际采用的次数；否则同样的名义预算可能对应不同的有效探索。本文有限预算优化的目标是高质量地使用这 $\mathcal{B}$ 次官方调用，而不是把公式下界当成可直接提交的方案。

3.14 可行性、最优性与求解效率的三重区分

可行性由唯一覆盖、商图无环、场景同步规则和物理存储约束决定；性能由 $T_s(p)$ 和 $D_s(p)$ 决定；计算效率则由生成并评价 p 的主机耗时决定。这三者在论文中分别检验。一个有效地压缩了 CPU 搜索时间的候选缓存，不一定改变 NPU 执行周期；一个周期很小的代理预测方案，若未经过正式评价与内存检查，不能称为高质量可行结果；一个正式评价优于对照的方案，也不能据此推断全局最优。

对每个固定候选池 $\mathcal{C}$ ，官方评价中的最小值 $\min_{p \in \mathcal{C}} T_s(p)$ 只提供该池上的上界，即全局最优 T_s^* 满足 $T_s^* \leq \min_{p \in \mathcal{C}} T_s(p)$ 。通用计算下界提供相反方向的 $L_0(k) \leq T_s^*$ 。于是合法的、但一般较宽的夹逼关系为

$$L_0(k) \leq T_s^* \leq \min_{p \in \mathcal{C}} T_s(p). \quad (20)$$

这个区间明确表明已找到方案的质量边界，也避免把某个方案自身的搬运下界误用为全局下界。当 $L_0(k)$ 很低时，不能从宽区间推断还剩多少可实现提升；当某一大图距 $L_0(k)$ 很远时，也不能断言差距都可消除，因为同步、内存与带宽尚未进入 L_0 。因此本文以同口径官方结果、负例和结构诊断来评价启发式，而不虚构最优性证明。

4 问题一：无核间同步机制下的切图与调度

4.1 场景 A 的组合优化模型

令 V_c 为所有非 COPY 算子。映射 π 把它们恰好分配给 q 个非空子图， m 把每个子图分配给一个核心， σ_c 给出核心 c 上各子图的顺序。把跨子图的数据前驱构成商图，再加入每核顺序边，所得扩展商图必须无环；否则即使原图无环，核心顺序仍可能与跨核等待构成死锁。合法方案集合记为 $\mathcal{P}_A(k)$ 。对于每份方案，官方内层启发式先产生实际 COPY 和核内 Pipe 序，再进行共享 DDR 事件仿真。我们以此可计算但不便线性化的函数定义目标：

$$\min_{p \in \mathcal{P}_A(k)} \text{lex}(T_i^A(p), D_i(p)), \quad T_i^A(p) = \max_{u \in \mathcal{T}(p)} f_u. \quad (21)$$

这里的字典序表示优先缩短最后一个 Task 的完成时间；仅在周期相同时比较额外搬运。它是算法的选优规则，不把较少搬运误说成必然较短的周期。

Task v 的开始须满足所有数据前驱释放和同核前一 Task 的间隔。以 F_u 表示前驱完成时刻， $P_c(v)$ 表示核心 c 上的前一 Task，可写为

$$R_v \geq \max_{u \in \text{pred}(v)} (F_u + 1000 \mathbf{1}\{m_u \neq m_v\}), \quad R_v \geq F_{P_{m_v}(v)} + 100. \quad (22)$$

同核依赖不额外施加跨核的 1000 周期，但跨 Task 张量仍须由 DDR 中转。公式 (22) 仅描述释放约束，不能单独算出时长；Cube、Vector、搬运 Pipe 的先后关系、L1/UB 溢出以及同时在途的 DDR 传输会改变 F_u 。实际评价必须采用题目给出的事件过程。

为了识别一份输入被多个子图重复读取的情形，我们把共享张量与多个消费算子的关系视为有向超边，保留原图方向和商图无环性。多层次无环划分研究说明，此类表示比仅统计两两割边更适合描述共享输入^[2]。我们借鉴这一表示和粗化思想，但没有照搬其平衡目标或宣称实现了文献的完整算法；本题的最终目标仍是式 (21)。

4.2 场景 A 的物理语义与可行性

场景 A 中子图与 Task 一一对应，即 $\mathcal{T}(p) = \{V_1, \dots, V_q\}$ 。两个 Task 即使被映射到同一个核心，前者的私有 L1/UB 状态也不能跨 Task 保留；跨子图张量必须先写至 DDR，后继 Task 再读回。因此子图边界上的数据路径由 π 决定，核心映射 m 并不能把同核边界通信改成免费的片上读取。这一点与场景 B 是根本区别。核心 c 上按 σ_c 串行执行的 Task 之间另有固定的 100 周期切换间隔；跨核数据前驱给目标 Task 施加 1000 周期等待。若有多个前驱，目标 Task 等到最后一个约束满足即可，而非把所有前驱延迟相加。

将 Task r 的实际开始、完成记为 S_r, F_r ，其数据前驱集合为 $\text{Pred}_\pi(r)$ 。若 $a \prec_c r$ 表示核心 $c = m(r)$ 上排在 r 前面的最后一个 Task，释放规则可写为

$$S_r \geq \max_{a \in \text{Pred}_\pi(r)} \{F_a + \delta_{ar}\}, \quad \delta_{ar} = 1000 \mathbf{1}\{m(a) \neq m(r)\}, \quad (23)$$

$$S_r \geq F_{\text{prev}_{\sigma_c}(r)} + 100 \quad \text{若该前一 Task 存在.}$$

式 (23) 只描述外层 Task 的可发射时刻。某 Task 的 $F_r - S_r$ 并非常数：展开出的四条 Pipe 上的操作依赖、共享 DDR 竞争以及 L1/UB 压力都会影响它。优化器因此必须先给出合法 (π, m, σ) ，再调用官方核内与多核评价过程得到真实 F_r 。若仅用一个预先标定的“子图长度”代入普通列表调度，便会在两个候选改变 DDR 发射重叠时给出相同预测，却可能得到不同的正式完成时间。

四条 Pipe 分别执行矩阵运算、向量运算和两类搬运。用 $P(e)$ 表示事件 e 所属 Pipe，核内候选排程给每条 Pipe 一个确定顺序。对相同核心及相同 Pipe 的前后事件有 $f_{\text{prevPipe}(e)} \leq s_e$ ；跨 Pipe 只由真实算子依赖与内存复用依赖约束，不人为强制串行。对任一私有存储池 $r \in \{\text{L1}, \text{UB}\}$ ，展开后的物理驻留集合 $\mathcal{R}_{c,r}(t)$ 必须满足

$$\sum_{x \in \mathcal{R}_{c,r}(t)} b_x \leq C_r, \quad C_{\text{L1}} = 524288, \quad C_{\text{UB}} = 131072 \quad (\forall c, t). \quad (24)$$

如果未经换出换入就会越界，官方核内启发式会引入附加搬运来维持容量合法。因而式 (24) 约束的是实际展开的执行，不能把“候选静态张量大小总和未超限”误认为充分条件，也不能把溢出惩罚写成只由子图算子数量决定的固定函数。

4.3 额外搬运的计数口径

令 $B_{\text{all}}(p)$ 为官方展开方案在 DDR 上的总逻辑 COPY 字节数， B_{orig} 为未切图输入中已经要求的基础 COPY 字节数。题目所报的额外搬运定义为

$$D(p) = B_{\text{all}}(p) - B_{\text{orig}}. \quad (25)$$

这个定义覆盖了子图边界传递、原始输入的重复读取，以及核内启发式因容量不足插入的换入换出。正文始终用式 (25) 与官方输出核对，而不用简单割边权和代替它。设输入张量 x 被 $r_x(\pi)$ 个不同 Task 消费，则在独立 Task 都需要从 DDR 读入这一语义下，相对一次读取的重复输入字节可用

$$D_{\text{input}}^{\text{proxy}}(\pi) = \sum_{x \in \mathcal{X}_{\text{input}}} (r_x(\pi) - 1)_+ b_x \quad (26)$$

作静态代理。它反映多消费超边的割开程度：同一张量的三个消费者分处三个 Task 时，不宜只看三条两两边的权重；若三个消费者聚在一个 Task，代理重复读入为零。但式 (26) 只涉及外部输入的一个组成项，不能再把同一张量的边界读入重复加进另一个“割边项”以免双计。中间张量的写出次数、各目标 Task 的读取及真实溢出，由官方展开后的事件记录裁决。

减少 $D(p)$ 并不等价于减少 $T(p)$ 。反例是合并两个可并行子图：该动作可能删除边界 COPY，却使原来可分别运行的矩阵链和向量链被一个 Task 的内部顺序或容量压力拖慢。反过来，少量额外搬运若释放关键任务的并行空间，也可能缩短 Makespan。因此式 (21) 采用完成时间优先的字典序；并在结果章节同时报告 $D(p)$ 的变化，而不预设它必与主目标同向。

4.4 方案条件下的下界及其推导

共同计算下界 $L_0(k)$ 已由式 (10) 给出。场景 A 还允许对固定方案 p 构造更贴近其 Task 结构的事后下界。令 $W_{M,r} = \sum_{e \in V_r \cap O_M} c_e$ ， $W_{V,r} = \sum_{e \in V_r \cap O_V} c_e$ ；同一 Task 内这两类 Pipe 可以重叠，却各自不能在本 Pipe 上并发，因此

$$F_r - S_r \geq \tau_r := \max\{W_{M,r}, W_{V,r}\}. \quad (27)$$

若核心 c 上有 n_c 个 Task，则该核从第一个 Task 起始到最后一个 Task 完成至少占用 $\sum_{r:m(r)=c} \tau_r + 100(n_c - 1)_+$ 周期。令 $\mathcal{P}(H(p))$ 为扩展商图的路径集合，在路径 $(r_1, \dots, r_h)$ 上，每个任务的最短计算时间和依赖释放必须依次经过。因此

$$T(p) \geq L_A(p) := \max \left\{ L_0(k), \frac{B_D(p)}{60}, \max_c \left[\sum_{r:m(r)=c} \tau_r + 100(n_c - 1)_+ \right], \right. \\ \left. \max_{(r_1, \dots, r_h) \in \mathcal{P}(H(p))} \left[\sum_{j=1}^h \tau_{r_j} + \sum_{j=1}^{h-1} \underline{\delta}(r_j, r_{j+1}) \right] \right\}. \quad (28)$$

对路径中的数据依赖弧，跨核时取 $\underline{\delta} = 1000$ ，同核时至少为 100 的任务间隔；对纯顺序弧，只取 100。若一对任务既有数据依赖又有顺序关系，同一间隔仅计一次。式 (28) 的每项分别由计算工作、带宽容量、同核串行和依赖链推出，取最大而不是相加，避免把可重叠资源时间重复计算。

这个界是针对所选划分和映射的条件界。例如合并任务会改变 $B_D(p)$ 、 n_c 和路径集；把方案 p_1 的 $L_A(p_1)$ 当作方案 p_2 的最优下界没有依据。我们用它寻找当前方案的主导限制：若同核负载项远大于其他项，优先考虑拆分或迁核；若关键路径项主导，优先处理路

径上的边界与核心切换；若带宽项主导，检查重复输入及边界流量。这是一条有物理解释的候选分配规则，不是由下界直接求得最优划分。

4.5 合并与拆分的边际判断

考虑相邻子图 A, B 与一份大小为 b 的边界张量。若把它们合成同一 Task，理论上可删除这份边界上的部分 DDR 事件与释放等待；但这份“被删除的字节数”不一定等于 b 或 $2b$ ，因为还取决于共享消费者、输出写回以及已有 COPY。令 $\Delta B_{\text{boundary}}$ 为按场景 A 展开规则实际消失的边界字节， ΔB_{spill} 为合并后新增的溢出字节。一个仅用于排序的净搬运近似为

$$\widehat{\Delta B} = \Delta B_{\text{boundary}} - \Delta B_{\text{spill}}. \quad (29)$$

若合并删除一个实际落在关键链上的跨核 Task 边，还可能减少 1000 周期释放；若删除的是同核相邻 Task 边，最多直接省下该处 100 周期间隔。然而被删除的等待与传输可以重叠，也可能原本根本不控制最终输出，所以不能把 $\Delta B_{\text{boundary}}/60 + 1000$ 作为必得的 Makespan 改善。真正的差值为

$$\Delta T_{\text{merge}} = T^A(p_{\text{old}}) - T^A(p_{\text{merged}}), \quad (30)$$

只能在两个完整方案通过官方评估后确定。式 (29) 给出有依据的候选顺序，式 (30) 才给出是否接受的结论。拆分动作同理：它增加边界开销，却可能降低单核长尾与局部溢出。固定算子个数切块无法表达这种结构差异，故初始划分同时覆盖整分量与少量主导分量拆分。

4.6 结构感知初解与有界搜索

首先构造整图保底方案与依赖贪心方案。对没有计算依赖的弱连通分量，尝试保持完整并按估计的最长 Pipe 工作量分核；这样能释放天然并行度，且减少不必要的分量内切边。若一个主导分量明显压过其余分量，则在满足无环性的拓扑窗口中有限拆分，以免完整保留造成单核长尾。共享输入复用、局部核内时长和容量压力同时参与候选排序；局部估计只负责提出方案，不能替代官方全局仿真。

将候选子图调度到核心时，先按后续关键工作量排序，再为每个子图寻找最早合法插入空隙和预计完成时间较小的核心。该思想与最早完成时间列表调度相近^[3]，但本题必须附加式 (22)、共享 DDR 和真实容量条件。随后在固定的评价机会内，尝试少量关键区域的边界调整、核心迁移与顺序修复；每个被提出的完整方案均检查操作覆盖和扩展商图无环，并由原始评价语义选优。

当前采用方案对每个多核配置限制为 12 次评价机会、确定性种子 0。基础粒度搜索保留机会，结构新候选须通过事件成本准入；失败的候选不能因预测好就替代实测较优方案。当前研究版本启用了局部成本、关键路径、一般映射空隙插入、通信排序、划分保护、共享输入及有限区域修复。实验中测试过而未被采用的事件池整体重排序、扩大的自适应组合等，不计入本轮主结果。固定预算使评价调用数随图数线性增长；候选构造与图规模相关，但不枚举指数数量的分组、映射和任务排列。

4.7 初解生成的结构依据

求解先利用计算图的弱连通分量。两个互不相连的分量之间没有计算数据前驱；若把完整分量分别放到不同核心，不会产生分量间新的中间张量通信，因此它们是低成本的并行初解。设分量 C 的矩阵、向量工作量分别为 $W_M(C), W_V(C)$ ，并定义两管道负载向量 $\mathbf{w}(C) = (W_M(C), W_V(C))$ 。若某分量的某项工作量超过平均每核份额，

$$W_M(C) > \frac{W_M}{k} \quad \text{或} \quad W_V(C) > \frac{W_V}{k}, \quad (31)$$

则在“整个分量必须落在一个核心”的候选中，该核心相应 Pipe 至少要执行 $W_M(C)$ 或 $W_V(C)$ 周期，比理想均摊项更大。式 (31) 是考虑拆分的判据，不是“拆分必优”的证明；如果沿分量内部切断一条巨量共享张量，通信、同步或溢出可能使最终时间更差。实现只在有限拓扑窗口提出切分，并保留不拆的完整分量方案作对照。

分量分核用负载向量而非操作数量。对当前核心 c 的估计负载记 $\mathbf{L}_c = (L_{M,c}, L_{V,c})$ ；将分量 C 放到 c 后，一个用于初解的平衡指标为

$$\hat{L}_c(C) = \max\{L_{M,c} + W_M(C), L_{V,c} + W_V(C)\}. \quad (32)$$

优先选择能控制当前最大 **Pipe** 长尾的核心，再用共享输入字节数和依赖释放风险破同分。式 (32) 只近似双计算管道的最短可能占用，尚未计入搬运 **Pipe**、DDR 或生命周期。它比“每核平均分配相同算子数”更贴合计算时间，却不能单独决定最终方案。

另一类初解利用共享输入超边。对输入 x 的 Task 消费集合 $\mathcal{C}_x(\pi) = \{\pi(v) : v \in C(x)\}$ ，式 (26) 说明切到多个 Task 会增加读入机会。生成候选时，先根据超边交集和拓扑位置给相关操作形成小的局部簇，然后只有在商图无环且负载不过分集中时才尝试合并。这样让“共享输入节省”与“并行空间丢失”在候选池中同时可见。弱连通分量、拓扑窗口、共享输入簇对应不同图结构，保留多种候选能够降低单一固定粒度的偶然性。

4.8 关键工作量优先与合法空隙插入

固定一种划分后，给 Task r 一个向后看的关键度。以 $\underline{\tau}_r$ 为式 (27) 给出的最短计算时长，从商图汇点反向递推

$$\text{rank}(r) = \underline{\tau}_r + \max_{s \in \text{Succ}_\pi(r)} \{\text{rank}(s) + \underline{c}_{rs}\}, \quad \max \emptyset = 0. \quad (33)$$

其中 $\underline{c}_{rs}$ 可以取零作为保守结构排序，也可用经场景 A 展开规则估计的边界搬运与释放代价。 $\text{rank}(r)$ 大，表示推迟它更可能拖慢最终输出，因而在已就绪 Task 中优先处理。这是排序启发式，不把估计值误认为准确关键路径。Task 间序与核心映射仍由合法性检查和官方评价决定。

对就绪 Task r ，列表调度并非只能附加在某核队尾。核心 c 上已有按时间排序的 Task 区间 $[S_j, F_j]$ 。考虑相邻区间 $(j, j+1)$ 之间的空隙，候选插入位置须同时满足

$$S_r \geq F_j + 100, \quad F_r + 100 \leq S_{j+1}, \quad S_r \geq \max_{a \in \text{Pred}_\pi(r)} (F_a + \delta_{ar}). \quad (34)$$

还要检查插入后扩展商图仍无环。若放在队头或队尾，相应的单侧不等式删除。 F_r 在真实仿真前未知，调度器以局部成本估计做插入排序；被选中的完整方案最终由官方事件评价。与永远追加在队尾相比，合法空隙插入可把一个已经满足前驱的任务放进另一个任务等待 DDR 或跨核释放时留下的区间，而不改变已排 Task 的相对次序。若局部估计误差使其实际上不缩短完成时间，保底方案仍在。

候选核的比较遵循预计最早完成时间，但显式保留释放时刻和空隙位置。记 $\hat{d}_r(c)$ 为局部核内准备得到的任务时长代理， $\hat{S}_r(c, g)$ 为在核心 c 空隙 g 中满足式 (34) 的最早估计开始时刻，则

$$(c^*, g^*) \in \arg \min_{c, g \text{ legal}} [\hat{S}_r(c, g) + \hat{d}_r(c)]. \quad (35)$$

式 (35) 只产生一个或少量候选，不意味着真实全局完成时间等于该任务预计完成时间。多个任务共享 DDR；把一个任务提前可能推迟同时搬运的关键任务，所以每个候选在送官方评价前还要经过较完整的事件成本排序。

4.9 代理排序、机会成本与局部修复

在官方评估调用受限时，搜索对象不是全部合法方案，而是当前结构附近的有界候选池。代理评价固定一次成本状态，保留已准备的 **Pipe** 顺序、依赖和传输工作量；对各候选仅重放受到局部改变影响的释放与资源事件。设 $\hat{T}_\theta(p)$ 为在同一已冻结状态 θ 下的预测完成时间，比较候选 p_a, p_b 时必须使用同一 θ ：

$$p_a \prec_\theta p_b \iff \hat{T}_\theta(p_a) < \hat{T}_\theta(p_b). \quad (36)$$

若用候选 a 的旧资源占用预测 a ，却用另一套已更新的占用预测 b ，排名差异不能解释为方案差异。更重要的是 $\hat{T}_\theta$ 不是严格上界或下界；代理预测改善但官方变差的负例曾出现。

因此代理只决定哪个合法候选优先占用宝贵评价机会，不能直接替换已验证方案。对于不同划分但同核内准备键的候选，复用已确定的核内子问题结果以降低主机计算秒数；这种工程复用不改变 NPU 仿真周期。

局部修复选择少量相互关联的瓶颈 Task，允许调整它们的边界、核心和局部顺序。相关性由关键链邻接、共享张量和在同一 DDR 拥塞窗口内搬运共同确定。区域外保持原相对顺序，区域内仅取有限种分配和优先级，逐项过滤覆盖重复、商图环与资源非法；对完全相同的完整方案按划分、映射、顺序的规范序列去重。保留至少一个结构不同的合法候选，比重复评价同一方案更有信息价值。区域规模及提案数设置硬上限，其作用是搜索联合改变而非组合穷举。

每个多核配置的完整官方评价机会封顶为 $B = 12$ 。基本划分与映射搜索先保留一部分机会，新初解和区域修复只能在剩余机会内竞争；同一指纹的重复方案不应假装是一次新探索。官方返回 (T, D) 后，采用规则为

$$p_{\text{best}} \leftarrow \arg \min_{p \in \{p_{\text{best}}, p_{\text{new}}\}} \text{lex}\{T^A(p), D(p)\}, \quad (37)$$

因此在一个配置内已评价方案的最优记录不会因一次失败提案而变差。相对上一轮不同完整算法版本的少数退步仍可能存在：版本间初解、排序、预算占用发生变化，新的有限候选池不一定包含旧版最优解。不能把式 (37) 误读为跨版本、跨核数或跨场景单调性的证明。

4.10 求解过程与复杂度边界

可将上述过程写为如下可复核的有限步骤：

1. 从原图生成整图保底、依赖贪心和完整分量等合法初解；对式 (31) 触发的分量有限地产生拓扑切分。
2. 对每种划分用式 (33) 排序就绪任务，按式 (34)、(35) 选取核心及合法空隙，得到完整 (π, m, σ) 。
3. 扩展商图做无环检查，并检查唯一覆盖与基本结构；对合法且不重复方案，用相同成本状态的代理排序。
4. 在至多 B 次完整官方评价内，轮流分配基础搜索与有限区域修复机会；每次按式 (37) 更新最优实测方案。
5. 输出每个算子子图编号与每核顺序，最终再作独立官方回放和逐例指标核对。

拓扑排序和无环检查对一个固定候选可在线性于商图节点、边数的时间完成；候选生成次数由结构档数、局部区域上限和评价预算限制。官方核内排程及事件仿真仍可能随大图、缓存溢出和同时在途搬运而耗时，故“组合搜索次数有界”并不自动保证每个大图从冷启动到最终复核都在十分钟内。本文所报最长约 364 秒仅对应已记录的热搜索阶段，求解器墙钟秒数与所输出方案的 NPU 执行周期必须分别解释。

4.11 超边流量、串并行收益与划分尺度

进一步说明为什么本问以共享张量超边而非普通边数参与初解。设一个输入张量 x 大小 b_x ，有三个消费算子 u_1, u_2, u_3 。若三者处于同一 Task，题目数据路径至多需要该 Task 按核内策略读取该张量；若分在三个 Task，场景 A 的 Task 边界不保留私有缓存，至少有三次独立的消费机会。相对共同读取一次，静态重复输入代理由式 (26) 给出 $2b_x$ 。但若用源到每个消费者的普通投影边表示，同一个张量可被计为三条边，简单地给每条边一个 b_x 权重会得到 $3b_x$ ，和“重复读取增量”不是同一量；若再把跨子图读入 $3b_x$ 叠加到 $2b_x$ ，甚至重复统计。对内部张量又有生产侧写出和消费侧读入，场景 A 的真实开销取决于官方构图细节，不能机械套用输入张量的 $r_x - 1$ 公式。因此超图权重只用于优先保留共享消费者，不作为正式 D 的替代。

对于分量 C ，可定义用于结构选择的三维轮廓

$$\mathbf{q}(C) = (W_M(C), W_V(C), R(C)), \quad R(C) = \sum_{x: C(x) \cap C \neq \emptyset} b_x, \quad (38)$$

其中 $R(C)$ 只表示潜在输入集合的字节规模。两分量没有原图依赖，分核不增加彼此的中间张量通信，但若它们共同消费原始输入，分到不同 Task 仍可能重复读入。这说明“弱连通分量完全独立”只能用于数据依赖和中间张量边的判断，不能宣称总输入搬运独立。轮廓中矩阵、向量工作量用于估计单核的两 Pipe 长尾； $R(C)$ 用于提醒共享输入代价。若只按 $W_M + W_V$ 做降序装箱，可能把一个 Cube 密集分量与另一 Cube 密集分量塞入同核，而让 Vector Pipe 和其它核心闲置。按式 (32) 比较添加后 $\max\{L_M, L_V\}$ ，可以把互补的 Cube/Vector 分量放在同核，利用两个计算 Pipe 的同时执行能力。

为了识别值得拆的主导分量，还要区分理论可并行余量和实际可拆边界。设完整分量 C 的算子投影图在某拓扑位置切成 C_1, C_2 ，其理想负载下界从 $\max\{W_M(C), W_V(C)\}$ 可能降为 $\max\{\max_j W_M(C_j), \max_j W_V(C_j)\}$ 。令这两个数的差为 G_{pipe} 。切开后需要传递的超边集合记为 $\partial(C_1, C_2)$ ，其潜在 DDR 工作量至少与其中需要跨 Task 的不同张量有关。 G_{pipe} 是释放并行的潜力，不是预计 Makespan 的真实节省；边界流量、100/1000 释放及局部溢出可抵消它。本文只在 G_{pipe} 足以提出合理候选、且商图保持无环时占用有限评价机会；如果整分量方案更快，保底机制选择整分量。由此，固定“每八算子一块”的规则被降为多种候选中的一类，而不作为普适划分尺度。

4.12 边界移动的局部合法性与非局部代价

设当前 $u \in V_a$ ，考虑把 u 移到相邻子图 V_b 。这一小动作需满足四层检查。第一， $V_a \setminus \{u\}$ 非空，且每个算子仍出现一次。第二，新的商图与核心顺序弧的并图 $H(p')$ 无环。第三，移动后的所有跨边数据都可按场景 A 插入合法的写出和读入事件，核内存储压力经官方排序处理后不违约。第四，候选必须在相同原图和同一核数下重新评价。前两项能用拓扑排序廉价地判定，后两项需要实际展开与仿真，不能仅凭原图局部邻接完成。尤其当 u 是共享张量的生产者时，它的移动可能改变多个消费 Task 的 COPY 结构；看似局部的划分变更可对多核 DDR 在途集合产生非局部影响。

可用集合差而非边计数，预估移动后的输入超边变化。记 $R_x(p) = \{\pi(v) : v \in C(x)\}$ 为张量 x 的消费者 Task 集合，移动 u 只可能改变包含 u 的那些 R_x 。代理差量为

$$\Delta \hat{D}_{\text{input}}(u : a \rightarrow b) = \sum_{x: u \in C(x)} b_x \left[(|R_x(p')| - 1)_+ - (|R_x(p)| - 1)_+ \right]. \quad (39)$$

若 a 中还有其它消费者而 b 中已有消费者，则可能 $|R_x|$ 不变，移动算子并不会改变该输入的重复读取代理；若 u 是 a 中唯一消费者而 b 中已有消费者，则 $|R_x|$ 减一。此增量可增量更新，因而适合从庞大候选集中筛出少量变体。但它仍不包含生产中间张量的写回、Pipe 排程和缓存溢出，所以不能代替式 (25)。

4.13 输出关键链与核心空闲的区别

令 $\mathcal{A}(z)$ 为最终输出 z 的算子祖先集合；若有多个输出，取最后完成的输出的祖先。某个算子或 Task 不在 $\mathcal{A}(z)$ 上，缩短它本身通常不能直接缩短当前实现时间，除非它在共享 Pipe 或 DDR 上抢占了关键祖先所需资源。这样可把候选分为两类：直接修改关键依赖路径的边界，或修改争用资源以间接帮助关键路径。单纯数核心忙碌百分比不能区分两类：一个核心全时忙碌但只做早早结束的非关键分支，与一个核心短时工作却在输出前形成最后释放点，对 Makespan 的贡献可能相反。因此我们同时观察最长 Task 链、跨核切换、Pipe 工作量和共享 DDR 时窗。

若某图有一个核心长期空闲，也不能简单强制把任务迁到它。考虑迁移一个 Task r 的估计净效应：释放原核心的计算负载 ΔC_{old} ，增加新核心的负载 ΔC_{new} ，并改变进入与离开 r 的 Task 边跨核状态。即使 $\Delta C_{\text{new}} = 0$ 、原核上有明显尾部队列，新增的关键链跨核释放仍可能使输出晚于原方案。一个必要的结构检查是：迁移后所有受影响的前驱和后继能否在合法顺序中释放；一个性能检查是完整仿真得到的 $T(p') < T(p)$ 。计算空闲只是候选触发信号，而不是接受条件。对问题一尤其如此，因为同核 Task 边界本就经 DDR，不存在“迁到同核就免传输”的捷径；迁核主要改变释放等待和并行排程。

4.14 多核加速比的两个上限直觉及其局限

若把全部计算视作一条完全可分的单类 **Pipe**，理想速度增益似乎至多为核数 k ；但题目以启发式整图单核结果作分母，该基准可能因整图规模与缓存压力而不是理论最优，故实测比值超过 k 不构成逻辑错误。另一方面，若只从固定计算链看， L_{op} 给出无论核数多少都无法突破的时间下界，因此对某个方案的参考单核时间 $T_{i,1}^0$ 有形式上的上限 $S_{i,k} \leq T_{i,1}^0/L_{op}$ 。该上限往往很松，因为没有算入同步、DDR 和容量。本文不拿它预测成绩，只用来说明当关键串行链占主导时增加核心不可能无限加速，以及为何需要在分量和关键链之间平衡切图。

更实际的平均指标也不能由“每个图最优地利用五核”推断。若图 i 的 $T_{i,1}^0$ 很小、受固定 Task 释放主导，则 $S_{i,5}$ 可能低于其它图；若图 j 的整图核内溢出异常大，切成多核后即使新增通信仍可能得到超过五倍的比值。式 (12) 给每张图相同权重，要求算法在不同结构之间稳定，而不是只针对总周期占比最大的几张大图取得很好的成绩。我们在报告中并列列出大图周期贡献与百图逐图均值，目的不是制造两个“官方分数”，而是说明总目标与统计呈现的不同关注点。

4.15 百图结果与解释

表 5 和图 1 给出固定单核分母下的百图结果。五核的逐图平均加速比为 3.665903，总执行周期为 73 170 501；后二者分别衡量“每图等权速度”和“这批图的总工作量”，不可相互替代。二至五核的总周期随核数降低，但加速不是线性的：更多核心既释放并行，也可能增加 DDR 读取和跨核等待。

表 5 问题一当前方案的百图汇总

核心数	平均加速比	总周期 (cycles)	总额外搬运 (bytes)
1	1.000000	312 479 373	1 241 686 120
2	1.834917	158 746 394	1 972 107 414
3	2.546238	109 880 862	1 848 970 300
4	3.161009	86 330 529	1 764 926 118
5	3.665903	73 170 501	1 767 728 284

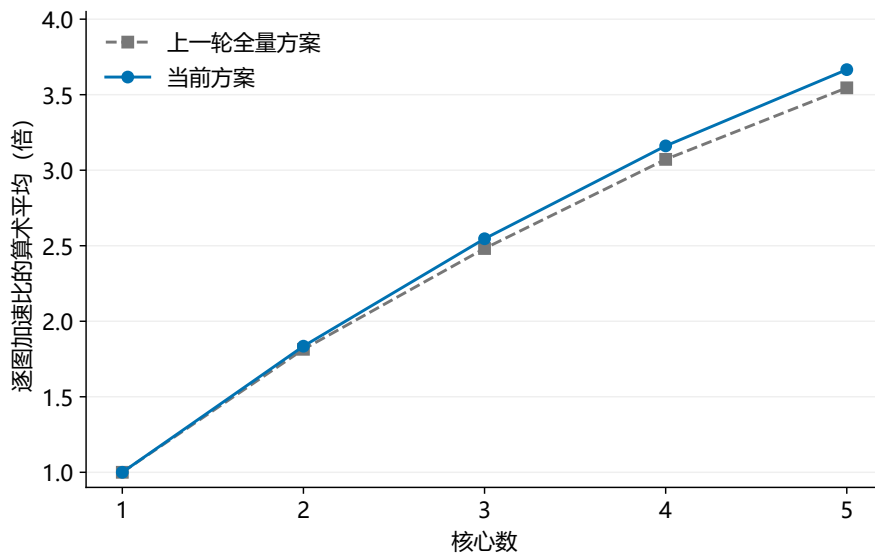


图 1 问题一的百图逐图加速比算术平均

与本工程上一轮相同百图方案相比，二至五核总周期分别下降 0.9958%、1.6807%、

2.7519%、3.6678%；400 个多核配置中 68 胜、330 平、2 负。负例是第 6 图的三核和四核，不能把各阶段收益简单相加，或说每张图都改善。第 59 图在二、四、五核取得本批次最大的逐图加速比，三核最大值来自第 97 图；这些仅为分布端点，不替代百图均值。由于单核分母本身是题目规定的核内启发式而非全局最优，少数多核比值超过核数可以出现，不构成评价错误。

五百份方案均通过官方结构及顺序校验，逐行指标与保存的官方评价相符；代表图还做了冷启动复现。大图采用增量无环检查、状态索引和已固定准备的复用，以减少求解器自身时间；这些工程措施不应被解释成 NPU 任务周期的额外收益。当前热基准搜索最长约 364 秒，但它不含所有配置的冷单核生成和额外最终复核，因此不能断言从零开始每图均在十分钟内结束。完整逐例周期与搬运见附录。

4.16 平均加速、总周期与执行效率的不同视角

表 5 中的平均加速比按式 (12) 逐图等权统计。为了评价这批图的总体执行负担，表 6 另把单核总周期 312 479 373 作为分母，列出合计周期缩减率与总周期比。数值均直接由同一 500 行结果表计算，不引入另一批算例。五核总周期较单核下降 76.5839%，其倒数形式的总周期比为 4.270565；规定的逐图平均加速比却为 3.665903。差异并非评估矛盾，而是式 (14) 的权重差异。论文主图仍采用规定的逐图等权均值，总周期比只用于说明大图的绝对时延贡献。并行效率定义为 $\eta_k = \bar{S}_k/k$ ，只是把相对核数的等权加速程度标准化。五核时

表 6 问题一百图总周期视角与逐图等权视角

核数	总周期比	总周期下降率	平均加速比	平均并行效率
1	1.000000	0.0000%	1.000000	1.000000
2	1.968419	49.1978%	1.834917	0.917459
3	2.843802	64.8358%	2.546238	0.848746
4	3.619570	72.3724%	3.161009	0.790252
5	4.270565	76.5839%	3.665903	0.733181

$\eta_5 = 0.733181$ 并不意味着其余 26.6819% 的周期都由 DDR 独自造成；关键链、Pipe 不均、释放等待和核内溢出都可能参与。任务执行时间越短是首要目标，不能为了让 η_k 的比值好看而故意牺牲 Makespan。图 2 展示合计周期随核数的实际下降，与图 1 共同报告两个互补视角。

4.17 典型结果与反例

第 59 图的单核参考为 829 473 周期，五核方案为 150 782 周期，逐图加速比约 5.5011；其额外搬运则从单核 1 691 648 增至五核 4 325 376 字节。这个例子说明完成时间改善可以伴随更多逻辑搬运，也说明超过五倍的比值并不等于突破硬件计算吞吐上限。赛题的单核分母来自固定的核内启发式，不是单核全局最优；多核切图改变核内问题规模和溢出行为，可能使官方启发式在五核下比把原单核结果简单除以五更有效。第 97 图单核 11 491 509、三核 3 509 566 周期，加速比约 3.2743，是三核配置的分布端点之一。端点代表有利结构，不代表百图平均水平，也不能据此承诺每张图接近理想线性加速。

第 6 图呈现相反的警示：三核 32 925 周期、四核 31 129 周期、五核 31 729 周期，五核比四核慢。其核数、划分与映射共同改变，不能仅凭这三个总数断言“多一核导致 DDR 拥塞”；需结合时间线中的关键依赖与在途传输核查。该图在前后算法版本比较中三核和四核亦是两处退步，说明有限预算下的新候选机制可能挤掉旧版有价值的候选。故当前选优只保证同一候选池内不接受实测变差方案，跨版本必须再做同图同核配置对照。

4.18 额外搬运与大图求解代价

二核百图总额外搬运量为 1 972 107 414 字节，高于三核、四核和五核的合计值；五核也略高于四核。这一序列不单调，表明“核数增加必然增加割边搬运”过于简单。算例的实际

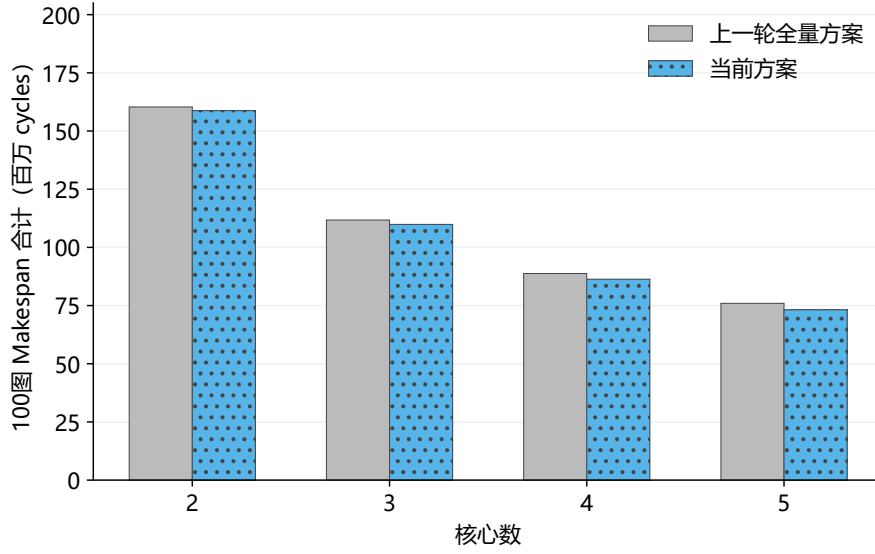


图 2 问题一百图总执行周期随核心数的变化

划分粒度、完整分量分配、共享输入重复读取和核内溢出都随核数变动；如果把所有差值都归咎于跨核边数，便忽略了场景 A 下同核跨 Task 仍须 DDR 的规则。对本文采用方案，搬运量是次级实测指标，不能由算子数或切边数推算为附录最终值。

大图还有另一种与 NPU 周期不同的成本：主机求解墙钟时间。官方核内启发式、合法性检查和全局事件评估对大型图更贵，若每生成一个局部候选就重新准备全部核内任务，求解过程会浪费大部分时间。对已有结构使用哈希键复用只读准备结果、对商图改变区域增量检查无环，减少的是找出方案的主机计算；它不改变候选的官方 $T^A(p)$ 。本轮 100 图多核热搜索记录的最长时长约 363.859 秒，含义仅限该阶段；冷启动单核基准和独立最终回放需另外计时。因此本文没有把“每张图端到端 5–10 分钟”作为已通过的全量实测结论。

5 问题二：同核驻留与跨核同步下的调度

5.1 场景 B 的模型扩展

问题二仍选择 (π, m, σ) ，但场景 B 不再把每个子图独立清空为一个 Task：映射到同一核心的全部子图组成该核的 Task，核内合法顺序决定缓存复用与张量生命周期。对于同一张量，若生产和消费能够在同核满足依赖，则中间结果可以驻留；跨核消费必须按实际生产核与消费核构造搬运，并在源 COPY_OUT 完成后经过 500 cycles 才能释放目标 COPY_IN。对于跨核数据事件 e ，有

$$a_e^{\text{in}} \geq f_e^{\text{out}} + 500. \quad (40)$$

这里的前驱是具体搬运事件，而非“源子图全部运行完”的粗粒度时刻。对多消费者共享张量，外部输入按消费核心而非每个算子独立读取；跨核中间张量按实际源核—目标核关系生成，避免把每条逻辑边简单等同一次物理 COPY。

合法集合 $\mathcal{P}_B(k)$ 除保留覆盖、子图商图和核上顺序的无环性外，还要求由官方核内展开得到的物理实例满足 L1/UB 在每个时刻的容量约束。四条 Pipe 各自串行，但不同 Pipe 在依赖允许时可重叠；同一 DDR 池服务所有在途外存搬运。张量能否驻留依赖实际产生、消费和溢出时刻，不能只按静态张量大小之和判断。令 $\mathcal{E}_B$ 表示题目的事件模拟，问题二的目标为

$$\min_{p \in \mathcal{P}_B(k)} \text{lex} (T_i^B(p), D_i(p)), \quad T_i^B(p) = \mathcal{E}_B(G_i, p; k). \quad (41)$$

这仍是依赖官方事件过程的组合优化模型，不把启发式估价器当作精确求解的线性目标。

与问题一相比，映射会同时改变通信、驻留、释放和并行，所以切图后固定映射、或仅按分组时长排序，均可能错过有效方案。处理器互连与片上存储的耦合也是相关体系结构研究关注的问题^[4]；本题仍以给定的缓存及 COPY 语义为准，不移植其他硬件的通信规则。

5.2 从独立 Task 到每核联合 Task

场景 B 保留同一个划分 π 、核心映射 m 和核上顺序 σ 决策，但改变了这些决策的物理含义。对每个核心 c ，把映射到 c 的全部子图按 σ_c 展开到一个 Task 的内部：子图之间仍有次序，然而私有 L1/UB 状态在这段次序中连续存在。若生产与最后消费同在 c ，中间张量可在容量允许时直接驻留；外层切出两个子图并不自动产生一次 DDR 往返。相反，两个核心之间的依赖须经源端写出、目标端读入与同步释放。由此，场景 A 的“每跨一个子图边界就增加传输”在 B 中不再成立，决策的风险从 Task 边数转向跨核心的逻辑张量集合与同核生命周期长度。

记 $c_p(x)$ 为内部张量 x 的生产核心， $\mathcal{C}(x; p) = \{m(\pi(v)) : v \in C(x)\}$ 为消费核心的去重集合。目标核心数

$$n_{\text{remote}}(x; p) = |\mathcal{C}(x; p) \setminus \{c_p(x)\}| \quad (42)$$

给出跨核心目的地的数目，而不是跨核算子边条数。若同一核心有三个消费算子，集合里仍只出现一次。对于原始外部输入，不存在内部生产核心；其各消费核心可共享一次该核心的加载机会。这个去重关系用于静态候选估价，但是否产生源端一次或多次 COPY、原图已经有多少基础 COPY、是否因容量溢出再读，均由官方 Task 构造及核内展开确定。因此问题二沿用

$$D^B(p) = B_{\text{all}}^B(p) - B_{\text{orig}}, \quad (43)$$

作为可交付的额外搬运定义；不直接把 $b_x n_{\text{remote}}(x; p)$ 宣称为总额外字节。对按边计费的朴素代理，令 $\mathcal{E}_x^{\text{remote}}$ 为张量 x 的跨核消费者边集合，则目标核心去重的读入字节代理满足

$$b_x n_{\text{remote}}(x; p) \leq b_x |\mathcal{E}_x^{\text{remote}}|. \quad (44)$$

不等号可严格成立，说明朴素边权和可能高估共享消费者的通信，从而错误惩罚本应考虑的划分。

5.3 跨核释放与事件路径

对一个源核心 a 到目标核心 b 的张量传递，设 e^{out} 为源端对应 COPY_OUT， e^{in} 为目标端 COPY_IN。在场景 B，后者的发射条件是

$$s_{e^{\text{in}}} \geq f_{e^{\text{out}}} + \delta_B, \quad \delta_B = 500 \text{ cycles}. \quad (45)$$

等待从实际写出完成开始计，不是从源子图的名义结束开始计。若源核还有与这份张量无关的矩阵计算，这段计算可能在写出完成后继续；目标核只需等待它自身需要的 COPY 就绪，而不是盲等源核整个 Task 完成。一个目标操作若依赖多个数据来源，其就绪时刻取相关发射与完成约束的最大值。把多条前驱的 500 周期全部相加，会夸大并行等待；把所有源 Task 一起拉长 500，则会错误阻塞无关 Pipe。表 2 所示三个场景在此处产生的差别，必须在事件层而非仅在子图时长层表达。

场景 B 的完整事件依赖包括：原算子数据边、各 Pipe 的顺序边、跨核 COPY 写出到读入的释放边，以及官方核内启发式插入的内存复用和换入换出边。若把所有源到汇路径记为 $\Gamma_B(p)$ ，则在真实事件服务时长确定后仍可使用式 (8) 解释总时间；但搜索前无法仅由子图大小求得这些路径。特别地，跨核释放 δ_B 可以与另一核心的计算并行，只有位于决定最终输出的路径时才直接增加 Makespan。

5.4 真实物理驻留与容量约束

令 $\mathcal{R}_{c,r}^{\text{phys}}(t)$ 表示时刻 t 实际驻留于核心 c 、私有池 r 的张量实例。场景 B 与 A 一样必须满足

$$\sum_{x \in \mathcal{R}_{c,r}^{\text{phys}}(t)} b_x \leq C_r, \quad r \in \{\text{L1}, \text{UB}\}, \quad \forall c, t. \quad (46)$$

不同的是，场景 B 的同核子图共享一段 Task 生命周期。为了估计不溢出时的压力，可定义某张量在核心 c 的理想驻留区间 $I_{x,c} = [t_{x,c}^{\text{first}}, t_{x,c}^{\text{last}}]$ ，并定义同时存活字节

$$L_{c,r}^{\text{ideal}}(t) = \sum_{x: x \text{ 属池 } r} b_x \mathbf{1}\{t \in I_{x,c}\}. \quad (47)$$

如果 $L_{c,r}^{\text{ideal}}(t) > C_r$ ，这份完全驻留的理想日程不可能直接执行；核内调度必须改变操作顺序、及时释放，或把部分张量换出并在再次使用时换入。反过来， $L_{c,r}^{\text{ideal}}(t) \leq C_r$ 也不足以断言不会有搬运开销：原始输入读入、输出写回、Pipe 冲突和启发式实际排程仍在。式 (47) 是压力诊断而非赛题评价器的替身。

若同核聚集更多子图，跨核消费核心集合可能缩小，式 (42) 的通信代理下降；但某些中间张量的 $t^{\text{last}} - t^{\text{first}}$ 可能变长，驻留峰值上升，触发换出换入。这正是“合并后反而更慢”的一种可检验机制。以生命周期面积

$$A_{c,r} = \int_0^{T(p)} L_{c,r}^{\text{ideal}}(t) dt \quad (48)$$

描述整体占用强度有助于排序候选，但它不能取代峰值约束 (46)：相同的面积可能由短时高峰或长时低占用形成，前者更容易引发容量违规。最终物理实例仍以官方核内输出和全局内存审计为准。

5.5 场景 B 的严格下界与局部判断

式 (10) 对 B 继续成立。对固定方案 p ，将所有源到汇的算子依赖链记为 Γ_c 。链 γ 上的固定计算周期和为 $C(\gamma)$ ，穿越不同核心的依赖次数为 $n_\times(\gamma; p)$ 。每次跨核依赖须经过式 (45) 的 500 周期释放，且链上先后发生，故有条件下界

$$T^B(p) \geq L_{\text{sync}}(p) := \max_{\gamma \in \Gamma_c} \{C(\gamma) + 500 n_\times(\gamma; p)\}. \quad (49)$$

这仍忽略写出、读入本身的服务与共享带宽竞争，因此是保守界。多个平行分支上的同步不能相加，但沿同一条连续路径发生的释放可以累积。结合实际展开 DDR 字节数得到

$$T^B(p) \geq \max \{L_0(k), L_{\text{sync}}(p), B_D^B(p)/60\}. \quad (50)$$

后两项随划分和映射变化，不能用于声称“当前方案离全局最优只差某百分比”。它们可用于判别：如果 $L_{\text{sync}}(p)$ 接近实测时间，优先消除链上核心切换；如果带宽项接近实测时间，优先检查多核心重复读取和在途拥塞。与问题一相比，跨核同步而非每个子图边界成为关键链的主要显式释放代价。

考虑移动一个边界子图 S 至相邻核心。该动作可能同时改变三项： S 的生产张量跨核目标数、消费张量的读入核心集合，以及本核张量的最后使用时刻。令 Δ_{comm} 、 Δ_{live} 、 Δ_{crit} 分别表示由这些因素构成的代理变化，则可用

$$\hat{\Delta}(S, c) = \alpha \hat{\Delta}_{\text{crit}} + \beta \hat{\Delta}_{\text{comm}} + \gamma \hat{\Delta}_{\text{live}} \quad (51)$$

为有限候选排序；权重不是赛题的真实目标系数，也不用于公布收益。对任意迁移、合并或拆分，真实判别始终是

$$\Delta T = T^B(p_{\text{old}}) - T^B(p_{\text{new}}). \quad (52)$$

尤其不能从“删除 b 字节跨核搬运与一次同步”推断必节省 $b/60 + 500$ 周期：传输可能原本与别的计算重叠，迁移可能令另一条链成为新的瓶颈，或推高 L1/UB 溢出。式 (51) 说明为什么值得提出该动作；式 (52) 才决定是否采用。

5.6 继承初解与有限联合调整

问题二的每份初始方案先以场景 B 官方规则复评，不能沿用场景 A 的周期作为成绩。求解器保留完整分量、共享输入顺序、主导分量切分、链及深度窗口等结构候选，并对场景 B 重新估计同核驻留和跨核传输。候选的核心及顺序安排借鉴依赖任务的最早完成时间框架，但处理重叠 Pipe、共享 DDR 和式 (40) 时，代理只能用于排序，最终比较仍由完整官方评价完成。

在结构初解之后，普通有界邻域把少数相关子图迁移到另一核心，并在合法位置插入；也可在不改变主要结构的前提下修复局部顺序。这样同时改变“任务在哪里执行”和“何时可以开始”，而不把所有 k^q 种映射或各核全排列列举出来。当前采用的规则在每个多核配置中保留已有效的基础候选，回收重复候选占用的评价机会，总场景 B 机会仍为 12 次。对于主导计算分量明显的图，结构准入开放相应候选；未达到结构条件时维持原有基础机会。该统一规则在全部百图上执行，不在看到某图结果后另换一套策略。

初始问题一方案只是合法起点，问题二另有自己的评价与改进。搜索中所有完整候选先做图结构、核上顺序与内存合法性检查；重复方案计入机会台账，不伪装成额外自由搜索。已有方案作为保底，后续方案仅按实测周期优先、同周期较少逻辑搬运的规则接受。研究期间测试过关键链引导邻域、通信代理改写和生命周期面积排序，但它们并未全部进入本轮采用配置，因此不能分别给这些未采用机制分摊收益。

5.7 场景 B 初解的重新构造

场景 A 已找到的 (π, m, σ) 可以作为 B 的一个结构种子，却不能把 $T^A(p)$ 搬进 B 的候选排名。变化不只在 1000 与 500 的数字：Task 组成、同核缓存是否保留、跨核 COPY 的生成单位、释放时刻都变了。对每张图、每个多核配置，先在 B 的官方规则下评价移植方案，同时保留整图单 Task 的合法备选；然后用 B 的成本模型重新把分量、共享输入链及深度窗口等候选映射到核心。这样的初始化保留问题一划分可能带来的结构价值，也让 B 专有的同核复用可以重新改变排名。

对弱连通分量整体分核，场景 B 的收益既来自分量间天然无依赖，也来自把同一分量内的相关张量留在一个核。对超过平均 Pipe 份额的主导分量，尝试沿拓扑顺序有限切开，恢复可并行空间。共享输入候选则把同一张量的消费者尽量安排在较少核心，使式 (42) 的目的地数下降。长链与深度窗口是另一种结构视角：前者保护可能控制最终输出的连续依赖，后者在近似同层的算子间寻找并行。所有结构都只是初解来源，没有一种可单凭静态图保证完成时间更短。

已采用的结构准入依赖一个可复核指标：最大弱连通分量的算子数占比

$$\chi(G) = \frac{\max_{C \in \text{WCC}(G)} |C|}{|\mathcal{O}_c|}. \quad (53)$$

当 $\chi(G) > 0.5$ 时，允许对候选菜单末段采用针对主导分量的另一种有界布局；当其不超过该阈值，保持原基础菜单。阈值并非物理常数，也没有被解释为通用最优切分点；它是全量实验前固定的结构门控规则。更关键的是，前部已有效的初解机会保持不变，新增布局只使用原菜单中预留的后部机会。若某个布局与移植种子、整图方案或前部布局完全相同，依据完整方案指纹识别重复，并用一个结构不同的旧候选补回该机会；没有把重复评价算作新的发现，也没有增加总预算。

5.8 联合映射和有界普通邻域

在相同划分下改变一个子图的核心，会同时改变跨核目的地集合、同核 Task 内的张量生命周期、跨核 COPY 释放以及相邻子图的资源排序；因此映射不能在划分之后靠“每核任务数相同”一次性决定。构造初解时，对就绪子图 r 和候选核心 c ，用场景 B 的释放估计

$\hat{R}_B(r, c)$ 与核内负载/驻留估计 $\hat{d}_B(r, c)$ 形成

$$\hat{F}_B(r, c) = \hat{R}_B(r, c) + \hat{d}_B(r, c), \quad \hat{R}_B(r, c) = \max \left\{ \hat{F}_{\text{same core}}, \max_{a \in \text{Pred}(r)} (\hat{F}_a + \hat{c}_{ar}(c)) \right\}. \quad (54)$$

这里 $\hat{c}_{ar}$ 对同核数据采用驻留代理，对跨核数据采用按消费核心去重后的传输和同步代理。它不把 500 周期加到所有前驱上；同一时间可并行完成的等待取最大。 $\hat{F}_B$ 的用途是按最早预计完成的方向生成候选；正式 B 评价会重新展开 COPY、Pipe 和物理缓存。

基础结构评价结束后，求解保留至多四个普通联合邻域机会。对当前实测较优方案，在一个小关联区域中选子图 r 、目标核心 c 及合法插入位置 g ，形成

$$\mathcal{N}_J(p) = \{\text{RepairOrder}(\text{MoveCore}(p, r, c), g) : r \in \mathcal{R}(p), c \neq m(r), g \in \mathcal{G}_{r,c}\}. \quad (55)$$

$\mathcal{R}(p)$ 是受预算和结构规则限制的少数相关子图， $\mathcal{G}_{r,c}$ 是不会破坏核上顺序与数据依赖的有限位置集合。RepairOrder 必须在迁核后同步修复被影响任务的局部顺序，否则孤立迁核可能只增加等待而看不到联合动作的价值。实现从这个邻域提出有限不同候选，逐个做覆盖和扩展商图无环检查，再由官方 B 仿真裁决。式 (55) 定义了可理解的邻域形状，不表示对其中所有组合穷举。

算法在每个多核配置保留十二个候选评价位置：移植方案和整图保底、六个结构候选以及四个有界普通联合动作。重复方案可由带输入、配置、评价代码身份的缓存直接复用相同官方结果，但它在台账中仍占原位置，只有明确定义的重复候选补位才改变菜单的有效覆盖数；不能因缓存命中就在对照组偷偷增加评价机会。最终胜出方案再由原始 B 评价过程独立回放，检查完整结果而不仅比较 Makespan。这个固定流程使研究中临时尝试的关键链邻域、替代通信代理或生命周期排序不被误写成所交成绩的组成部分。

5.9 搜索正确性和代价控制

只要移植或整图保底中至少有一个合法方案，逐项过滤并以官方结果更新 incumbent 的算法返回合法方案。其形式化性质是：设 $\mathcal{A}_B$ 为本次实际完成官方 B 评价的合法候选集合，返回 p_B^* 满足

$$p_B^* \in \mathcal{A}_B, \quad \text{lex}(T^B(p_B^*), D^B(p_B^*)) = \min_{p \in \mathcal{A}_B} \text{lex}(T^B(p), D^B(p)). \quad (56)$$

证明直接由每次把新候选与已有最优实测方案作字典序比较的归纳过程得到。该性质仅限实际评价的有限候选集，不能扩展为 $\mathcal{P}_B(k)$ 的全局最优。若代理把真正优秀的候选排在第十三位，它仍会被预算截断；这正是选择后悔与预算机会成本必须审计的原因。

结构分析、拓扑排序和指纹计算各自随图规模多项式增长；关键成本仍在官方核内准备、物理内存检查与共享 DDR 事件模拟。十二个位置的上限约束的是正式候选机会，而不是保证所有图在固定墙钟秒数内结束。已有成效的基础候选先保留，新邻域若只造成菜单替换而没有形成不同方案，扩大的搜索种类反而可能使结果变差。论文因此只陈述当前经过全百图核验的统一规则，不以开发阶段某几张图的局部改善作为泛化成绩。

5.10 同核复用和跨核传播的集合论表示

场景 B 中“同核子图同一 Task”改变了切图的边际成本。对内部张量 x ，以生产核心 $c_p(x)$ 和去重后的消费核心集合 $\mathcal{C}(x; p)$ 描述传播需求。若 $c_p(x) \in \mathcal{C}(x; p)$ ，在生产核心上继续消费无需因子图边界而复制到 DDR；其它消费核心分别需要获得该张量。一个仅用于静态排序的跨核读取机会数是

$$R_x^{\text{remote}}(p) = \sum_{c=1}^k \mathbf{1}\{c \in \mathcal{C}(x; p), c \neq c_p(x)\} = n_{\text{remote}}(x; p). \quad (57)$$

若把 x 的三个消费者映射到同一个远端核心， $R_x^{\text{remote}} = 1$ ；把它们映射到三个不同的远端核心则为 3。从式 (57) 可推出，按算子边数惩罚跨核访问会高估第一个布局的读取需求，并

可能错误偏好把互不相关计算都聚在源核。对外部输入则没有 $c_p(x)$ ，应从不同消费核心的数量出发，而不能凭空添加一次“源核心写出”。

然而 $b_x R_x^{\text{remote}}$ 仍不是精确额外搬运：一次跨核传输可能要求源端写出，原始图可能已含相关 COPY，多个消费者有不同的私有缓存生存区间，核内启发式也可能额外换入。严谨比较应先在两方案 p, p' 上分别构造官方 COPY 事件集合 $\mathcal{E}_D(p), \mathcal{E}_D(p')$ ，计算 $D^B(p), D^B(p')$ ，再将其作为次指标。静态 R_x^{remote} 的职责只是决定哪些共享张量的消费者有合并到同核的潜力。本文把“跨核目标集合减少”与“正式额外搬运必减少”明确分开，避免把代理变量写成赛题评分变量。

如果一个远端核心内的两个消费算子之间插入很多与 x 无关的工作， x 的逻辑活跃区间变长。设首次使用在 t_1 ，最后一次使用在 t_2 ，那么理想完全驻留占用面积为 $b_x(t_2 - t_1)$ ；把两次使用靠近可能缩小这部分面积，并在某些拥塞情况下减少溢出。但是移动顺序还会改变其它张量的生命周期，不能仅按一个张量的面积贪心。对候选顺序可定义加权峰值代理 $\max_t L_{c,r}^{\text{ideal}}(t)$ ，并与跨核读取机会数一起观察：如果为了减少 R_x^{remote} 而令峰值显著超过 L1/UB 容量，便应保留原布局参加官方对照，而不是无条件接受聚集。

5.11 同步释放的精确落点与链式传播

场景 B 的 500 周期是源端特定 `extttCOPY_OUT` 完成与目标端对应 `extttCOPY_IN` 可发射之间的约束。若源任务还有后续的无关计算，目标不必等源 Task 完成；若目标核心在释放期间执行另一条独立分支，释放不一定增加 Makespan。设某目标读取的源写出在 f_{mout} 完成，目标 DDR Pipe 最早在 a_{mpipe} 空闲，其数据前驱允许的最早时刻为 a_{mpred} ，则它的发射下界为

$$s_{\min} \geq \max\{f_{\text{mout}} + 500, a_{\text{mpipe}}, a_{\text{mpred}}\}. \quad (58)$$

写成 $f_{\text{mout}} + 500 + a_{\text{mpipe}} + a_{\text{mpred}}$ 显然会把可并行等待相加。若目标之后有矩阵操作 v 必须等该读取完成，其开始还要满足 $s_v \geq f_{\min}$ ，但同核心的其它 Vector 操作只服从自己的依赖与 Pipe 顺序。式 (58) 与事件释放式 (7) 共同决定同步开销如何传播到最终输出。这一传播链较“每个跨核边附加常数 500”更准确。

在相同图中，单纯迁移一个子图可能改变多条同步边的方向与数量。设子图 S 原在核心 a ，迁至核心 b ；其前驱生产核心集合为 $P(S)$ ，后继消费核心集合为 $Q(S)$ 。若 $P(S)$ 中许多生产者在 b ，迁移可删去若干输入同步；但如果 $Q(S)$ 中多数消费者仍在 a ，输出同步可能增加。潜在净变化可以先用集合差

$$\Delta \hat{N}_x(S : a \rightarrow b) = \sum_{x: C(x) \cap S \neq \emptyset} (R_x^{\text{remote}}(p') - R_x^{\text{remote}}(p)) \quad (59)$$

估计，而非把所有边计一次。但即使 $\Delta \hat{N}_x < 0$ ，也可能只删除非关键分支上的同步，或者增加更长的同核内存活跃区间。真实接受规则仍取完整官方 T^B 的差值。这里的集合表达是候选筛选的合理数学依据，未假定每少一次同步就能赚到整整 500 周期。

5.12 划分与映射联合搜索的可解释性

把边界、核心和局部顺序分成完全独立的三次优化并不充分。给定一个边界动作 a_π ，其潜在收益可能只有在随后迁核 a_m 并插入到释放空隙 a_σ 后才能显现；只评价中途的 a_π ，会误把整个联合动作淘汰。为避免对三维组合穷举，本文只构造少量由共同瓶颈触发的联动候选。令 $\mathcal{R}$ 为关键输出附近且共享某些张量的有限区域，候选写成

$$p' = \text{RepairOrder}(\text{MoveCore}(\text{ShiftBoundary}(p, \mathcal{R}), \mathcal{R})). \quad (60)$$

每一步允许“保持原值”以覆盖单动作，但总提案数和官方机会数设硬上限。候选必须在完整动作结束后统一判定是否合法，不能用某个中间状态的差周期代表最终联合动作；同时必须保留原方案，避免错误代理将有效布局丢失。实际 r07 交付仍采用式 (55) 的有限普通联合迁移与局部顺序修复，式 (60) 描述更一般的可解释邻域形状，不宣称交付版遍历了其中所有边界组合。

有限联合候选的评价顺序应兼顾结构差异。例如两个方案虽然算子分组不同，但如果由官方展开后得到完全相同的 Task、COPY 及核上顺序，其官方结果也会一致，重复占用评价机会没有信息价值。规范指纹首先包含原图哈希、核数和场景参数，然后包含每个可切算子的子图标识、每核有序子图列；仅凭生成过程名称或代理分数无法识别同一方案。若不同标识只是子图编号的置换，应在指纹前规范化编号。这个去重过程不改变算法的数学可行集，却直接提高固定预算中真正不同候选的覆盖率；它是“更多算子”之外的一种求解效率改进。

5.13 场景转换的独立对照原则

问题二相对问题一增添同核复用并改变同步语义，因此 $T_{i,k}^B < T_{i,k}^A$ 不能全部归因于“问题二算法更聪明”。若要分离场景机制和搜索机制，应对同一个 p 同时计算 $T^A(p)$ 与 $T^B(p)$ ，再在 B 中比较移植 p^A 与新选 p^B 。形式上，若符号与合法性允许，可写总差

$$T^A(p^A) - T^B(p^B) = [T^A(p^A) - T^B(p^A)] + [T^B(p^A) - T^B(p^B)]. \quad (61)$$

第一项是固定 A 方案的场景变化，第二项是 B 场景中的重新选方案。若某移植方案不满足 B 的物理约束，应先执行正式转换与合法化，不能强行套式。论文中的三问主表主要报告各场景最终效果，不把式 (61) 两项写成已经全量测得的独立因果估计；这个分解为将来做更完整的统一消融提供准确口径。它也解释问题二为何必须重新运行官方评价，而不能只把 A 的 1000 改成 500 再报告预计收益。

5.14 百图结果与验证

表 7 显示五核逐图平均加速比为 4.159974，百图总周期为 65 769 876。两至五核在相同固定单核参照下均完成一百张图；图 3 为规定的平均加速比曲线。总额外搬运量没有随核心数严格单调变化，表明较短 Makespan 与较少逻辑 COPY 不总能同时获得。

表 7 问题二当前方案的百图汇总

核心数	平均加速比	总周期 (cycles)	总额外搬运 (bytes)
1	1.000000	312 479 373	1 241 686 120
2	1.981113	153 876 051	1 128 799 934
3	2.815714	103 980 830	1 079 358 470
4	3.550822	79 744 878	1 092 834 382
5	4.159974	65 769 876	1 087 391 086

相对本工程上一轮完整百图方案，当前配置在 400 个多核组合中 13 胜、387 平、无退步；这些组合总周期从 403 398 480 降为 403 371 635，降幅约 0.006655%。同时，额外搬运合计由 4 387 116 718 增至 4 388 383 872 bytes，增加约 0.028884%。这一小幅变化来自整套候选选择与机会分配，不能表述为某一局部算子带来的独立效果，也不应掩盖次指标的代价。

四百份多核方案均由场景 B 官方评价器最终回放，物理张量生命周期容量检查通过，非法候选未进入交付结果；一百份单核点使用题目规定的固定整图基准。五百份标准方案与逐例周期、搬运和加速比一同保存。个别图在增加核心数后仍出现周期上升，原因可能是同步及带宽代价抵消新增并行；因此“核心越多每例必然越快”不是模型约束。本问的改进仅对给定百图与固定配置成立，未提供全局最优证书或独立未见图的泛化保证。

5.15 全量指标及其权重

场景 B 五核的 100 图总周期为 65 769 876，相对固定单核 312 479 373 周期下降 78.9523%；总周期比为 4.751102。但赛题规定的五核逐图平均加速比为 4.159974，两者相差明显。这并不表示某一套结果错算，而是式 (14) 所示的加权方式不同：总周期比给仍耗时很长的图更高权重，逐图平均让每张图的速度贡献相同。为避免在论述“效果”时偷换口径，表 8

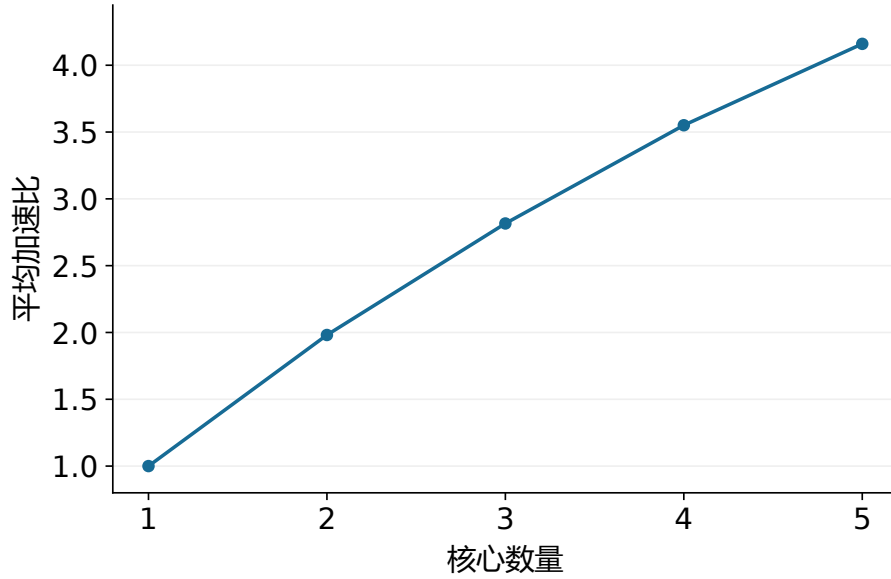


图3 问题二的百图逐图加速比算术平均

同时保留两列，并只以逐图平均加速比作为规定折线图的点。问题一与问题二都使用同一

表8 问题二的逐图平均与百图总周期口径

核数	平均加速比	总周期比	总周期下降率	总额外搬运 (bytes)
1	1.000000	1.000000	0.0000%	1 241 686 120
2	1.981113	2.030721	50.7564%	1 128 799 934
3	2.815714	3.005163	66.7239%	1 079 358 470
4	3.550822	3.918488	74.4800%	1 092 834 382
5	4.159974	4.751102	78.9523%	1 087 391 086

张原始图和同一固定单核参考，但两问的 Task、跨图搬运与同步规则不同，且各自重新选了方案。因此例如“五核 B 总周期小于 A”是场景与算法共同变化后的结果，不能直接宣称“500 周期同步比 1000 周期快多少”或“同核驻留单独贡献多少”。如果要单独归因某个机制，应在同一方案上做合法的成对反事实评价，并保持其余规则与硬件参数相同；本文的问题三正采用了同方案硬件对照来避免这一类混淆。

5.16 非单调图与特定结构的解释

增加核心数并不自动扩展同一方案的无损可行域：现行算法对每个 k 分别划分、映射与排序，有界预算产生的候选池也不同。全 100 图、相邻核心数的 400 个比较中，有 12 处较多核配置反而更慢。第 64 图四核为 10 160 周期、五核为 11 391 周期；第 71 图三核为 9 867 周期、四核为 10 732 周期；第 44 图三核 44 484、四核 49 989 周期，但五核又降至 43 028。这些点说明“平均曲线单调”与“每图曲线单调”是两个不同命题，也说明重新划分时可能失去先前核数下的有利结构。仅凭周期与搬运汇总尚不足以把每个反例唯一归因为同步、DDR 或缓存；需要固定方案与时间线才能分析具体控制路径。

第 5 图从单核 95 618 周期逐渐降至五核 50 673 周期，同时额外搬运从单核 0 增至五核 1 688 564 字节。它体现了主目标与次指标的取舍：如果为了控制搬运而保留整图在一个核心，可能错过可用并行；如果只追求更多任务并行，又会扩大跨核 COPY 和同步等待。当前目标先比较周期，在相同周期下选较少搬运；论文不把这一个图的结果推广为所有图的规律。

5.17 自身版本对照与结果可信范围

当前完整 B 方案与上一轮本工程的百图 B 方案在同一 400 个多核配置上比较, 13 组周期改善、387 组持平、无退步。四种多核配置合计从 403 398 480 降至 403 371 635 周期, 减少 26 845 周期, 即约 0.006655%; 同时逻辑额外搬运合计从 4 387 116 718 增至 4 388 383 872 字节, 增加 1 267 154 字节, 即约 0.028884%。数字说明当前机制有小幅已观测收益, 也说明收益不是“毫无代价”的。新结构准入、重复机会补位与普通联合邻域共同组成这一次版本变化, 现有全量对照不支持把 26 845 周期分摊给单一模块。

对全部 500 个配置, 保存的方案均可在官方 B 规则下重放; 400 个多核配置的物理张量容量审计与完整结果字段核对通过。单核 100 个点采用题目规定的整图核内启发式参考, 而不是从多核最优候选回推出单核。这样的复核证明结果与本次数据、配置和官方评估语义一致, 却不是全局最优证书或对未见计算图的统计泛化检验。若更换核数上限、DDR 带宽或内存容量, 结构门控、局部成本和候选排序均须重新检查。

6 问题三：共享 FIFO 缓存下的多处理器调度

6.1 缓存状态与双带宽事件模型

问题三继承场景 B 的合法方案集 $\mathcal{P}_B(k)$ 、跨核释放与核内内存约束, 新增容量为 C 的共享只读 L2。它不是每核各有一份, 也不改变已生成的逻辑 COPY 数量。对时刻 t 按进入先后排序的逻辑张量队列记为 $Q(t)$, 有

$$\sum_{x \in Q(t)} s_x \leq C, \quad C = 1\,048\,576 \text{ bytes.} \quad (62)$$

正字节 COPY_IN 事件 e 在发射时刻 a_e 查询逻辑张量标识。若标识已在 $Q(a_e^-)$ 中则 $h_e = 1$, 改由 L2 服务; 否则 $h_e = 0$, 仍由 DDR 服务。命中不刷新 FIFO 次序。搬运在完成时按官方处理顺序进行填充: 未驻留且大小不超过 C 的张量, 从队首淘汰至有足够空间后入队; 超容量张量旁路。两个同时发射的冷读不能因为较早一个尚在途中而合并为一次命中。完成时的重新插入、相同时间事件次序等均保持官方语义。

设 $A_D(t), A_L(t)$ 分别为 DDR 和 L2 池的在途操作集合, $b_D = 60$ 、 $b_L = 250$ bytes/cycle。每条搬运按所属池的独占带宽形成初始服务工作量, 活跃集合不变的时间段内, 各自池内公平分享服务:

$$\dot{w}_e(t) = -\frac{1}{|A_r(t)|}, \quad e \in A_r(t), \quad r \in \{D, L\}. \quad (63)$$

事件到达或完成后须结算剩余工作并重算完成时刻; 实际评价还依官方整数周期取整。两池独立, 不能把 $60 + 250$ 当作共同带宽, 也不能给每个核心各分配一条 250 bytes/cycle 的 L2 通道。L2 命中只改变 COPY 的服务池, 不能绕过 (40) 中的跨核同步, 不能占用别的 Pipe 插队, 亦不能使超出 L1/UB 的物理实例合法。

于是问题三在相同合法方案集上求

$$\min_{p \in \mathcal{P}_B(k)} \text{lex}(T_i^{L2}(p; C, b_L), D_i(p)). \quad (64)$$

字节命中率为

$$H_i = \frac{\sum_{e: \text{COPY_IN}, s_e > 0} h_e s_e}{\sum_{e: \text{COPY_IN}, s_e > 0} s_e}. \quad (65)$$

分母为零时只在统计程序中定义为零并标注“无有效读取”。官方命中率按字节而非按次数统计。 D_i 是插入额外逻辑 COPY 的字节量, 不因命中自动扣减; 同一方案分别在无 L2 和 L2 下评价, 该字段应相同。命中字节可作为实际 DDR 服务减量的辅助诊断, 但 H_i 越高未必总周期越短, 因为同步、计算、L2 带宽竞争以及关键路径变化仍可能占主导。

6.2 缓存微事件及状态转移的完整定义

问题三的方案空间仍是满足场景 B 合法性条件的 $\mathcal{P}_B(k)$ ，并未新增“选择某张量是否命中”的自由变量。命中由方案诱导的 COPY 发射时刻与全核共享 FIFO 状态内生决定。为区别实际缓存与外层预测，以离散事件序号 j 表示官方确定的同刻处理顺序，记

$$Q_j = (x_1, \dots, x_{m_j}), \quad u_j = \sum_{r=1}^{m_j} b_{x_r} \leq C, \quad Q_0 = \emptyset. \quad (66)$$

队列从左至右按进入缓存的先后排序， $C = 1048576$ bytes。对正字节 COPY_IN 事件 e ，在发射时读取其逻辑张量标识 ℓ_e ，确定

$$h_e = \mathbf{1}\{\ell_e \in Q_{j(e)-}\}, \quad r(e) = \begin{cases} L, & h_e = 1, \\ D, & h_e = 0, \end{cases} \quad (67)$$

其中 L, D 分别是 L2 与 DDR 服务池。查询命中不把张量移到队尾，这与最近最少使用策略不同。服务池在发射时已经确定；即使传输在途期间该张量被别的完成事件淘汰，本次请求也不临时改道。对于非正字节读取与 COPY_OUT，按官方特殊规则执行，但不把它们计入“正字节读入命中率”的分子或分母。

填充发生在读入完成时。令事件完成的逻辑张量为 x 。若标识无效、 $b_x > C$ 或 x 已在 Q_j 中，队列保持；否则选择最短的队首淘汰前缀长度

$$h^* = \min \left\{ h \in \{0, \dots, m_j\} : u_j - \sum_{r=1}^h b_{x_r} + b_x \leq C \right\}, \quad (68)$$

再定义

$$Q_{j+1} = (x_{h^*+1}, \dots, x_{m_j}, x), \quad u_{j+1} = u_j - \sum_{r=1}^{h^*} b_{x_r} + b_x. \quad (69)$$

对于 $b_x \leq C$ ，取 $h = m_j$ 总能容纳，故式 (68) 的集合非空。式 (69) 是 FIFO：只删除队首前缀，幸存者相对次序不变，新张量放队尾。原先命中的在途读取若在完成前被淘汰，完成时若缓存中仍无该逻辑张量，也按相同规则重新插入；若已由别的完成事件插入，则不刷新队列。两个几乎同时发射的冷读在尚无完成填充时均可未命中，不能为了提高命中率私自合并。

这些规则给出三个可证明的不变量。首先， Q_0 不含重复标识，插入前先判驻留，故归纳可得所有 Q_j 中标识唯一。其次， $b_x > C$ 的张量旁路，其他新入张量按式 (68) 淘汰到足够空间，所以 $0 \leq u_j \leq C$ 恒成立。第三，命中只读取、不重排，淘汰只删前缀，故幸存张量的 FIFO 相对顺序保持。由发射时的二值判定，每个有合格逻辑键的正字节读取恰好计入命中或未命中之一。这是实现缓存事件账本时的局部校验条件，也说明只按访问频次估算 L2 收益会遗漏顺序与在途状态。

6.3 两个共享服务池及命中率

问题三保留共享 DDR 总带宽 $b_D = 60$ bytes/cycle，并新增全核共同的 L2 读带宽 $b_L = 250$ bytes/cycle。两条带宽属于不同池；一个 L2 命中请求不能同时按 DDR 与 L2 服务，也不能把两者相加为一个 310 bytes/cycle 的通用资源。对事件 e 的正字节 b_e ，定义独占参考服务工作量

$$q_e^r = \left\lceil \frac{b_e}{b_r} \right\rceil, \quad r \in \{D, L\}. \quad (70)$$

在活跃集合不变时各池分别按式 (63) 分享，事件到达或完成后结算剩余工作，再由官方的整数事件顺序决定下一次状态转移。整数上取整、零字节请求及同刻多个完成事件的处理

可以改变下一次发射与 FIFO 次序，故这里的连续服务式是物理解释，精确主结果仍取官方事件仿真。

令 $R(p) = \sum_{e \in \mathcal{R}_{in}(p)} b_e$ 为固定方案下所有合格正字节读入量， $X(p)$ 为其余必须经 DDR 的逻辑搬运量， $H(p, C, b_L) = \sum_e h_e b_e$ 为命中字节。固定方案展开的 COPY 集合不因只读 L2 本身改变，因此两池的字节工作量可写成

$$D_D = X + R - H, \quad D_L = H, \quad H_B = \begin{cases} H/R, & R > 0, \\ 0 \text{ (标记为无有效读取)}, & R = 0. \end{cases} \quad (71)$$

H_B 是字节命中率，不是命中请求次数除以请求总次数。式 (71) 还说明：命中会改变由哪个服务池承担读取，却不自动删除方案中的逻辑 COPY 事件。因此同一个 p 在 B 与 L2 两种硬件环境下评价，逻辑额外搬运 $D(p)$ 应相同，虽然实际 DDR 服务字节可减少。若缓存感知搜索最终改用另一个 p_L ，其划分和通信本身改变， $D(p_L)$ 则可能升高。

仅由共享带宽的容量限制可得

$$T^{L2}(p) \geq \max \left\{ \frac{X + R - H}{b_D}, \frac{H}{b_L} \right\}. \quad (72)$$

两项取最大而非相加，因为两个服务池可并行运行，计算 Pipe 也可与两者重叠。若只观察 H_B 增加，式 (72) 的 DDR 项下降，但 L2 项上升，且真实关键路径可能转移；“命中率升高必然缩短 Makespan”不成立。对于完成时间的更强条件界，还可与式 (10)、固定方案每核计算 Pipe 负载取最大；这些界不能反推出真实完成时间等于各资源忙碌时长之和。

6.4 乐观命中上限与容量、带宽包络

对固定方案 p ，设逻辑张量 x 被正字节读取 $m_x \geq 1$ 次、每次大小为 b_x 。空缓存必须经历至少一次冷读； $b_x > C$ 的张量根本不能驻留。因此无需知道访问次序便有乐观上限

$$0 \leq H \leq H_{\max}(p, C) = \sum_{x: b_x \leq C} (m_x - 1)b_x. \quad (73)$$

等号通常难达到：并发冷读可能多次未命中，不同张量之间的 FIFO 淘汰也可能让已读张量再次变冷。式 (73) 只回答“最多有多少重复读取字节有资格转向 L2”，并不保证这些字节真实命中。

为了说明硬件收益的理论边界而非预测实测，将 H 放宽为区间 $[0, H_{\max}]$ 内可任意选择的连续变量 z 。对固定 p 的两池负载，构造

$$\Phi(p; C, b_L) = \min_{0 \leq z \leq H_{\max}(p, C)} \max \left\{ \frac{X + R - z}{b_D}, \frac{z}{b_L} \right\}. \quad (74)$$

第一项随 z 下降，第二项随 z 上升，其无约束交点解由

$$\frac{X + R - z}{b_D} = \frac{z}{b_L} \implies z_0 = \frac{(X + R)b_L}{b_D + b_L}.$$

由于 $z_0 \geq 0$ ，受上限截断后的最优放宽值为

$$z^* = \min\{H_{\max}(p, C), z_0\}, \quad \Phi(p; C, b_L) = \max \left\{ \frac{X + R - z^*}{b_D}, \frac{z^*}{b_L} \right\}. \quad (75)$$

真实 H 必在放宽区间中，故 $\Phi(p; C, b_L) \leq T^{L2}(p)$ 。容量 C 增大时 $H_{\max}$ 不减，放宽可行域扩大；带宽 b_L 增大时，对于每个 z 的第二项不减，所以 Φ 对两参数均不减。这是乐观包络的单调性，不是 FIFO 真实周期的单调性证明。真实队列与完成顺序会随容量、带宽改变，甚至使某个固定方案在带宽增大时略慢。

全空间最优值也必须与有限候选和方案条件界分开。定义

$$T^* = \min_{p \in \mathcal{P}_B(k)} T^{L2}(p), \quad U_A = \min_{p \in \mathcal{A}} T^{L2}(p), \quad (76)$$

其中 $\mathcal{A} \subset \mathcal{P}_B(k)$ 是实际完成官方评价的有限集合。由式 (10) 和集合包含关系，严格成立

$$L_0(k) \leq T^* \leq U_{\mathcal{A}}. \quad (77)$$

若还定义全空间放宽下界 $L^* = \inf_{p \in \mathcal{P}_B(k)} \max\{L_0(k), \Phi(p; C, b_L)\}$ ，则 $L^* \leq T^*$ 。但只在有限候选集上取 $\min_{p \in \mathcal{A}} \Phi(p; C, b_L)$ 不能替代 L^* ，更不能用 $T(p) - \Phi(p)$ 宣称剩余优化潜力。本文没有求得全空间 L^* 或 T^* ，只把这些式子用于解释何种资源可能限制已选方案。

6.5 同一张量并发读取的时序例子

缓存状态不能仅由“这一轮请求的张量集合”决定；发射与完成顺序是必需的状态变量。设某个合格张量 x 尚未驻留，两个核心的读取请求 e_1, e_2 在 $t_1 < t_2$ 发射，而第一请求的完成时刻为 $f_1 > t_2$ 。在两个发射瞬间 $x \notin Q(t_1^-)$ 且 $x \notin Q(t_2^-)$ ，所以按官方发射时查询规则，两次均为未命中：

$$h_{e_1} = h_{e_2} = 0, \quad t_1 < t_2 < f_1. \quad (78)$$

若代理只统计“ x 在这一批被读取两次”，便可能误用式 (73) 给它一次命中。式 (73) 确实允许一次命中的乐观上限，但实际顺序 (78) 实现的是零次。第二个 DDR 请求在 t_2 加入还会降低第一个请求的瞬时带宽份额，两个请求彼此影响，即使它们来自不同核心且不具有算子依赖。第一个完成后， x 才可能入 FIFO；第二个请求已经选定 DDR 池，不能中途转为 L2。第二个完成时若 x 仍在队列，按规则不刷新其入队位置；若中间被其它事件淘汰，才可能重新插入。这个例子证明缓存状态和传输在途状态必须同时进入事件模型。

再设缓存只能容纳两个大小均为 b 的张量，初始队列 $Q = (x, y)$ ，依次发生对 x 的命中、一个新张量 z 的读入完成、之后对 x 的查询。由于 FIFO 命中不刷新顺序，第一次命中后仍是 (x, y) ； z 填充时删除队首 x ，队列变成 (y, z) ；最后一次 x 查询为未命中。若把该机制错误实现成 LRU，第一次命中会形成 (y, x) ，填充时淘汰 y ，最后一次查询反而命中。这两个序列仅有替换语义不同，却可能给出不同 DDR 工作量和 Makespan。因而问题三即使沿用问题二同一划分，也必须重新按 FIFO 规则执行官方仿真，不能用 LRU 的经验命中率系数乘原 DDR 时间。

还需区分缓存队列大小和实际占用的物理通道。一次 L2 命中读取在逻辑上仍是 `extttCOPY_IN`，其目的端私有存储在执行时仍必须符合 L1/UB 容量；共享 L2 的一兆字节不能被算作每核额外的一兆字节 UB。对张量 x ，若它在 L2 中驻留并被多个核心命中，这只说明源端读取可从 L2 带宽池服务，不能证明这些核心的计算都可同时持有私有副本，也不删除跨核源写出后的 500 周期释放。缓存提升的是某些读入的服务途径，不直接修改核内计算 Pipe 或算子依赖。由此可在理论上解释为什么一次命中有时只降低 DDR 忙碌时长，却没有缩短最终输出控制路径。

6.6 固定方案消融与缓存事件引导搜索

首先将问题二的 500 份方案在无 L2 的场景 B 与启用 L2 的场景下分别官方重放。两次使用相同划分、映射和顺序，因此其差异刻画固定方案上的硬件效应。随后以该 Q2 方案为保底，在真实 L2 时间线中定位反复未命中的张量与相关消费子图，有限尝试合法的核心迁移和空隙插入；普通关键度候选保留为后备。新方案再由完整 L2 官方事件评价，并按周期优先、同周期较少逻辑搬运接受。每个配置最多评估四个新增候选；继承 Q2 种子的既有求解成本与本问新增成本分别记账。

这种拆分避免把“硬件新增 L2”与“因 L2 改变调度”混成一个不可解释的总效应。固定方案比较不需要宣称算法等预算公平；新增邻域是否优于另一种搜索方式则必须在同一候选机会下单独验证。三张开发图的同预算普通邻域对照为二胜六平一负，因此本文只报告最终统一规则的百图成绩，不将该小样本写成普遍优势。开发集中将本问候选上限从四增为十二也未使九个配置的采用周期继续下降，这只是局部预算检查，不是搜索收敛证明。

6.7 硬件作用与调度作用的成对识别

令 $p_{B,i,k}$ 为问题二对图 i 、核数 k 的选用方案， $p_{L,i,k}$ 为问题三重新搜索后的方案。定义三次相互不同的评估：

$$T_B = T^B(p_B), \quad T_{\text{fixed}} = T^{L2}(p_B), \quad T_{\text{selected}} = T^{L2}(p_L). \quad (79)$$

T_B 与 T_{fixed} 的方案完全相同，只改变有无共享 L2； T_{fixed} 与 T_{selected} 的硬件完全相同，只改变被选方案。对同一图与同一核数，可以逐项分解

$$R_{\text{hw}} = \frac{T_B}{T_{\text{fixed}}}, \quad R_{\text{search}} = \frac{T_{\text{fixed}}}{T_{\text{selected}}}, \quad R_{\text{total}} = \frac{T_B}{T_{\text{selected}}} = R_{\text{hw}} R_{\text{search}}. \quad (80)$$

式 (80) 只是逐配置的恒等式，不能把 100 张图的 $\bar{R}_{\text{hw}}$ 与 $\bar{R}_{\text{search}}$ 相乘后称为 $\bar{R}_{\text{total}}$ ，因为平均乘积一般不等于平均值乘积。表 9 的每一列都先在逐图层面计算，再取算术平均。三条五点加速比曲线沿用相同的无 L2 单核参考；问题三的单核 L2 比值允许大于 1，不能把它强制定为赛题前两问的单核点。

这一成对设计还限定了“硬件收益”的解释范围：它是在固定已选 Q2 方案上增加 L2 的局部反事实，不是“所有可能调度下增设缓存的纯因果平均”。Q2 方案原本按无 L2 规则选优，未必是 L2 的最佳方案；问题三再搜索的收益则取决于新增候选池。只有三次评价都使用相同图、核数、原始 DDR 配置及官方代码版本，式 (80) 才有明确含义。

6.8 从 FIFO 账本构造有界候选

给定种子方案 p_B ，先在题目默认 $C = 1 \text{ MiB}$ 、 $b_L = 250 \text{ bytes/cycle}$ 下评价，得到完整事件时间线。遍历正字节 COPY_IN 事件，记录其逻辑张量、发射时 FIFO 队列、命中标志、服务池、完成时刻及相关消费子图。对重复未命中的张量，区分“首次冷读”“容量淘汰后再读”和“并发冷读”三种情况；仅有命中率数字无法指明可操作的修复位置。再把影响最终输出的较晚读入或关键链等待映射回少数相关子图，生成迁移核心或合法顺序插入候选。这样选择的仍是 (π, m, σ) 的局部变化，算法没有直接修改 FIFO 替换策略或手动指定某次命中。

设关键再读 e 的发射时刻为 a_e ，其最近一次缓存填充为 f_x ，中间将 x 淘汰的事件为 v_x 。一个看似自然的操作是让消费 e 前移到淘汰之前；但这可能压住同核的矩阵/向量计算，或把新的 DDR 请求放进更拥堵的窗口。另一操作是把 v_x 的相关子图后移，让 x 暂时保留；这又可能推迟最终输出。故缓存账本只负责发现候选区域，并用是否满足原 DAG、跨核 COPY 释放、Pipe 顺序及 L1/UB 容量来过滤提案。最终仍由完整官方 L2 评价计算候选的 T^{L2} 和 D 。

每个配置至多评估四个新增且不同的候选，问题二种子与最终独立重放单独记账。若新候选均未改善实测字典序目标，则返回 p_B 在 L2 下的评价结果；若有改善，则返回已评价方案中的最佳者。因而相对固定 L2 种子有逐配置不退步保障，而相对无 L2 的 Q2 时间没有普遍保障：同一方案一旦加入共享 FIFO 与双带宽竞争，事件次序可能变，五百组中确有小幅负例。

搜索选择只应用了事件引导的迁核与插入、普通关键度后备和有限官方比较。开发阶段增加候选预算以及更复杂的关键链“消费前移 + 淘汰源后移”组合，未达到全量采用条件，不能把其开发样本收益写成当前五百组成绩。特别是在九个开发配置中，事件引导与普通邻域同预算对照为二胜六平一负；把候选上限从四增至十二也未让这九组最终时间继续下降。它支持“本轮采用较小预算”的工程选择，但不是搜索已收敛或更大预算永不收益的证明。

6.9 命中敏感性与排序风险

把张量读入从 DDR 改由 L2 服务，除了使独占工作量从 $\lceil b/60 \rceil$ 改为 $\lceil b/250 \rceil$ ，还会改变其完成时刻、后继算子发射、并发传输集合以及后续 FIFO 填充与淘汰次序。可用一个条件

性扰动链表达：

$$(C, b_L, p) \longrightarrow (Q_j, h_e, r(e))_e \longrightarrow (s_e, f_e)_e \longrightarrow \mathcal{H}(p) \longrightarrow T(p). \quad (81)$$

箭头表示事件规则的依赖，不等于每一环都是可独立操纵的变量。固定 p 增加 C 或 b_L 时，第一步可能使更多读取命中或更早完成，却也可能改变后续淘汰和竞争，最终 T 不保证逐参数单调。这与式 (74) 的乐观界单调并不矛盾：界允许任选 z 平衡两个池，真实 FIFO 不允许自由选择命中分配。

由此可见，在问题三里用“命中字节最多”“额外搬运最少”或“传输独占时间最低”直接选方案都可能误判。对同一候选池若要改善排序器，应该固定 Q2 种子、缓存初态、候选集合与评价预算，比较代理排名和官方 T^{L2} 排名；记录前两名官方覆盖率及选择遗憾，而不是仅展示代理预测下降的候选。只有在这些诊断证明筛选更可靠后，扩大缓存邻域才有根据。本文仍以已核验的有限候选算法交付，不把这些后续想法冒称为已实现收益。

6.10 百图结果与指标边界

表 9 把无 L2、固定 Q2 方案启用 L2 和重新调度后的 L2 三列并列。前三列加速比均使用同一个无 L2 整图单核基准；后两列则先在每张图、相同核数上计算时间比，再对百图取算术平均。故“五核两个平均加速比之比”不等于“逐图同核时间比的平均”。五核优化 L2 的平均加速比为 4.240375，无 L2 对照为 4.159974；其逐图同核时间比均值为 1.022567。

表 9 问题三的百图成对评价（各比值均逐图计算后平均）

核数	无 L2 加速比	同方案 L2	优化 L2	同核硬件比	同核综合比
1	1.000000	1.008622	1.008622	1.008622	1.008622
2	1.981113	1.984989	1.986721	1.001750	1.002467
3	2.815714	2.828518	2.834213	1.004810	1.006671
4	3.550822	3.586810	3.598194	1.011699	1.014923
5	4.159974	4.227368	4.240375	1.019090	1.022567

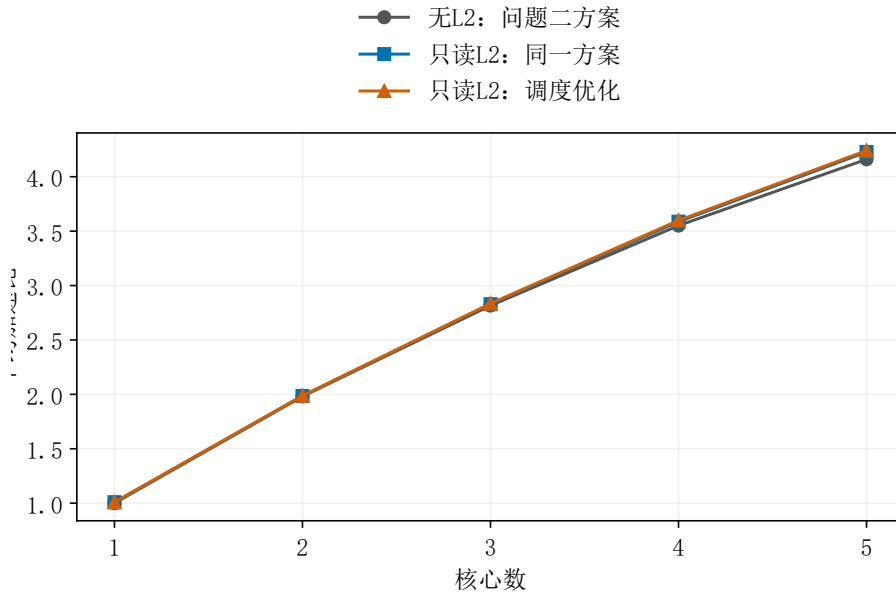


图 4 无 L2、固定方案 L2 与优化方案 L2 的平均加速比

五核百图总周期在无 L2 时为 65 769 876，固定方案启用 L2 后为 64 721 316，进一步调整方案后为 64 562 282。全 500 个配置中，新增搜索使 68 组周期降低，其他组保留不劣的

已评价方案；它不保证相对于无 L2 都改善。固定方案启用 L2 有六组退步，最终优化方案仍有四组比无 L2 慢，不能隐藏。全五百组逻辑额外搬运量从无 L2 方案的 5 630 069 992 增至优化 L2 方案的 5 634 488 280 bytes，增加 4 418 288 bytes；这种次指标代价与周期优先的目标一致，但应在方案比较时明确呈现。图 5 给出按同核配对的收益，而不是从总体命中率直接推导周期百分比。

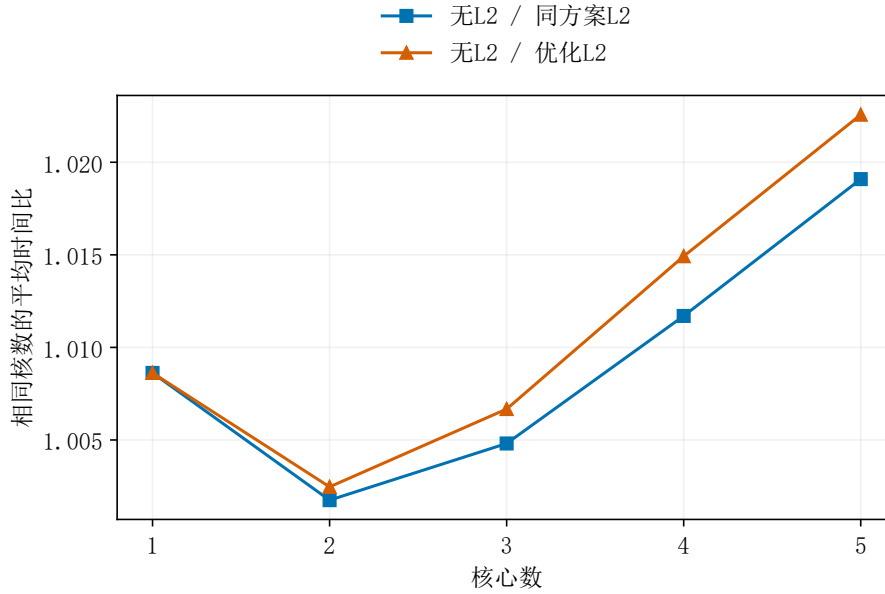


图 5 相同核数下的 L2 硬件效应与调度后综合时间比

6.11 反例、灵敏度与检验

第 44 图单核从无 L2 的 154 407 降至 107 814 cycles，216 次命中、4 408 416 命中字节全部来自溢出后的重新读取，说明共享 L2 的作用不局限于原始输入复用。以同一方案改变容量时，结果见表 10。这是事后机制诊断，揭示阈值与局部非单调，不是独立确认实验。FIFO 对容量不具备一般的命中集合嵌套性质，相关经典讨论可参见 Belady 等^[5]；本题具体数值只来自自己的官方事件重放。

表 10 第 44 图单核固定方案的 L2 容量检查

容量 (KiB)	0	64	128	256	512	1024	2048
周期	154407	154272	154407	154407	134726	107814	107814

第 5 图五核固定方案在 L2 带宽分别为 60、125、250、500 bytes/cycle 时，周期分别为 48 200、48 202、48 259、48 253。带宽变化会改变发射与完成次序，进而改变冷读重叠、FIFO 状态和关键路径；不能从“服务速度变快”直接断言固定计划 Makespan 必单调。主实验始终使用题目固定的 1 MiB、250 bytes/cycle。最明显的固定配置退步为第 67 图双核：无 L2 32 040 130，有 L2 32 217 720 cycles。时间线复核将差异定位于搬运关键路径，但该观测分解并非独立因果效应估计。

全部 500 组均完成无 L2/L2 官方成对评价、胜出方案再次回放、FIFO 事件账本和全局物理内存检查；无 L2 周期与问题二冻结指标逐项相同。官方测试集上的回归一致性强于单个演示图，却仍不是未见图的泛化保证。本模型解释固定参数下的结果，未求得所有合法划分上的全局最优，也未把仿真周期当作真实芯片实测时间。

6.12 缓存收益的逐层分解与负例复核

表 11 以百图周期总和为口径列出三种评价。对给定核数 k ，记 $A_k = \sum_i T_{i,k}^B(p_{i,k}^B)$ ， $F_k = \sum_i T_{i,k}^{L2}(p_{i,k}^B)$ ， $O_k = \sum_i T_{i,k}^{L2}(p_{i,k}^C)$ 。固定方案的硬件节省为 $A_k - F_k$ ，重新选方案的追加节

省为 $F_k - O_k$ 。这两个差量具有共同的求和基准，因此恒有

$$A_k - O_k = (A_k - F_k) + (F_k - O_k). \quad (82)$$

式 (82) 是总周期的代数分解，不代表“硬件效应”和“算法效应”在任意方案上独立；后一项是以已经启用 L2 的方案为参照计算的。表中单核固定方案与最终方案一致，多核新增搜索仍取得有限但非零的周期缩减。五核的总节省为 1207594 周期，其中 1048560 周期来自固定方案打开 L2，159034 周期来自在 L2 场景中重新选择候选。

表 11 百图总周期的成对分解 (cycles)

核数 k	无 L2: A_k	固定方案 L2: F_k	优化 L2: O_k	$A_k - F_k$	$F_k - O_k$
1	312479373	311669310	311669310	810063	0
2	153876051	153835706	153824098	40345	11608
3	103980830	103101443	103060720	879387	40723
4	79744878	78650750	78583168	1094128	67582
5	65769876	64721316	64562282	1048560	159034

每图等权的平均加速比与表 11 的加总节省回答不同问题。设 $g_{i,k} = T_{i,k}^B/T_{i,k}^{L2}$ ，则平均同核时间比为 $100^{-1} \sum_i g_{i,k}$ ；加总周期比为 $A_k/O_k = \sum_i (T_{i,k}^{L2}/O_k) g_{i,k}$ 。后者把多核执行时间长的图赋予更高权重。因此五核平均同核时间比 1.022567，并不能由 65769876/64562282 直接得到；反过来，不能拿单个超大图的大幅节省冒充多数图均有相同收益。论文同时给出两者，让赛题规定的公平逐图指标和计算资源的实际合计开销都可核验。

逻辑搬运与实际服务分离。 设一份方案中的正字节读取集合为 E_{read} ，其中命中 L2 的集合为 E_{hit} 。若不计其它原始 DDR 写入或溢出，读取的实际 DDR 服务字节为 $\sum_{e \in E_{\text{read}} \setminus E_{\text{hit}}} b_e$ ；逻辑额外搬运 $D(p)$ 却仍按问题二、三共同的方案展开规则记录。由于所选方案可能改变划分与排程， $D(p^C) - D(p^B)$ 可以为正，也可以为负；不能把同一方案的命中字节从 D 中直接扣除。表 12 从保存的逐配置记录聚合逻辑额外搬运，并列出逐图平均字节命中率。五核命中率较高，但额外逻辑搬运亦较无 L2 方案增加，这并不矛盾：为改善时间所选的新方案在边界或核内展开上可能产生更多逻辑 COPY，其中一部分恰好可由 L2 服务。命中率本身没有包含这些新 COPY 在关键路径上的位置。

表 12 方案改变后的逻辑额外搬运和 L2 字节命中率

核数	无 L2 方案额外字节	所选 L2 方案额外字节	字节差量	平均命中率
1	1241686120	1241686120	0	0.092037
2	1128799934	1128372862	-427072	0.138071
3	1079358470	1079972310	613840	0.203754
4	1092834382	1094063150	1228768	0.232050
5	1087391086	1090393838	3002752	0.265921

为什么“更快的存储”仍可能使方案变慢。 在一次已经确定的调度中，如果只把单个 COPY 的持续时间缩短、而保留全部其它事件的先后与资源分配，则最终最长路径不可能增长；但问题三的正式事件仿真没有保持这些条件。一次 L2 命中会改变搬运完成时刻，可能改变别的核上冷读进入 DDR 的先后、L2 填充与淘汰顺序、核内启发式的后续发射和不同 Pipe 的竞争关系。因此比较 $T^B(p)$ 与 $T^{L2}(p)$ 时，事件图本身也可能不同。 $T^{L2}(p) > T^B(p)$ 的负例并不违反固定 DAG 上的单调性定理，而是说明“事件图固定”这个定理前提在这里不成立。图 67 双核正是需要保留的反例：仅凭命中数预测必然获益会误导搜索。

逐图风险与策略边界。 对每个配置分别记硬件差量 $\Delta_i^H = T_i^B(p_i^B) - T_i^{L2}(p_i^B)$ ，方案差量 $\Delta_i^S = T_i^{L2}(p_i^B) - T_i^{L2}(p_i^C)$ 。因为 p_i^B 始终进入候选池， $\Delta_i^S \geq 0$ 是有限候选选择规则保证的；而 Δ_i^H 没有非负保证。若把两个差量混为一个“算法收益”，便会错误归因给缓存事件引导搜索。最终仍有四组相对无 L2 的负例，分别是第 53 图双核、第 67 图双核、第 84 图双核及第 97 图四核；其中第 67 图的时间损失显著大于另外三组的个位数或百位数损失。报告均值时保留这些负例，可防止小幅多数收益遮蔽极端风险。此处的“负例”是跨硬件场景的成对比较，不表示问题三选择器在 L2 场景中放弃了更优的已评价保底方案。

容量与带宽的条件敏感性。 容量 C 影响一次完成后的 FIFO 队列状态；带宽 b_L 则影响完成顺序，进而影响后续队列状态。所以敏感性函数 $T(C, b_L; p)$ 通常不能分解为“容量因子乘带宽因子”。第 44 图的容量扫描出现 64 KiB 比 128 KiB 快的情况，提醒我们 FIFO 没有一般的容量包含性保证；第 5 图的带宽扫描围绕 48 200 周期小幅摆动，提醒我们新增带宽不一定触及最终关键路径。本文只在官方固定 $C = 1 \text{ MiB}$ 、 $b_L = 250 \text{ bytes/cycle}$ 下选择交付方案；参数扫描用于验证模型对阈值与顺序变化的解释，不能替代全量固定配置评价。

7 模型检验与敏感性分析

7.1 结果口径与可复核性

三个问题均固定相同的 100 张计算图和 1 至 5 核配置，逐组保存算法方案与赛题评估输出。单核加速比的基准采用题目要求的启发式核内调度结果，而非数学最优值；平均加速比先在每张图上求单核与多核时间之比，再对 100 张图取算术平均。该值与单核总周期除以多核总周期不同，后者更突出大图的绝对时间贡献，本文同时报告两者，但只将前者作为赛题指定指标。额外搬运量均沿用官方评估输出的定义，不将缓存命中率或代理估计值混入结果。

每个问题的 500 组方案均经过格式、依赖与官方评估复核。问题一的固定交付证据为 20260924-A-q1-delivery-r02；问题二为 20260924-A-q2-delivery-r07；问题三为 20260924-A-q3-delivery-r03。本文表格和曲线从这些冻结记录生成。对局部优化，保留原方案作为候选，只有官方评价确认改善才替换；这避免由代理排序误差直接造成无条件退步，但不能保证跨场景结果单调，也不能证明全局最优。

7.2 核数与任务粒度

图 1、3 和 4 表明三种场景的平均加速比总体随核数增加。增核同时提高并行能力，也会增加跨子图边、同步释放点或共享资源竞争，因此单图并非必然单调。问题二记录中有 12 个相邻核数配置出现较多核更慢的情况；问题三为 10 个。对这类图，单纯增加核数不是优化动作，应重新联合检查划分、核映射和执行顺序。

子图粒度的作用具有双向性。小子图提供更多可并行任务与调度空隙，但增加跨图数据搬运和任务释放等待；大子图削减边界代价，却可能压缩多核并行度，也可能提高核内活跃数据量并触发换入换出。因此本文生成分量整体、分量内切分及局部边界调整等不同结构的候选，并使用官方完成时间选择，不预设某个算子数量为最优粒度。对大图采用有界候选筛选和局部复用，目的是减少重复核内排程开销；搜索阶段的计时不能代替冷启动、单核基准构建和最终复核在内的端到端计时。

7.3 共享缓存参数与评价目标

问题三对典型图进行了 L2 容量和带宽扫描，而非对 100 图全部硬件参数做全组合搜索。在案例 case_044 的单核检查中，缓存命中可消除部分溢出重读，使完成时间从 154407 周期降至 107814 周期；这一大幅收益不应外推至全部图。相反，case_005 五核在已检查的带宽值 60、125、250、500 下分别得到 48200、48202、48259、48253 周期，表现为近似持平且略有非单调；case_067 两核的 L2 方案为 32217720 周期，高于对应无 L2 方案的 32040130 周期。此类负例说明，缓存降低逻辑搬运与缩短关键路径是两个不同命题，L2 仲裁、传输重叠和任务释放均须进入评价。

问题三固定配置下，所选 L2 方案的 500 组逻辑额外搬运合计比无 L2 对照多 4418288 字节；有 4 组方案时间也高于无 L2。本文因此不以减少搬运量或提高命中率单独宣称性能收益。论文报告的结论只对应给定的硬件配置、FIFO 规则、官方评价程序和当前 100 图集合并；若换用真实芯片带宽、缓存替换策略或不同图分布，需重新校准并复核。

7.4 全量实验的索引、版本与证据合同

任何一张图的一个核数配置是一次独立的优化单元。令索引集合 $\mathcal{I} = \{1, \dots, 100\} \times \{1, \dots, 5\}$ ，则每问均应有 $|\mathcal{I}| = 500$ 条终态记录。每条记录至少保存图标识 i 、核数 k 、方案 $p_{i,k}$ 、官方完成时间 $T_{i,k}$ 、官方额外搬运 $D_{i,k}$ ，以及用于重放的输入与评价器身份。只报告五个均值而不保存这 500 条明细，无法检查平均公式、重复配置或隐匿负例。三问对应冻结证据版本 Q1-r02、Q2-r07、Q3-r03；逐例表直接由相应目录的 CSV/JSON 记录生成，完整目录路径见附录的结果引用。原始赛题输入和固定硬件参数不在论文中被重写为“新数据”，后续若修改搜索器，应产生新的版本证据，而不能覆盖冻结旧结果。

正式结果的核对由三个相互独立的层次组成。方案层检查非原始 COPY 算子的唯一覆盖、子图编号及每核顺序、商图与顺序弧无环，原图算子与张量属性未被改写。执行层重新调用赛题评价程序，验证每个 Task 的 COPY、Pipe 依赖、容量及场景规则有合法时间线，并将最终周期和搬运与记录逐项比较。统计层从逐例结果独立聚合五点平均加速比、总周期和搬运总量，检查单核固定基准是否一致。三层不能互相代替：CSV 行数为 500 只证明覆盖索引，不证明其中每个方案可运行；官方输出可以证明该场景的执行结果，但若论文把总周期比错误写成平均加速比，统计结论仍不成立。

给每条记录附上内容哈希 $h_i = H(G_i)$ 、硬件配置哈希 $h_{s,k} = H(s, k, \vartheta)$ 、方案规范指纹 $h_p = H(\text{canon}(p))$ 和评价器版本 h_E ，则缓存键应至少包含

$$K = (h_i, h_{s,k}, h_p, h_E). \quad (83)$$

若改变场景 A/B/C 或 L2 容量、带宽，却复用旧键的评价结果，可能把不同问题的周期混淆；若只用方案名而不含完整映射与顺序，也可能把同名不同方案合并。式 (83) 描述可复核的缓存合同，不表示所有历史开发运行都已按此键存储。交付结果以独立官方重放为准，缓存只用于减少确定相同配置的重复计算。

7.5 指标从逐例记录到论文表图的推导

设 $\mathbf{T}_k = (T_{1,k}, \dots, T_{100,k})$ 为某问固定场景下的周期向量， $\mathbf{T}_1^0$ 为题目规定的整图启发式单核基准。赛题五点曲线按式 (12) 逐元素相除再求平均：

$$\bar{S}_k = \frac{1}{100} \mathbf{1}^\top (\mathbf{T}_1^0 \oslash \mathbf{T}_k), \quad (84)$$

其中 $\oslash$ 为逐元素相除。总周期效率为 $C_k = \mathbf{1}^\top \mathbf{T}_k$ ；不同核数的总周期比 C_1^0/C_k 是以多核周期为权重的平均比值，见式 (14)。例如问题一五核的 $\bar{S}_5 = 3.665903$ ，而总周期比约为 $312479373/73170501 = 4.270565$ ；两者差别并不是程序错误，而是权重口径不同。若把后者绘成赛题规定的平均加速比，将夸大这一指标。问题二五核对应的均值为 4.159974，总周期比为 4.751102；问题三五核 L2 优化方案均值为 4.240375，但同核无 L2 到 L2 的逐图时间比均值仅为 1.022567，这些数也不能直接相互替换。

对逻辑搬运，按核数汇总为 $D_k^\Sigma = \sum_i D_{i,k}$ 。由于 $D_{i,k}$ 的物理口径并不等于实际 DDR 服务字节，不能用 $D_k^\Sigma/60$ 与 C_k 的比例作为拥塞强度的准确估计；在问题三尤其不能从 L2 命中字节直接减去 D_k^Σ 。可以报告相对变化 $\Delta D_k = D_k^\Sigma(\text{new}) - D_k^\Sigma(\text{base})$ ，但是否接受新方案仍按每配置字典序决策。聚合后周期变快而 $\Delta D_k > 0$ 并不矛盾，因为新增搬运可能没有落在关键路径上；反过来，搬运下降也不保证更快。三问的附录表以每配置为最细粒度保留这种不一致，从而允许独立读者挑选任何一张图复核。

7.6 成对比较、胜负台账与负例定义

公平比较两个算法版本 A, B 时，应在同一集合 $\mathcal{J} \subseteq \mathcal{I}$ 上建立成对差值 $\delta_j = T_j^A - T_j^B$ ，并记录

$$N_+ = \sum_{j \in \mathcal{J}} \mathbf{1}\{\delta_j > 0\}, \quad N_0 = \sum_{j \in \mathcal{J}} \mathbf{1}\{\delta_j = 0\}, \quad N_- = \sum_{j \in \mathcal{J}} \mathbf{1}\{\delta_j < 0\}. \quad (85)$$

同时给出总周期差 $\sum_j \delta_j$ 和搬运差。只报告 N_+ 会掩盖少数大幅退步，只报告总差会掩盖多数配置毫无变化。对开发集上的参数选择，若先观察结果再改变规则，再用同一小集合报告“独立验证”，会把调参和验证混同；全量固定规则回放和开发集消融必须明确区分。本文最终百图成绩不是随机抽样推断，也没有给固定一百图构造没有意义的“总体置信区间”；结论限于题目给定集合和参数。

问题二 r05 到 r07 的最终对照是一例：在 400 组多核配置上， $N_+ = 13$ 、 $N_0 = 387$ 、 $N_- = 0$ ，周期和从 403 398 480 降至 403 371 635，净减少 26 845 cycles，约为 0.006655%；额外搬运增加 1 267 154 bytes，约为 0.028884%。由此能说 r07 在这批配置中取得小幅、全量可复核的周期改善，且搬运次指标存在代价；不能把 13/400 描述成普遍的大收益，也不能因为净改善大于零就声称新结构候选总能更快。问题三的 68 组“新增搜索改善”则是有 L2 场景内的成对比较，与四组“最终 L2 比无 L2 慢”的跨场景负例属于不同参照，必须分开登记。前者包含保底所以没有场景内退步，后者说明硬件场景改变未保证固定方案的单调性。

若需评价候选排序而不是最终算法，应在同一候选池中保存所有 $\hat{T}_j, T_j$ 。Spearman 等级相关系数可用于度量总体单调关系，但在正式预算 B 很小的场景中，更直接的是前两名覆盖率与式 (19) 的选择遗憾。定义最优候选被前二名覆盖的比例为 $100^{-1} \sum_i \mathbf{1}\{j_i^* \in \{\hat{j}_{i,1}, \hat{j}_{i,2}\}\}$ ，前提是每配置确实有至少两个不同的、均已官方评价的候选。若很多配置只产生一个独特候选，不能把它当成排序器的完美命中。本文没有在缺完整候选池官方真值的地方填造相关系数；已有代理预测改善但正式评价输赢接近随机的开发证据，只能证明需要继续校准，不是某个数值相关系数的证据。

7.7 三问的约束回归与反事实检查

问题一的核心回归是验证每个边界 Task 确实经 DDR 中转，即便两个 Task 被映射到同核也不保留私有缓存；场景 A 的 100 周期间隔与跨核 1000 周期释放分别落在正确的 Task 边。若误把场景 B 的同核驻留带入问题一，计算出的 D 将系统性偏低。问题二的回归是验证同核所有子图组成一个 Task，同核消费者可在容量允许时复用，跨核输入在源端 extttCOPY_OUT 完成后再延迟 500 周期释放；若把 500 加到整个 Task，周期会系统性偏高且错误阻塞不相关 Pipe。问题三再检查共享 L2 是全核单实例、只读、FIFO、发射时查找和完成时填充，并与 DDR 在不同带宽池服务；若把缓存命中当成已消除的逻辑 COPY，命中率与额外搬运的物理意义都会错位。

一个有力的反事实检查是固定方案改变单一场景机制。问题二到问题三保存同一 p^B ，先关闭 L2 得到 $T^B(p^B)$ ，再打开 L2 得到 $T^{L2}(p^B)$ ；之后才允许重新调度得 $T^{L2}(p^C)$ 。表 11 的三列恰与这三步对应。问题一到问题二也可用式 (61) 设计类似对照，但两场景的 Task 构造差别大，须先验证移植方案在 B 中合法。若不做固定方案重放，就无法区分 L2 本身改善与搜索器改进；若只看最终 Q3 比 Q2 的平均加速比，则还可能误读两种统计权重。固定方案反事实不是现实硬件的受控实验，却是仿真模型中清楚的机制消融。

7.8 核数非单调与结构解释的边界

对同一图的相邻核数，定义 $\eta_{i,k} = T_{i,k} - T_{i,k-1}$ 。若 $\eta_{i,k} > 0$ ，说明本次有限搜索在 k 核配置给出的方案较 $k-1$ 核方案慢，而不是证明“多一个物理核心必然使最优时间更差”。通常可以让新核心闲置并复用旧的映射，但不同核数的固定评估配置、候选生成与预算若未包含嵌入旧方案，就不能直接从有限解得最优值单调性。问题二记录中的 12 个相邻核数退步、问题三中的 10 个，提示算法在某些图上存在映射或候选覆盖不足。示例包括第 64 图问题二四核 10 160、五核 11 391 cycles，以及第 71 图三核 9 867、四核 10 732 cycles。它们

值得进一步分析，但不能只凭这两个数判定 DDR、同步或溢出中的哪一项负责；原因必须由具体时间线中关键输出的激活约束确认。

理论上若允许把旧 $k-1$ 核方案原样嵌入 k 核且新核心闲置，二者使用相同场景参数和评价语义，则全局最优满足 $T_{i,k}^* \leq T_{i,k-1}^*$ 。推导是 $k-1$ 核的每个合法方案都属于 k 核可行集，因此后者最小值不会增大。这个集合包含关系不自动保证交付的有界启发式输出也单调，因为其候选池 $\mathcal{C}_{i,k}$ 未必包含上一核数的嵌入方案。一个可行的后续增强是将已验证的 $k-1$ 核方案作为 k 核保底，重新评价后允许其与本核新方案竞争；但在未对百图证明输入输出接口完全同构、并检验新增保底的机会成本之前，本文不把它写成交付版已实现机制。

7.9 粒度与内存压力的双向敏感性

子图数量 q 与最终周期的关系不应预设为单峰或单调。令 $T(q; p)$ 表示保留其它条件的某候选族中、具有 q 个子图的方案时间。把一个子图拆成两个，至少可能改变三项：并行度上限变高，边界流量及场景 A 的 Task 间隔变多，核内峰值驻留可能下降。三项交互，故 $T(q+1) - T(q)$ 的符号由最终事件时间线决定。若只把 q 当作连续超参数做大规模扫描，也会违反题目“避免暴力迭代”的求解效率目标。本文使用整分量、少数拓扑窗口和小区域边界修复，形成离散而可解释的几类尺度，再在相同正式预算下比较。

内存压力也不能直接由“张量总字节超过容量”判断。对一个 Task，若每个张量的生命周期区间两两不重叠，即便总字节数远大于 $L1/UB$ ，也可能只需有限容量轮流处理；若许多张量生命周期重叠，总量较小也可能在局部峰值触发换出。理想存活峰值 $P_{c,r} = \max_t L_{c,r}^{\text{ideal}}(t)$ 与面积 $A_{c,r}$ 分别刻画瞬时压力和长期占用，实际是否发生溢出还依赖核内启发式选择的次序。若保存方案的溢出搬运为零，继续以“减少溢出”为这批方案的主要方向就缺证据；若某些未选候选溢出较大，则它们仍可作为该机制的反例。问题三第 44 图的收益来自已经发生的溢出重读命中，是 L2 的场景效应，不表示问题一、二应追求更高溢出以便缓存命中。

7.10 求解耗时与评估周期的不同量纲

设 $t_{i,k}^{\text{wall}}$ 为求解器在 CPU 上从读取输入到输出方案的墙钟秒数， $T_{i,k}$ 为方案在模拟 NPU 上的执行周期。这两个量没有直接换算关系。增加官方评价次数可能降低 T ，却提高 t^{wall} ；复用已经准备的核内子问题可能大幅降低 t^{wall} ，却使同一方案的 T 完全不变。题目要求算法单用例尽量在五至十分钟给出高质量方案，因此两类指标都要报告，并说明计时区间：冷启动含输入解析、单核基准、候选生成、官方评价和最终复核；热搜索仅记录某一阶段。已记录的搜索阶段最长约 364 秒，但这不包含全部冷启动与最终复核，不能据此宣称所有大图端到端低于十分钟。另一方面，大图的单核基准计算本身可达十余分钟，若把它与多核搜索混在一个数字里，也不能准确判断哪一部分最值得工程优化。

设一次完整官方评价平均耗时 c_E ，候选准备 c_P ，预算 B ，则粗略墙钟成本为 $c_{\text{init}} + B(c_P + c_E) + c_{\text{replay}}$ ；真正的大图成本还随核内启发式和事件数变化。若某两个候选复用同一核内准备键，后者可减少 c_P 而不改变 c_E ；若重复方案通过式 (83) 直接取得已校验的正式结果，连重复的 c_E 也可避免。把节省的时间用于一个结构不同的候选，在保持正式评价机会数不变时可能提高结果质量，但是否如此仍须成对对照。本文将主机耗时控制作为独立的后续改进方向，而不把搜索运行更久所得的改善伪装为“算法本身更快”。

7.11 适用范围、失败样例与可迁移性

三个问题的结果建立在赛题给定的算子周期、张量字节数、L1/UB 容量、共享 DDR 带宽、场景同步规则及官方启发式调度上。若硬件改为不同 Pipe 组合，式 (10) 须按资源种类重写；若 DDR 改为不公平服务，式 (9) 不再适用；若 L2 改为 LRU 或允许写入，式 (69) 和缓存状态推导须整体改写。即便硬件相同，换一批图的分量数、共享输入程度和关键链深度也可能改变结构候选的有效性。因此本方法可迁移的是“场景规则展开、结构初解、事件反馈、有界正式评价”的求解框架，而非从这 100 图学出的某个固定分块粒度或收益百分比。

失败样例是检验模型诚实度的重要证据。问题一某些图在增核后出现周期退步；问题二存在 12 组核数非单调；问题三仍有四组相对无 L2 更慢。这些现象并不否定所有启发式，只表明目前有限候选池和代理排序尚未穷尽结构空间。若下一轮扩展搜索，应先冻结现有候选池、辨别真正不同的方案和被新动作取代的基础候选，随后用相同官方预算对照其净收益；若直接扩大评价次数，结果改善无法归因于模型修正。论文最终交付的正面结论限定为：统一规则在给定百图和固定配置下输出经官方复核的合法方案，并获得所列的平均加速比、总周期和逻辑搬运；它不包含对全局最优、未见图泛化或真实芯片能耗的额外承诺。

7.12 逐图分布：平均值之外的结构异质性

五点平均曲线是赛题指定的主结果，但均值不能描述一百张图之间的离散程度。为避免只列举最好的例子，表 13 从冻结逐例账本取五核逐图加速比的最小值、十分位、四分位、中位数、四分之三位、九十分位和最大值。分位数按排序后作线性插值；它是描述这一百张给定图的经验分布，不是未知总体的概率分位数。问题一的中位数 4.310373 高于均值 3.665903，反映一批低加速图把算术平均拉低；问题二的中位数 4.648439 亦高于均值 4.159974。这比只说“有些图较难”更具体：算法已有大量接近五倍的配置，但低分位结构仍值得定向检查。

表 13 五核逐图加速比的经验分位数（100 图）

场景	最小	10%	25%	中位数	75%	90%	最大
问题一：A	1.001729	1.590521	2.442082	4.310373	4.909619	4.996015	5.501141
问题二：B	1.678167	2.539582	3.553248	4.648439	4.921746	5.036342	5.827795

图 6 从全部一百张图绘制经验累积分布，而不是选取几个“代表图”。纵坐标为 $\hat{F}(s) = 100^{-1} \sum_i \mathbf{1}\{S_{i,5} \leq s\}$ 。图中场景 B 曲线在较多区间位于 A 曲线下方，意味着在相应阈值以下的图占比较小，但两场景的算法与物理语义同时变化，不能据此把整段位移全部归因于缓存驻留。它的实际用途是定位低分位结构：在场景 A 中接近单核时间的极端图，通常需要检查原图的可并行宽度、Task 释放点、共享输入重复读取以及大图核内成本，不能仅通过加大搜索预算盲目修复。

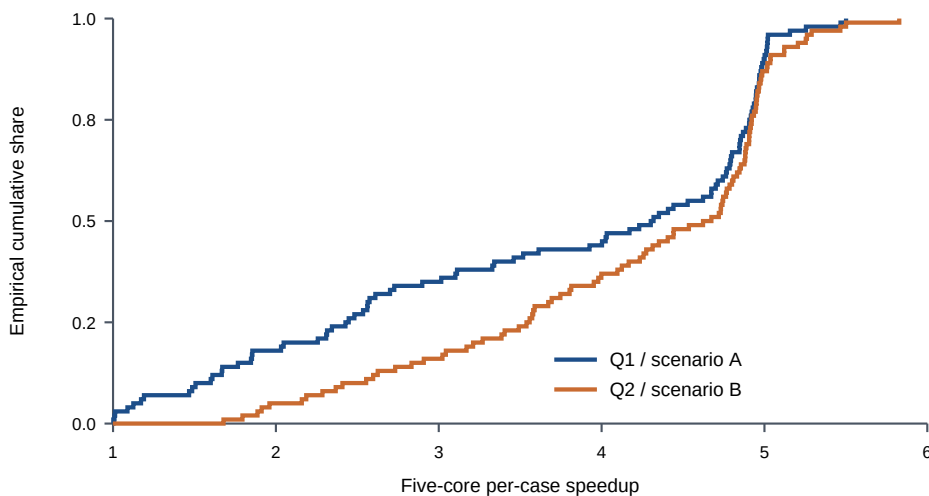


图 6 问题一、二五核逐图加速比的经验累积分布；数据来自冻结的各 100 条五核记录

7.13 大图周期贡献与等权指标的资源分配冲突

问题二五核总周期为 65 769 876。按五核绝对周期从高到低排序，前十张图的周期合计约占总周期的 70.6449%。这十张图是第 67, 28, 79, 72, 14, 73, 97, 16, 91, 21 图；其中第 67 图五核仍需 13 461 153 周期，单独贡献超过总周期的五分之一。这样的集中度说明若目标是减少这一批图的总运行负担，改善少数大图的绝对周期很重要。但赛题主图的 $\bar{S}_5$ 每张图只占 1/100，大图改进对规定平均加速比未必同样显著。两种目标之间没有逻辑矛盾，区别在于“每个 case 公平等权”和“每个 cycle 公平等权”的利益分配。

可用边际效应精确表示这层差异。若只把第 i 图五核时间从 T_i 降至 $T_i - \delta$ ，其中 $0 < \delta < T_i$ ，则平均加速比提升

$$\Delta \bar{S}_5 = \frac{T_{i,1}^0}{100} \left(\frac{1}{T_i - \delta} - \frac{1}{T_i} \right) = \frac{T_{i,1}^0 \delta}{100 T_i (T_i - \delta)}, \quad (86)$$

而总周期直接减少 δ 。对于小改动 $\delta \ll T_i$ ，均值的边际系数约为 $T_{i,1}^0 / (100 T_i^2)$ ；总周期的边际系数恒为 1。由此，若把全部 CPU 搜索时间都给绝对周期最大的图，可能提升总周期比却错过对平均加速比更有效的小图；反过来，只追逐易于抬高逐图比例的小图，又可能让最耗时图占据绝大部分总周期。本文先完成全一百图的合法交付，再依据两种指标分别分析，不用一种统计量替代另一种。

在求解预算上，可以考虑一个受限的优先级分配模型。设每张图的额外评价机会为 b_i ，主机总预算为 B_{CPU} ，官方评估一次的预计成本为 c_i ，则形式上约束 $\sum_i b_i c_i \leq B_{\text{CPU}}$ 。若仅以总周期为目标，可估计每次机会的预计绝对改进 $\hat{g}_i^{\text{cycles}}$ ；若以题目平均加速比为目标，可估计式 (86) 对应的 $\hat{g}_i^{\text{mean}}$ 。两类收益都有代理误差，不应通过自适应地不断尝试直到某图变好来制造不公平预算。交付版采用固定每配置上限和结构门控，保证可复核；上述模型是解释下一轮如何在额外、独立的开发预算里决定优先分析哪类图，而非宣称当前百图成绩已通过跨图最优预算分配取得。

7.14 L2 收益分布与负收益口径

问题三的五核同核时间比 $g_i = T_{i,5}^B / T_{i,5}^{L2}$ 分布也不均匀。其最小值和前四分之一分位数均为 1，中位数约 1.004997，四分之三位约 1.031714，九十分位约 1.060672，最大值约 1.176398。图 7 显示 29 张图的五核时间完全持平、25 张图在 1 到 1.01 之间，只有 8 张图的时间比达到或超过 1.08。因此 1.022567 的五核逐图平均同核比主要来自具有明确缓存机会的那部分图，不能把它解释为所有算例都取得约 2.26% 的均匀改进。

五核直方图恰好没有相对无 L2 的退步，不能据此隐去其它核数下的四组负例。对一百图中每张图的 k 核分别优化后，跨场景比值的符号会随 k 改变：同一图在一核或五核可能命中有效，在两核却因 FIFO 顺序、DDR 与同步路径耦合略慢。图 7 只展示 $k = 5$ 的横截面；表 11 和附录五百行保留全部 $k = 1, \dots, 5$ 。若把仅五核的无负例错写成全五百组无负例，就会和第 67 图双核等正式结果冲突。由此也可见，报告分布时必须明确图集合、核数、场景和方案是否固定，否则一个“平均收益”没有可验证的分母。

7.15 结果图生成的最小复现合同

新增的两张分布图由仓库图表/论文诊断图.py 从冻结的三份 CSV 重新生成，不读取开发阶段的搜索日志。程序先筛选 `cores=5` 并断言每组恰有 100 行，再按实测 `speedup` 或 `combined_ratio` 绘图；直方图还检查六个桶的数量总和为 100。这一检查只保障绘图输入的基本形状，不能代替上述方案层和官方评价层核验。图中的英文坐标是为了避免图形库字体差异影响跨平台复现，中文标题与物理解释在图注和正文给出。若后来修改了逐例方案，应重新生成 CSV 与图表并更新交付版本，而不能让正文数值、图像与附录指向不同的运行。

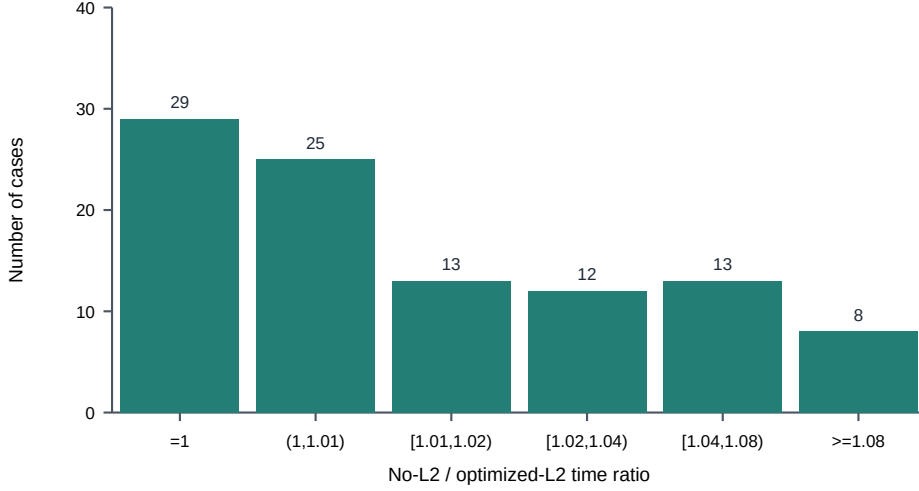


图7 问题三五核同图同核无 L2 与优化 L2 的时间比直方图

8 模型评价与结论

8.1 模型特点

本文以同一计算图表示为基础，按三个场景逐步扩展状态与约束：场景 A 把每个子图视为独立 Task 并令跨图数据经 DDR；场景 B 引入同核复用和跨核同步；场景 C 进一步显式记录共享 L2 的 FIFO 状态与带宽竞争。这样的分层建模保留了问题之间可比较的划分、映射和顺序决策，又避免将某场景不存在的通信或缓存收益误用于另一场景。

求解采用结构化初解、关键路径优先调度与有界局部修复。连通分量、共享输入、双计算管道负载和缓存压力共同决定候选的结构；关键链等待与资源空隙决定局部调度；官方评估程序负责最终合法性和完成时间裁决。通过保留原方案、冻结预算及记录逐组结果，算法可区分预测收益与实际收益，适合在 100 图、1 至 5 核的固定配置上复现。

8.2 模型优点的边界化表述

第一，三个问题共用统一的算子、张量与方案表示，场景变化由 Task 构造和事件服务规则承担。这样可以复用覆盖、无环与核心顺序检查，而不把场景 A 的每子图 DDR 规则硬塞给 B，也不把 C 的 L2 命中率写成独立优化变量。统一不是“用一套固定成本系数解释三问”，而是对相同的决策 p 使用相应的正式评价器。式 (2) 把这种共性与差异分开，降低了跨问迁移时的概念错误。

第二，模型将静态结构、局部预测和官方结果分为三个证据层。静态结构可以证明一个商图是否无环、某一分量是否主导算子数，不能证明其周期；局部预测可以决定有限候选的优先次序，不能直接替代已验证方案；官方结果可以确定指定参数下的 **Makespan** 与额外搬运，却不能证明全局最优。这样划界，使得每一个正面结论都可追溯到足够强的证据。尤其对共享 DDR 和 FIFO，线性边权代理易失真，因此最后一层正式评价至关重要。

第三，方案的主要目标和次要目标遵循字典序。这比把搬运字节随意乘上一个大常数并加到周期里更符合题意，因为二者量纲不同，常数若取值不当会让一字节差异压倒真实周期改善。定义

$$(T_a, D_a) \prec (T_b, D_b) \iff [T_a < T_b] \vee [T_a = T_b \wedge D_a < D_b]. \quad (87)$$

由此，只要 T_a 比 T_b 少一周，即使 D_a 略高，仍优先 a ；两者周期完全相同才看搬运。由于官方周期是离散整数，这一判别可直接执行而不需要人为调节权重。在论文结果中某些方案的逻辑搬运上升而周期下降，正是式 (87) 主次关系的体现，而非模型自相矛盾。

第四，百图全量结果和逐例附录相互约束。平均加速比、总周期、逻辑额外搬运、Q3 同方案缓存效应分别从相同的五百行账本聚合；负例和相邻核数非单调也保留。这样的证据链允许读者从主文曲线一路追到一个算例、一种方案与一份官方评估记录。与只报告若干开发图相比，它更能检查候选规则是否在多种结构下稳定；但全量覆盖仍只覆盖赛题给定图集，不自动成为对未见图分布的统计保证。

8.3 模型缺点、已观察到的失效及可操作改进

成本预测仍是最主要的算法弱点。候选之间的共享 DDR 竞争不是独立子图时长的简单和：把一个 Task 提前可能改变所有同刻传输的服务份额，延后另一个更关键的读取；对问题三，命中改变的是事件的服务池，并可能改变完成顺序和后续 FIFO 队列。即便代理保留四条 Pipe 的局部依赖，如果冻结的背景状态与候选修改后的全局状态不同，排名仍可能错。问题一过去已有代理看好而官方接近四胜五负的审计现象，因此新邻域若仅依赖代理选择，会迅速耗尽有限预算而难以稳定提升。可操作的下一步是：固定同一完整候选池，逐对保存代理值与官方值，核对前二覆盖率、选择遗憾及误差最大的三类事件；仅对被证实误排的关键区域补充更精确的局部事件重放，最后仍用同预算正式评价验证结果。

模型的另一个缺点是缺少紧的、与划分无关的全局最优下界。式 (10) 严谨但忽略 DDR 与同步；条件带宽界和场景 A/B 路径界较具体，却随方案本身改变。若把它们直接拿去计算“当前距最优只差某百分比”，推理无效。未来可以尝试在聚合图上构造更强的资源割集下界，例如对每一种合法切分都必经的张量集合求最小必传量，再把它与算子链、Pipe 工作量耦合。但这样的下界必须对所有可行方案成立，且求解下界本身不应比原题穷举还难。在尚未证明前，本文只用同版本官方对照评价启发式质量。

有限候选预算是求解效率与搜索质量的有意取舍，也造成可见的失败：问题二、三都有相邻核数退步，问题三有四组 L2 相对无 L2 的负例；在大量配置上增加一个搜索算子只带来持平，说明该算子可能重复了现有候选或挤掉有效基础搜索。可以把已验证的少核方案作为多核保底，但需确认增加一个空闲核心不改变评价语义，并从原预算中为这项新保底安排机会。对有主导分量的图，可在现有整分量方案取得实测收益后，才开放一次结构不同的细切候选；对很多小分量的图，保留完整分量分配和两 Pipe 负载均衡，不必固定给每张图相同的拆分机会。这种按结构和实测改善调整机会的策略应在正式规则冻结后做全百图对照，不能根据已见结果事后给个别图额外试验次数再与旧版比较。

大图端到端墙钟时间仍未全量证明满足题目建议的五至十分钟。现有最长热搜索约 364 秒只是部分阶段；第 67 图等大图单核整图核内基准本身可能耗时很长。若仅把搜索次数削到很少而牺牲周期质量，虽然墙钟变短，也不符合主目标；若只重跑相同候选或重复核内准备，则增加墙钟而未扩大信息量。工程上可优先缓存相同输入与硬件下的官方核内准备结果、对不受局部动作影响的子图复用 Pipe 序、对规范相同的完整方案只作一次正式评价，并把冷启动、搜索与最终复核分别计时。这些操作要通过独立官方重放检查缓存键正确，不能为了提速跳过结果核验。

8.4 可推广的求解思想与必须重新校准的部分

对其它具有多核、私有片上存储和共享带宽的计算设备，可迁移的是四步：先把工作负载表示为带共享张量的有向依赖图；再针对硬件通信与同步语义生成结构不同的合法划分；用执行资源和关键输出的事件信息排序有限候选；最后用可信评价器按主次目标选择方案。这一框架尤其适用于“图规模大而精确整数规划不可直接求解、官方仿真较昂贵”的离散调度问题。并非只有神经网络算子可以抽象成类似的图， workflow 或数据流作业也可能共享这一方法学。

不能直接移植的是所有数值常数、FIFO 状态方程和具体切图阈值。例如目标设备若提供跨 Task 的私有缓存保持，场景 A 中同核边界必须经 DDR 的前提就消失；若有网络互联而非共享 DDR，通信的拥塞资源会从一个全局池变成多条链路；若 L2 为组相联 LRU，完成时的 FIFO 淘汰式也不适用。对新设备应重新建立 Task 事件生成规则、服务池状态和存储容量，验证评价器与实际设备的关系，再把本文的有界结构搜索嵌入新规则。保持逻

辑框架，不等于保证原来在一百图上的 3.665903、4.159974 或 4.240375 继续成立。

8.5 主次目标、帕累托候选与可交付选择

字典序使最终选择没有权重任意性，但搜索阶段仍有理由保留少量非最优的结构候选。假设候选 p_a 的周期较短而搬运较多，另一个候选 p_b 的周期较长而搬运较少；根据式 (87) 最终只能选择 p_a ，但 p_b 可能是后续局部修复更好的出发点：它较少的 DDR 字节可为迁核或拆分腾出通信预算。保留候选是搜索状态的策略，不改变最终评分规则。若一次只保留当前字典序最优，所有局部动作都必须从 p_a 出发，可能因邻域狭窄而无法到达 p_b 附近的优质方案；若无限保留帕累托前沿，又会超过题目建议的求解时间。因此可仅保存结构差异显著的少数候选，并把它们的后续官方评价机会纳入总预算。

对两个方案定义弱支配： $p_a \preceq p_b$ 当且仅当 $T(p_a) \leq T(p_b)$ 且 $D(p_a) \leq D(p_b)$ ，并至少一个严格。若 p_b 被支配，它在当前目标和两个单项指标上均不优于 p_a ；但这仍不足以证明从 p_b 出发的所有后继都不优，因为局部结构相异可产生不同的可达邻域。若两方案互不支配，选择 T 更短者用于交付，同时仍可在固定小容量的工作池中保存另一者。这个区别防止把“保留多候选”误写成“改变赛题评分方式”。当前交付版的核心保证是已评价的最短周期方案输出；未来若引入有限帕累托工作池，必须比较相同总预算下的最终 T ，而不能把更多评估次数带来的改善算作方法优势。

图 6 与表 12 也提醒，对于不同场景的方案不能把 (T, D) 直接放入一个共同的帕累托前沿。场景 A 和 B 的 Task/COPY 构造不同，场景 C 虽和 B 使用相同的逻辑 D 定义，但读取走 L2 或 DDR 的物理服务不同；若把 Q1 的 D 与 Q3 的 D 当作同一资源消耗严格比较，会忽略它们对应的带宽池差异。字典序仅在同一场景、同一图、同一核数、同一固定参数下用于候选选择。跨场景的表格回答的是“各题规定的模型分别能达到什么数值”，而非在同一硬件上公平选择 A/B/C 模式的工程结论。

8.6 复杂度与质量之间的可审查权衡

若图有 n 个可切算子、 q 个候选子图和 e 条投影依赖，单次商图拓扑检验是 $O(q + e_\pi + e_\sigma)$ ；对固定划分，反向关键度递推也是同阶。一个朴素空隙插入会查看至多 $q + k$ 个位置，每次检查受影响顺序与依赖；若不复用局部状态，最坏可能多次重建商图。与这些多项式阶段相比，官方核内启发式可能在大图上插入大量换入换出，而全局事件仿真随同时在途搬运的到达、完成次数增长。求解时间因此更接近

$$t_{\text{solve}} = t_{\text{parse}} + t_{\text{seed}} + \sum_{j=1}^{B_{\text{distinct}}} (t_{\text{legal}}^{(j)} + t_{\text{inner}}^{(j)} + t_{\text{event}}^{(j)}) + t_{\text{replay}}, \quad (88)$$

而不是仅用 $O(n + e)$ 概括。 B_{distinct} 是真正不同并需要正式评估的候选数，名义槽位可能因重复而更大；缓存可以减少重复 t_{inner} ，但不能省略最后一次独立回放。式 (88) 使五至十分钟的要求可以被分阶段审计：若 t_{inner} 主导，应优先优化核内准备复用；若 t_{event} 主导，可优化同等语义下的事件队列实现；若 B_{distinct} 过多，则应校准排序和去重。盲目降低所有阶段的精度可能牺牲合法性或主目标。

搜索预算 B 选得过低时，结构候选尚未覆盖关键布局；选得过高时，后续提案的边际收益可能接近零而 CPU 成本继续增长。可定义第 b 次新增正式评价的实际增益 $g_b = T_{b-1}^{\text{best}} - T_b^{\text{best}} \geq 0$ 。由于 **incumbent** 只接受更好方案， g_b 非负，但这并不保证 $g_{b+1} \leq g_b$ ：一次晚出现的结构候选仍可能带来大幅改善。由此不能用“最近三次均持平”作严格的搜索收敛证明。合理的停止规则是在预定硬预算内，结合过去一段候选的结构新颖性和实际收益，决定剩余机会投向何种邻域；若没有这些日志，就应保守维持基础搜索份额。对于本题一百图，所有这类自适应规则还须固定版本后全量对照，而不能在看见最终分数后临时修改某张图的预算。

当算法在同一图上对多个核数重复求解时，计算资源充足的本机或服务器可把不同 (i, k) 配置并行运行；这不改变每配置的候选预算，也不改变 NPU 方案周期。但是若多个进程同时争用主机内存、磁盘或官方评价器的缓存，观察到的墙钟秒数会随部署方式变化。

因此提交中的“单用例十分钟”应在明确的主机硬件、并发数和冷/热状态下报告；把百图并行墙钟时长除以一百不能代表任何单个配置的求解时间。本文的全量成绩是方案执行周期的统计，与运行批次的服务器调度方式分离。

8.7 结论

在指定 100 图与固定硬件配置下，本文完成三种场景的 500 组方案求解与官方评估。场景 A 的五核平均加速比为 3.665903，场景 B 为 4.159974，场景 C 所选 L2 方案为 4.240375。它们对应不同的通信、同步与缓存规则，不应把三者差值解释成单一算法模块的纯收益。三问共同表明：有效的多核方案取决于切图通信、关键路径等待、核内计算与共享存储竞争的联合平衡；最终判断依据是官方模拟的总完成时间，额外搬运量作为同时时间下的次级目标。

参考文献

- [1] 中国研究生数学建模竞赛组委会. 通用神经网络处理器下的多核调度问题 [Z]. 2026.
- [2] Popp M, Schlag S, Schulz C, Seemaier D. Multilevel Acyclic Hypergraph Partitioning[EB/OL]. arXiv:2002.02962, 2020.
- [3] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260–274. DOI: 10.1109/71.993206.
- [4] Liu Y, Xue Y, Cheng Y, et al. Scaling Deep Learning Computation over the Inter-core Connected Intelligence Processor with T10[C]//Proceedings of the 30th Symposium on Operating Systems Principles. 2024. DOI: 10.1145/3694715.3695955.
- [5] Belady L A, Nelson R A, Shedler G S. An anomaly in space-time characteristics of certain programs running in a paging machine[J]. Communications of the ACM, 1969, 12(6): 349–353. DOI: 10.1145/363011.363155.

A 核心程序与复现入口

本附录直接从仓库引用三问的当前提交入口，避免把初版求解器误列为现行算法。完整源码、输入和官方评价程序随交付仓库提供。百图批量复现与校验由 `q1_delivery_r02.py`、`q2_delivery_r07.py` 及问题三冻结交付脚本与交接单约定。问题一入口调用 `q1_experimental.py` 和结构候选模块；问题二入口调用 `q2_reserve_fast.py`；问题三入口调用 `q3_solver.py` 与 `q3_neighborhood.py`。入口代码和核心模块应在相同提交版本下一起使用；纸面列表只是定位复现入口，不能替代仓库中的全部依赖。

A.1 问题一求解器

程序 1 问题一当前提交入口

```
1  """Portable scene-A baseline entry; standard library only, explicit graph
   input."""
2  import argparse
3  import gzip
4  import json
5  import platform
6  import time
7  from pathlib import Path
8  from q1_io import PROCESSED, official, verify, sha, write_json
9
10 from q1_ablation import CONFIGS
11
12 ADOPTED_POLICY = 'routes_gate_reuse'
13 BASELINE_FEATURES = tuple(CONFIGS[ADOPTED_POLICY])
14
15
16 def main():
17     parser = argparse.ArgumentParser(description=__doc__)
18     parser.add_argument('graph', type=Path)
19     parser.add_argument('-n', '--cores', type=int, choices=range(1, 6),
20                         required=True)
21     parser.add_argument('--config', type=Path, default=PROCESSED / 'data/
22                         config.txt')
23     parser.add_argument('--output', type=Path, required=True, help='New
24                         output directory')
25     parser.add_argument('--backend', choices=['official', 'counter'],
26                         default='counter')
27     parser.add_argument('--budget', type=int, default=12)
28     parser.add_argument('--seed', type=int, default=0)
29     parser.add_argument('--verify-final', action='store_true',
30                         help='Additional original official replay, outside
31                         search budget; time is recorded')
32     args = parser.parse_args()
33     if args.cores > 1 and args.budget < 8:
34         parser.error('Protected baseline partition budget requires --
35                     budget >= 8')
36     verify()
37     mod = official()
38     from evaluation_validation import read_evaluation_config,
39     validate_graph
```

```

33 from q1_experimental import solve_experimental
34 raw = json.loads(args.graph.read_text(encoding='utf-8-sig'))
35 validate_graph(raw)
36 settings = read_evaluation_config(str(args.config))
37 waits = mod.read_scene_a_config(str(args.config))
38 args.output.mkdir(parents=True, exist_ok=False)
39 started = time.perf_counter()
40 if args.cores == 1:
41     from singlecore_evaluate import build_singlecore_plan
42     plan = build_singlecore_plan(raw)
43     evaluator = mod.evaluate_scene_a
44     if args.backend == 'counter':
45         from q1_fast_evaluator import evaluate_scene_a as evaluator
46         result = evaluator(raw, plan, settings['bandwidth'], settings['
capacity'],
47                             waits['task_cross_core_wait_cycles'], waits['
task_same_core_wait_cycles'])
48         stats = {'evaluations': [], 'evaluation_count': 1,
49                 'singlecore_makespan': result['makespan'], 'mode': '
fixed_whole_graph'}
50     else:
51         def progress(row):
52             print(row['candidate'], row['makespan'], flush=True)
53         plan, result, stats = solve_experimental(raw, settings, waits,
args.cores,
54             args.budget, args.seed, BASELINE_FEATURES, on_evaluation=
progress,
55             evaluator_backend=args.backend)
56         solve_seconds = time.perf_counter() - started
57         verification_seconds = None
58         if args.verify_final:
59             check_start = time.perf_counter()
60             checked = mod.evaluate_scene_a(raw, plan, settings['bandwidth'],
settings['capacity'],
61                 waits['task_cross_core_wait_cycles'], waits['
task_same_core_wait_cycles'])
62             if checked != result:
63                 raise RuntimeError('Full final result differs from official
replay')
64             verification_seconds = time.perf_counter() - check_start
65             write_json(args.output / (args.graph.stem + '_multicore_res.json'),
plan)
66             write_json(args.output / 'search.json', stats)
67             (args.output / 'evaluation.json.gz').write_bytes(gzip.compress(
68                 json.dumps(result, ensure_ascii=False, separators=(',', ':')).
encode(), mtime=0))
69             meta = {'algorithm': ADOPTED_POLICY, 'features': sorted(
BASELINE_FEATURES),
70                     'seed': args.seed, 'budget': args.budget, 'cores': args.cores,
71                     'backend': args.backend, 'python': platform.python_version(),
72                     'input_sha256': sha(args.graph.read_bytes()), 'config_sha256':
sha(args.config.read_bytes()),

```

```

73         'solve_seconds': solve_seconds, 'verification_seconds':
verification_seconds,
74         'total_seconds': time.perf_counter() - started,
75         'makespan': result['makespan'], 'added_copy_bytes': result['
data_movement_bytes']['added_copy_bytes'],
76         'singlecore_makespan': stats['singlecore_makespan'],
77         'speedup': stats['singlecore_makespan'] / result['makespan'],
78         'source_sha256': {p.name: sha(p.read_bytes()) for p in sorted(
Path(__file__).parent.glob('*.*.py'))}}
79     write_json(args.output / 'run.json', meta)
80     print(json.dumps({k: meta[k] for k in ('makespan', 'speedup', '
solve_seconds')}, ensure_ascii=False))
81
82
83 if __name__ == '__main__':
84     main()

```

A.2 问题二求解器

程序 2 问题二 r07 当前提交入口

```

1  """Current Q2 r07 submission: raw graph to standard solution, with
optional seed."""
2  import argparse, gzip, json, time
3  from pathlib import Path
4  from q1_io import ROOT, PROCESSED, sha, write_json
5  from q2_evaluator import load
6  from q2_reserve_fast import solve
7  from q2_single_reference import fixed_reference_fast
8
9
10 def main():
11     ap=argparse.ArgumentParser(description=__doc__)
12     ap.add_argument('graph',type=Path);ap.add_argument('-n','--cores',
required=True,type=int,choices=range(1,6))
13     ap.add_argument('--migration',type=Path)
14     ap.add_argument('--output',required=True,type=Path)
15     a=ap.parse_args()
16     assert not a.output.exists()
17     start=time.perf_counter();settings,delay,provenance=load()
18     raw=json.loads(a.graph.read_text(encoding='utf-8-sig'))
19     arm='r07_reserve_fast'
20     reference=fixed_reference_fast(raw,settings);migration_seconds=0.;
migration_stats=None;migration=None
21     if a.cores==1:
22         from singlecore_evaluate import build_singlecore_plan
23         plan=build_singlecore_plan(raw);result=reference;stats={'role':'
prescribed_single_reference','slots':0}
24     else:
25         if a.migration:
26             migration=json.loads(a.migration.read_text(encoding='utf-8-sig
'))
27         else:

```

```

28         from q1_io import official
29         from q1_experimental import solve_experimental
30         from q1_submit import BASELINE_FEATURES
31         waits=official().read_scene_a_config(str(PROCESSED/'data/
config.txt'))
32         t=time.perf_counter()
33         migration,_,migration_stats=solve_experimental(raw,settings,
waits,a.cores,12,0,BASELINE_FEATURES,evaluator_backend='counter')
34         migration_seconds=time.perf_counter()-t
35         plan,result,stats=solve(raw,settings,delay,provenance,a.cores,
migration)
36         assert stats['slots']==12 and stats['replay_equal']
37         a.output.mkdir(parents=True)
38         write_json(a.output/f'{a.graph.stem}_multicore_res.json',plan)
39         write_json(a.output/'search.json',stats)
40         if migration is not None:write_json(a.output/'migration_plan.json',
migration)
41         if migration_stats is not None:write_json(a.output/'migration_search.
json',migration_stats)
42         with gzip.open(a.output/'evaluation.json.gz','wt',encoding='utf-8') as
f:json.dump(result,f)
43         write_json(a.output/'run.json',dict(problem=2,algorithm=arm,cores=a.
cores,makespan=result['makespan'],
44                 speedup=reference['makespan']/result['makespan'],
added_copy_bytes=result['data_movement_bytes']['added_copy_bytes'],
45                 input_sha256=sha(a.graph.read_bytes()),solver_sha256=sha(
Path(__file__).with_name('q2_reserve_fast.py').read_bytes()),
46                 migration_sha256=sha(a.migration.read_bytes()) if a.
migration else None,provenance=provenance,
47                 migration_source='imported' if a.migration else '
generated_Q1_12_slots' if a.cores>1 else 'not_applicable',
48                 migration_generation_seconds=migration_seconds,
49                 elapsed_seconds=time.perf_counter()-start,timing_scope='
includes fresh fixed reference, B search, final replay and output;
includes seed generation only when --migration omitted'))
50         print(result['makespan'])
51
52
53 if __name__=='__main__':main()

```

A.3 问题三求解器

程序 3 问题三当前提交入口

```

1  """Graph-to-plan Q3 entry. Optional Q2 seed; cold mode computes adopted Q2
   first."""
2  import argparse,json,time
3  from pathlib import Path
4  from q1_io import PROCESSED,official,sha,write_json
5  from q3_solver import load,solve
6  from q3_run import dump_gz
7
8

```

```

9 def main(argv=None):
10     p=argparse.ArgumentParser(description=__doc__)
11     p.add_argument('graph',type=Path);p.add_argument('-n','--cores',type=
int,required=True,choices=range(1,6))
12     p.add_argument('--seed',type=Path);p.add_argument('--budget',type=int,
default=4)
13     p.add_argument('--output',type=Path,required=True)
14     a=p.parse_args(argv);assert a.budget>=0 and not a.output.exists()
15     settings,delay,cache,provenance=load();raw=json.loads(a.graph.
read_text(encoding='utf-8-sig'))
16     t=time.perf_counter()
17     if a.seed:seed=json.loads(a.seed.read_text(encoding='utf-8-sig'))
18     elif a.cores==1:
19         from singlecore_evaluate import build_singlecore_plan
20         seed=build_singlecore_plan(raw)
21     else:
22         from q1_experimental import solve_experimental
23         from q1_submit import BASELINE_FEATURES
24         from q2_reserve_fast import solve as solve_b
25         waits=official().read_scene_a_config(str(PROCESSED/'data/config.
txt'))
26         migration,_,_=solve_experimental(raw,settings,waits,a.cores,12,0,
BASELINE_FEATURES,evaluator_backend='counter')
27         seed,_,_=solve_b(raw,settings,delay,provenance,a.cores,migration)
28         if len(seed['core_schedules'])!=a.cores:raise ValueError('Seed core
count mismatch')
29         seed_seconds=time.perf_counter()-t
30         plan,r,anchor,stats=solve(raw,seed,settings,delay,cache,a.budget)
31         a.output.mkdir(parents=True)
32         write_json(a.output/f'{a.graph.stem}_multicore_res.json',plan)
33         write_json(a.output/'seed.json',seed);write_json(a.output/'search.json
',stats)
34         dump_gz(a.output/'evaluation.json.gz',r)
35         write_json(a.output/'run.json',dict(input_sha256=sha(a.graph.
read_bytes()),provenance=provenance,
36             seed_seconds=seed_seconds,seed_source='imported' if a.seed
else 'cold_Q2_r07',
37             makespan=r['makespan'],seed_12_makespan=anchor['makespan'],
cache=cache))
38         print(r['makespan'])
39
40
41 if __name__=='__main__':main()

```

B 问题一逐用例官方结果

每行对应一张图和一个核心数，共 500 行；加速比的分母为该图固定单核基准。额外搬运量为官方评价输出的逻辑字节数。

表 14 问题一全部算例的官方评价结果

算例	核数	Makespan	额外搬运量	加速比
case_001	1	233110	0	1.000000
case_001	2	116868	1152	1.994644
case_001	3	78545	2304	2.967853
case_001	4	58984	3456	3.952089
case_001	5	47502	4608	4.907372
case_002	1	261945	0	1.000000
case_002	2	138077	477696	1.897094
case_002	3	95931	532992	2.730556
case_002	4	74618	536064	3.510480
case_002	5	60651	536064	4.318890
case_003	1	602212	0	1.000000
case_003	2	462800	8565562	1.301236
case_003	3	378576	9989392	1.590729
case_003	4	329107	11891502	1.829836
case_003	5	294109	11891502	2.047581
case_004	1	392357	0	1.000000
case_004	2	199074	110592	1.970910
case_004	3	134043	221184	2.927098
case_004	4	104188	331776	3.765856
case_004	5	88341	442368	4.441392
case_005	1	95618	0	1.000000
case_005	2	91571	2718104	1.044195
case_005	3	88392	2736536	1.081749
case_005	4	87624	2736440	1.091231
case_005	5	87840	2736440	1.088547
case_006	1	82837	16400	1.000000
case_006	2	42379	56200	1.954671
case_006	3	32925	337092	2.515930
case_006	4	31129	533902	2.661088
case_006	5	31729	91264	2.610766
case_007	1	1012562	0	1.000000
case_007	2	507494	27648	1.995220
case_007	3	338094	55296	2.994913
case_007	4	254398	82944	3.980228
case_007	5	203834	110592	4.967581
case_008	1	487605	0	1.000000
case_008	2	244266	0	1.996205
case_008	3	163359	0	2.984868
case_008	4	123060	0	3.962335
case_008	5	100603	0	4.846824
case_009	1	153261	0	1.000000
case_009	2	92259	182950	1.661204

算例	核数	Makespan	额外搬运量	加速比
case_009	3	92004	193702	1.665808
case_009	4	92192	225586	1.662411
case_009	5	91903	214604	1.667639
case_010	1	86969	0	1.000000
case_010	2	45626	48384	1.906128
case_010	3	34518	395696	2.519526
case_010	4	30262	575880	2.873868
case_010	5	28842	958646	3.015360
case_011	1	184697	0	1.000000
case_011	2	92646	180224	1.993578
case_011	3	69625	1081344	2.652740
case_011	4	46880	540672	3.939782
case_011	5	47047	1630208	3.925798
case_012	1	59088	0	1.000000
case_012	2	29489	71816	2.003730
case_012	3	20230	43320	2.920811
case_012	4	15724	86400	3.757822
case_012	5	13406	53568	4.407579
case_013	1	263949	0	1.000000
case_013	2	132198	8192	1.996619
case_013	3	88351	16384	2.987504
case_013	4	66982	24576	3.940596
case_013	5	54363	32768	4.855306
case_014	1	17698626	71010720	1.000000
case_014	2	8909568	69998520	1.986474
case_014	3	6002519	84088488	2.948533
case_014	4	4543360	76510104	3.895493
case_014	5	3694137	88763688	4.791004
case_015	1	177362	180256	1.000000
case_015	2	90480	199944	1.960234
case_015	3	61018	298252	2.906716
case_015	4	48866	365824	3.629558
case_015	5	44068	331776	4.024735
case_016	1	7715523	0	1.000000
case_016	2	5440637	134500220	1.418129
case_016	3	4478897	134500220	1.722639
case_016	4	4266542	134500220	1.808379
case_016	5	4158373	134500220	1.855419
case_017	1	162810	22528	1.000000
case_017	2	81117	110100	2.007101
case_017	3	53799	196920	3.026264
case_017	4	41101	285984	3.961217
case_017	5	33081	359460	4.921556
case_018	1	1004367	1970176	1.000000
case_018	2	501235	2622524	2.003785
case_018	3	336388	4077600	2.985740
case_018	4	253676	4938752	3.959251

算例	核数	Makespan	额外搬运量	加速比
case_018	5	203806	5603348	4.928054
case_019	1	81144	0	1.000000
case_019	2	40136	42540	2.021726
case_019	3	27848	83520	2.913818
case_019	4	24465	285312	3.316738
case_019	5	23450	568740	3.460299
case_020	1	532746	0	1.000000
case_020	2	266580	0	1.998447
case_020	3	177950	0	2.993796
case_020	4	133704	0	3.984518
case_020	5	107650	0	4.948871
case_021	1	6567937	4501504	1.000000
case_021	2	3168548	50104320	2.072854
case_021	3	2124156	50071552	3.092022
case_021	4	1550324	4608000	4.236493
case_021	5	1249916	11845632	5.254703
case_022	1	56780	0	1.000000
case_022	2	28303	6912	2.006148
case_022	3	18978	13824	2.991885
case_022	4	14336	20736	3.960658
case_022	5	11479	24612	4.946424
case_023	1	122614	0	1.000000
case_023	2	61065	73728	2.007926
case_023	3	40682	207264	3.013962
case_023	4	30516	281280	4.018023
case_023	5	24940	280992	4.916359
case_024	1	2555343	0	1.000000
case_024	2	1803647	44243528	1.416764
case_024	3	1483888	44243528	1.722059
case_024	4	1413568	44243528	1.807726
case_024	5	1378992	44243528	1.853051
case_025	1	4863026	0	1.000000
case_025	2	2431840	27648	1.999731
case_025	3	1622164	55296	2.997863
case_025	4	1217444	82944	3.994456
case_025	5	975282	110592	4.986277
case_026	1	99203	0	1.000000
case_026	2	51839	218580	1.913675
case_026	3	42174	381752	2.352231
case_026	4	42089	217872	2.356982
case_026	5	43946	414504	2.257384
case_027	1	2363952	1400832	1.000000
case_027	2	1116984	1843200	2.116371
case_027	3	804739	19496960	2.937539
case_027	4	559196	3317760	4.227412
case_027	5	558753	5529600	4.230764
case_028	1	43836040	277412256	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_028	2	20808183	310682880	2.106673
case_028	3	13957556	310848768	3.140667
case_028	4	10676362	310848768	4.105897
case_028	5	9004662	286344264	4.868149
case_029	1	37555	0	1.000000
case_029	2	19740	0	1.902482
case_029	3	12743	0	2.947108
case_029	4	11028	0	3.405423
case_029	5	9383	0	4.002451
case_030	1	2808034	5519360	1.000000
case_030	2	1399339	5285020	2.006686
case_030	3	932367	8365156	3.011726
case_030	4	700342	10404864	4.009518
case_030	5	560306	11168828	5.011608
case_031	1	1414369	3491328	1.000000
case_031	2	712262	5250048	1.985743
case_031	3	473822	7544832	2.985022
case_031	4	356417	7926272	3.968298
case_031	5	284848	7931904	4.965346
case_032	1	41443	0	1.000000
case_032	2	20900	96344	1.982919
case_032	3	14009	96406	2.958313
case_032	4	10933	63656	3.790634
case_032	5	10280	80036	4.031420
case_033	1	1774196	4592640	1.000000
case_033	2	888544	6312520	1.996745
case_033	3	590697	7602264	3.003564
case_033	4	443447	7214688	4.000920
case_033	5	353592	8227976	5.017636
case_034	1	842996	1079832	1.000000
case_034	2	419167	1915392	2.011122
case_034	3	278067	2762240	3.031629
case_034	4	205314	3207748	4.105887
case_034	5	167930	3885104	5.019925
case_035	1	169571	0	1.000000
case_035	2	95406	0	1.777362
case_035	3	95782	42278	1.770385
case_035	4	94869	47410	1.787423
case_035	5	95983	83532	1.766677
case_036	1	931510	0	1.000000
case_036	2	466068	1152	1.998657
case_036	3	311345	2304	2.991890
case_036	4	233584	3456	3.987902
case_036	5	187182	4608	4.976493
case_037	1	210001	0	1.000000
case_037	2	105374	204800	1.992911
case_037	3	74038	1064960	2.836395

算例	核数	Makespan	额外搬运量	加速比
case_037	4	53320	614400	3.938503
case_037	5	43800	1925120	4.794543
case_038	1	1364555	5627904	1.000000
case_038	2	674500	6790656	2.023062
case_038	3	450730	7879736	3.027433
case_038	4	338163	8930344	4.035199
case_038	5	274658	9635364	4.968197
case_039	1	1435262	7929856	1.000000
case_039	2	708557	11550720	2.025613
case_039	3	482318	15020032	2.975759
case_039	4	395323	17604608	3.630606
case_039	5	431161	20480000	3.328831
case_040	1	253454	0	1.000000
case_040	2	120926	283200	2.095943
case_040	3	109777	642836	2.308808
case_040	4	109607	1022796	2.312389
case_040	5	108131	727212	2.343953
case_041	1	6003957	30898176	1.000000
case_041	2	2990398	33669120	2.007745
case_041	3	1991911	34851840	3.014169
case_041	4	1490603	39164928	4.027871
case_041	5	1201884	40673280	4.995455
case_042	1	1686740	6629376	1.000000
case_042	2	845091	6614304	1.995927
case_042	3	559018	8452672	3.017327
case_042	4	419432	8676192	4.021486
case_042	5	336330	10425184	5.015134
case_043	1	677656	49160	1.000000
case_043	2	353710	589446	1.915852
case_043	3	267112	2381570	2.536973
case_043	4	266590	2584974	2.541941
case_043	5	264401	2875646	2.562986
case_044	1	154407	4438112	1.000000
case_044	2	119062	5274528	1.296862
case_044	3	123085	5678624	1.254475
case_044	4	120120	5792192	1.285440
case_044	5	131771	6309600	1.171783
case_045	1	125614	0	1.000000
case_045	2	63059	0	1.992007
case_045	3	42601	0	2.948616
case_045	4	31878	0	3.940461
case_045	5	26164	0	4.801024
case_046	1	276455	3176288	1.000000
case_046	2	152419	5255872	1.813783
case_046	3	128115	4762304	2.157866
case_046	4	116069	5013664	2.381816
case_046	5	111389	4672800	2.481888

算例	核数	Makespan	额外搬运量	加速比
case_047	1	326508	0	1.000000
case_047	2	276535	4380996	1.180711
case_047	3	241969	5051902	1.349379
case_047	4	225654	5078720	1.446941
case_047	5	222583	5064318	1.466904
case_048	1	222220	0	1.000000
case_048	2	197496	1716280	1.125187
case_048	3	197613	1716280	1.124521
case_048	4	197613	1716280	1.124521
case_048	5	197613	1716280	1.124521
case_049	1	196276	0	1.000000
case_049	2	156968	2635492	1.250420
case_049	3	132866	4012736	1.477248
case_049	4	123251	4012736	1.592490
case_049	5	117489	4013760	1.670590
case_050	1	149690	0	1.000000
case_050	2	108323	51814	1.381886
case_050	3	104366	3416202	1.434279
case_050	4	98003	3448972	1.527402
case_050	5	99382	3424394	1.506208
case_051	1	607628	0	1.000000
case_051	2	429872	10225204	1.413509
case_051	3	352843	10225204	1.722092
case_051	4	336378	10225204	1.806384
case_051	5	328989	10225204	1.846955
case_052	1	180301	0	1.000000
case_052	2	90903	102912	1.983444
case_052	3	60160	181248	2.997025
case_052	4	45777	254976	3.938681
case_052	5	37846	333312	4.764070
case_053	1	1472686	5405536	1.000000
case_053	2	774757	5959552	1.900836
case_053	3	509253	6480220	2.891855
case_053	4	491606	10892964	2.995663
case_053	5	474708	20272004	3.102299
case_054	1	815051	0	1.000000
case_054	2	510200	6569360	1.597513
case_054	3	402111	6732340	2.026930
case_054	4	362394	11175330	2.249074
case_054	5	333071	11208098	2.447079
case_055	1	118835	69632	1.000000
case_055	2	56902	176664	2.088415
case_055	3	38966	168968	3.049710
case_055	4	32118	173568	3.699950
case_055	5	24831	280068	4.785752
case_056	1	277285	0	1.000000
case_056	2	231882	4095676	1.195802

算例	核数	Makespan	额外搬运量	加速比
case_056	3	199699	6006214	1.388515
case_056	4	189615	6006214	1.462358
case_056	5	186638	6006214	1.485684
case_057	1	59726	94216	1.000000
case_057	2	30047	182060	1.987753
case_057	3	20691	18228	2.886569
case_057	4	16471	32328	3.626131
case_057	5	14319	189060	4.171101
case_058	1	4539717	28696576	1.000000
case_058	2	2268326	33972224	2.001351
case_058	3	1532367	31514624	2.962552
case_058	4	1152464	42893312	3.939140
case_058	5	1002440	33316864	4.528667
case_059	1	829473	1691648	1.000000
case_059	2	369318	360448	2.245959
case_059	3	254823	4196352	3.255095
case_059	4	185216	1081344	4.478409
case_059	5	150782	4325376	5.501141
case_060	1	888789	2068520	1.000000
case_060	2	441525	2746508	2.012998
case_060	3	295983	3159296	3.002838
case_060	4	223107	3960200	3.983689
case_060	5	180081	3733888	4.935496
case_061	1	77191	0	1.000000
case_061	2	39298	34560	1.964248
case_061	3	28342	43008	2.723555
case_061	4	24024	70656	3.213079
case_061	5	21941	323072	3.518117
case_062	1	2854809	0	1.000000
case_062	2	1463796	3431424	1.950278
case_062	3	982412	3469824	2.905918
case_062	4	744188	3511296	3.836140
case_062	5	605685	3469824	4.713356
case_063	1	1070397	0	1.000000
case_063	2	552512	1903104	1.937328
case_063	3	371366	1903104	2.882324
case_063	4	283231	1915392	3.779237
case_063	5	229053	1920000	4.673141
case_064	1	19116	0	1.000000
case_064	2	19089	17994	1.001414
case_064	3	19089	17994	1.001414
case_064	4	19076	15692	1.002097
case_064	5	19083	7366	1.001729
case_065	1	53068	0	1.000000
case_065	2	26410	13984	2.009390
case_065	3	17964	27936	2.954130
case_065	4	13800	41904	3.845507

算例	核数	Makespan	额外搬运量	加速比
case_065	5	10949	66132	4.846835
case_066	1	359070	0	1.000000
case_066	2	242005	4846614	1.483730
case_066	3	176353	4877334	2.036087
case_066	4	162001	5725374	2.216468
case_066	5	140194	5725374	2.561237
case_067	1	63834834	264449408	1.000000
case_067	2	32203165	262601984	1.982253
case_067	3	21585693	247821184	2.957275
case_067	4	16358776	262601984	3.902177
case_067	5	13455316	262606080	4.744209
case_068	1	362095	0	1.000000
case_068	2	243395	4401532	1.487685
case_068	3	224168	4401532	1.615284
case_068	4	225518	4426108	1.605615
case_068	5	226325	4426684	1.599890
case_069	1	25144	0	1.000000
case_069	2	21862	514378	1.150124
case_069	3	21862	514378	1.150124
case_069	4	21862	514378	1.150124
case_069	5	24830	71044	1.012646
case_070	1	117300	0	1.000000
case_070	2	58498	129536	2.005197
case_070	3	39333	259072	2.982229
case_070	4	30080	333312	3.899601
case_070	5	25374	449024	4.622842
case_071	1	18919	0	1.000000
case_071	2	18901	93316	1.000952
case_071	3	18757	57220	1.008637
case_071	4	18449	155940	1.025476
case_071	5	18757	57220	1.008637
case_072	1	23897828	108613680	1.000000
case_072	2	11841342	110234400	2.018169
case_072	3	7929522	115280672	3.013779
case_072	4	5990048	120472240	3.989589
case_072	5	4830220	127451824	4.947565
case_073	1	10081626	141570432	1.000000
case_073	2	5168597	124388224	1.950554
case_073	3	3836078	131784832	2.628108
case_073	4	3249411	131636352	3.102601
case_073	5	3017992	124387200	3.340508
case_074	1	1682239	6070280	1.000000
case_074	2	831774	6869040	2.022471
case_074	3	551928	7438392	3.047932
case_074	4	414717	8323084	4.056354
case_074	5	335011	8036364	5.021444
case_075	1	1441809	0	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_075	2	1008737	12444746	1.429321
case_075	3	778382	12461130	1.852315
case_075	4	698816	12461130	2.063217
case_075	5	709725	12444746	2.031504
case_076	1	6185113	28280832	1.000000
case_076	2	3065214	31095808	2.017841
case_076	3	2050028	34332672	3.017087
case_076	4	1540201	37442560	4.015783
case_076	5	1236760	38841548	5.001062
case_077	1	495320	0	1.000000
case_077	2	248734	57344	1.991364
case_077	3	168976	114688	2.931304
case_077	4	127832	114688	3.874773
case_077	5	115141	225046	4.301856
case_078	1	2296143	12644568	1.000000
case_078	2	1179848	11466240	1.946135
case_078	3	840459	12294144	2.732011
case_078	4	621588	14093136	3.693995
case_078	5	635493	14093136	3.613168
case_079	1	31052251	32448512	1.000000
case_079	2	14956594	242368512	2.076158
case_079	3	10106197	40280064	3.072595
case_079	4	7609995	31555584	4.080456
case_079	5	6022007	50976768	5.156462
case_080	1	292138	548864	1.000000
case_080	2	145757	2707456	2.004281
case_080	3	100325	3047424	2.911916
case_080	4	111314	4190208	2.624450
case_080	5	113455	4571136	2.574924
case_081	1	151795	0	1.000000
case_081	2	76860	0	1.974954
case_081	3	51943	0	2.922338
case_081	4	39588	0	3.834369
case_081	5	30943	0	4.905633
case_082	1	721041	0	1.000000
case_082	2	497483	5740350	1.449378
case_082	3	378014	7216752	1.907445
case_082	4	328283	5723710	2.196401
case_082	5	312290	5703230	2.308883
case_083	1	1057993	13934688	1.000000
case_083	2	543456	25772480	1.946787
case_083	3	436908	19450976	2.421546
case_083	4	417384	19407232	2.534819
case_083	5	391729	19753696	2.700829
case_084	1	2507412	0	1.000000
case_084	2	1254320	27648	1.999021
case_084	3	836116	18432	2.998881

算例	核数	Makespan	额外搬运量	加速比
case_084	4	629028	0	3.986169
case_084	5	503472	0	4.980241
case_085	1	3187899	0	1.000000
case_085	2	2334501	34301210	1.365559
case_085	3	1626396	34301210	1.960100
case_085	4	1324330	34301210	2.407179
case_085	5	1169044	34301210	2.726928
case_086	1	118403	0	1.000000
case_086	2	102016	2674522	1.160632
case_086	3	97297	3334272	1.216923
case_086	4	98592	3359618	1.200939
case_086	5	99507	3359618	1.189896
case_087	1	3974071	7902720	1.000000
case_087	2	1987606	10142208	1.999426
case_087	3	1374901	11036160	2.890442
case_087	4	1046846	12607488	3.796233
case_087	5	833349	12487680	4.768796
case_088	1	243623	0	1.000000
case_088	2	163881	2935364	1.486585
case_088	3	153688	2932484	1.585179
case_088	4	153194	2957060	1.590291
case_088	5	151301	2932484	1.610188
case_089	1	693092	0	1.000000
case_089	2	346832	238720	1.998351
case_089	3	243614	477440	2.845042
case_089	4	178160	716160	3.890278
case_089	5	147432	954880	4.701096
case_090	1	1178363	21233592	1.000000
case_090	2	625263	18405960	1.884588
case_090	3	550183	25354368	2.141766
case_090	4	527825	19402440	2.232488
case_090	5	509204	19405512	2.314128
case_091	1	9186769	53947008	1.000000
case_091	2	4571759	57904096	2.009460
case_091	3	3070918	60928832	2.991538
case_091	4	2315897	69360288	3.966830
case_091	5	1880163	73234432	4.886156
case_092	1	4815554	66000096	1.000000
case_092	2	2412929	113053056	1.995730
case_092	3	2109464	112651936	2.282833
case_092	4	2037129	113053056	2.363893
case_092	5	1661234	66980256	2.898781
case_093	1	74205	0	1.000000
case_093	2	37827	8192	1.961694
case_093	3	25103	16384	2.956021
case_093	4	19779	24576	3.751706
case_093	5	15876	32768	4.674036

算例	核数	Makespan	额外搬运量	加速比
case_094	1	94892	0	1.000000
case_094	2	47498	106272	1.997810
case_094	3	32728	212544	2.899413
case_094	4	24495	318552	3.873933
case_094	5	21804	424800	4.352045
case_095	1	2084853	0	1.000000
case_095	2	1042787	36864	1.999309
case_095	3	695775	0	2.996447
case_095	4	524422	0	3.975525
case_095	5	420852	0	4.953886
case_096	1	84194	0	1.000000
case_096	2	43782	116620	1.923028
case_096	3	34182	661880	2.463109
case_096	4	27087	719200	3.108281
case_096	5	27053	661824	3.112187
case_097	1	11491509	10584064	1.000000
case_097	2	5247114	85295104	2.190063
case_097	3	3509566	85327872	3.274339
case_097	4	2622226	14221312	4.382349
case_097	5	2101905	22016000	5.467188
case_098	1	177430	0	1.000000
case_098	2	93465	98340	1.898358
case_098	3	79351	1479456	2.236015
case_098	4	79805	1479456	2.223294
case_098	5	73104	3282976	2.427090
case_099	1	1622999	5484544	1.000000
case_099	2	808380	6221840	2.007718
case_099	3	540127	8245256	3.004847
case_099	4	405336	9838592	4.004083
case_099	5	334609	10674180	4.850434
case_100	1	151223	0	1.000000
case_100	2	80622	16896	1.875704
case_100	3	58758	698388	2.573658
case_100	4	58793	534528	2.572126
case_100	5	59649	534528	2.535214

C 问题二逐用例官方结果

每行对应一张图和一个核心数，共 500 行。

表 15 问题二逐例执行周期与额外搬运量

算例	核数	执行周期	额外搬运（字节）	加速比
1	1	233110	0	1.000000
1	2	116868	1152	1.994644
1	3	78545	2304	2.967853
1	4	58984	3456	3.952089
1	5	47502	4608	4.907372
2	1	261945	0	1.000000
2	2	134224	216576	1.951551
2	3	90565	423936	2.892343
2	4	68574	377856	3.819888
2	5	55189	514560	4.746326
3	1	602212	0	1.000000
3	2	341181	6564470	1.765081
3	3	274348	9497892	2.195066
3	4	217774	7333194	2.765307
3	5	199233	7908308	3.022652
4	1	392357	0	1.000000
4	2	199074	110592	1.970910
4	3	134043	221184	2.927098
4	4	104188	331776	3.765856
4	5	88341	442368	4.441392
5	1	95618	0	1.000000
5	2	61371	882310	1.558032
5	3	54267	1461452	1.761992
5	4	52462	1495922	1.822614
5	5	50673	1688564	1.886961
6	1	82837	16400	1.000000
6	2	40815	17280	2.029572
6	3	30449	118176	2.720516
6	4	26843	203696	3.085981
6	5	27216	219928	3.043688
7	1	1012562	0	1.000000
7	2	507494	27648	1.995220
7	3	338094	55296	2.994913
7	4	254398	82944	3.980228
7	5	203834	110592	4.967581
8	1	487605	0	1.000000
8	2	244266	0	1.996205
8	3	163359	0	2.984868
8	4	123060	0	3.962335
8	5	100603	0	4.846824
9	1	153261	0	1.000000
9	2	90801	547342	1.687878
9	3	72361	737384	2.118006

算例	核数	执行周期	额外搬运（字节）	加速比
9	4	62725	838982	2.443380
9	5	56088	898504	2.732510
10	1	86969	0	1.000000
10	2	42106	311944	2.065478
10	3	30856	48384	2.818544
10	4	26340	483232	3.301784
10	5	22006	562432	3.952059
11	1	184697	0	1.000000
11	2	92646	180224	1.993578
11	3	64180	1660928	2.877797
11	4	46880	540672	3.939782
11	5	46194	2033664	3.998290
12	1	59088	0	1.000000
12	2	25535	94960	2.314000
12	3	18128	189892	3.259488
12	4	14948	39168	3.952903
12	5	12519	25344	4.719866
13	1	263949	0	1.000000
13	2	132198	8192	1.996619
13	3	88351	16384	2.987504
13	4	66982	24576	3.940596
13	5	54363	32768	4.855306
14	1	17698626	71010720	1.000000
14	2	8909568	69998520	1.986474
14	3	5951263	56526504	2.973928
14	4	4526637	59878296	3.909884
14	5	3663108	62443416	4.831587
15	1	177362	180256	1.000000
15	2	87493	109824	2.027156
15	3	61018	298252	2.906716
15	4	48866	365824	3.629558
15	5	42569	326144	4.166459
16	1	7715523	0	1.000000
16	2	4027025	404200	1.915936
16	3	2877849	341100	2.681003
16	4	2244367	605676	3.437728
16	5	2088965	444880	3.693467
17	1	162810	22528	1.000000
17	2	76658	87552	2.123849
17	3	49850	175104	3.265998
17	4	39668	259200	4.104316
17	5	33081	331776	4.921556
18	1	1004367	1970176	1.000000
18	2	502491	2638848	1.998776
18	3	337417	3971072	2.976634
18	4	253676	4938752	3.959251
18	5	205917	5320704	4.877533

算例	核数	执行周期	额外搬运（字节）	加速比
19	1	81144	0	1.000000
19	2	38506	354360	2.107308
19	3	27716	54336	2.927695
19	4	22276	762456	3.642665
19	5	18267	255068	4.442109
20	1	532746	0	1.000000
20	2	266580	0	1.998447
20	3	177950	0	2.993796
20	4	133704	0	3.984518
20	5	107650	0	4.948871
21	1	6567937	4501504	1.000000
21	2	3174339	33017856	2.069072
21	3	2090922	34854912	3.141168
21	4	1550324	4608000	4.236493
21	5	1250168	11431936	5.253644
22	1	56780	0	1.000000
22	2	27860	145236	2.038047
22	3	18978	13824	2.991885
22	4	14336	20736	3.960658
22	5	11479	20736	4.946424
23	1	122614	0	1.000000
23	2	60965	74016	2.011220
23	3	40682	207264	3.013962
23	4	30516	281280	4.018023
23	5	24940	280992	4.916359
24	1	2555343	0	1.000000
24	2	1335245	396856	1.913763
24	3	954537	332124	2.677050
24	4	748435	595068	3.414248
24	5	695849	432232	3.672267
25	1	4863026	0	1.000000
25	2	2431840	27648	1.999731
25	3	1622164	55296	2.997863
25	4	1217444	82944	3.994456
25	5	975282	110592	4.986277
26	1	99203	0	1.000000
26	2	51429	21936	1.928931
26	3	41688	95664	2.379654
26	4	42226	172128	2.349335
26	5	38198	835212	2.597073
27	1	2363952	1400832	1.000000
27	2	1120962	1744896	2.108860
27	3	744579	18481152	3.174884
27	4	559196	3317760	4.227412
27	5	499622	21954560	4.731481
28	1	43836040	277412256	1.000000
28	2	20517451	75738432	2.136525

算例	核数	执行周期	额外搬运（字节）	加速比
28	3	13687852	44350128	3.202551
28	4	10347838	74883456	4.236251
28	5	8286707	73953792	5.289923
29	1	37555	0	1.000000
29	2	18880	16388	1.989142
29	3	12743	0	2.947108
29	4	10168	49164	3.693450
29	5	8787	65552	4.273927
30	1	2808034	5519360	1.000000
30	2	1390496	1292288	2.019448
30	3	933720	8985600	3.007362
30	4	697351	6002688	4.026715
30	5	559882	7484416	5.015403
31	1	1414369	3491328	1.000000
31	2	712262	5250048	1.985743
31	3	473822	7544832	2.985022
31	4	360134	7808000	3.927341
31	5	284848	7931904	4.965346
32	1	41443	0	1.000000
32	2	17566	6216	2.359274
32	3	11910	12360	3.479681
32	4	10728	74980	3.863069
32	5	10414	12360	3.979547
33	1	1774196	4592640	1.000000
33	2	889132	6452256	1.995425
33	3	596821	7053888	2.972744
33	4	443447	7214688	4.000920
33	5	361176	7894656	4.912275
34	1	842996	1079832	1.000000
34	2	415251	943616	2.030088
34	3	270947	2288640	3.111295
34	4	206454	3059200	4.083215
34	5	167389	3463168	5.036149
35	1	169571	0	1.000000
35	2	93692	801138	1.809877
35	3	75717	1276568	2.239537
35	4	63767	1505978	2.659228
35	5	58308	1058710	2.908194
36	1	931510	0	1.000000
36	2	466068	1152	1.998657
36	3	311345	2304	2.991890
36	4	233584	3456	3.987902
36	5	187182	4608	4.976493
37	1	210001	0	1.000000
37	2	105374	204800	1.992911
37	3	74038	573440	2.836395
37	4	53320	614400	3.938503

算例	核数	执行周期	额外搬运（字节）	加速比
37	5	43013	942080	4.882268
38	1	1364555	5627904	1.000000
38	2	674500	6790656	2.023062
38	3	449590	8107008	3.035110
38	4	338182	8954880	4.034972
38	5	275278	9592320	4.957007
39	1	1435262	7929856	1.000000
39	2	708430	9166848	2.025976
39	3	478013	11911168	3.002559
39	4	368961	14536704	3.890010
39	5	338925	14536704	4.234748
40	1	253454	0	1.000000
40	2	134642	1227916	1.882429
40	3	105492	1985768	2.402590
40	4	89539	1836692	2.830655
40	5	78938	1915296	3.210798
41	1	6003957	30898176	1.000000
41	2	2990398	33669120	2.007745
41	3	1991911	34851840	3.014169
41	4	1490603	39164928	4.027871
41	5	1196379	40630272	5.018441
42	1	1686740	6629376	1.000000
42	2	840111	7262496	2.007758
42	3	570685	8474176	2.955641
42	4	430510	8605536	3.918004
42	5	350713	9936736	4.809460
43	1	677656	49160	1.000000
43	2	349881	3210754	1.936819
43	3	250900	4852554	2.700901
43	4	207895	5894554	3.259607
43	5	199041	6609206	3.404605
44	1	154407	4438112	1.000000
44	2	54510	886880	2.832636
44	3	44484	891584	3.471068
44	4	49989	1389344	3.088820
44	5	43028	655488	3.588524
45	1	125614	0	1.000000
45	2	63059	0	1.992007
45	3	42601	0	2.948616
45	4	31878	0	3.940461
45	5	26164	0	4.801024
46	1	276455	3176288	1.000000
46	2	112410	942688	2.459345
46	3	96500	2958528	2.864819
46	4	71871	2098976	3.846545
46	5	77698	2660736	3.558071
47	1	326508	0	1.000000

算例	核数	执行周期	额外搬运（字节）	加速比
47	2	197413	1264006	1.653934
47	3	156690	1857324	2.083783
47	4	143256	2256146	2.279192
47	5	135565	2575320	2.408498
48	1	222220	0	1.000000
48	2	150049	467846	1.480983
48	3	114626	1177612	1.938653
48	4	114384	1270930	1.942754
48	5	101641	1273560	2.186322
49	1	196276	0	1.000000
49	2	126174	1027062	1.555598
49	3	83711	2408560	2.344686
49	4	75814	2607184	2.588915
49	5	76846	2907980	2.554147
50	1	149690	0	1.000000
50	2	87435	1022694	1.712015
50	3	60421	1831624	2.477450
50	4	57912	2053712	2.584784
50	5	57017	1337182	2.625357
51	1	607628	0	1.000000
51	2	319230	394084	1.903418
51	3	228581	328736	2.658261
51	4	183794	591064	3.306027
51	5	170016	427456	3.573946
52	1	180301	0	1.000000
52	2	90903	102912	1.983444
52	3	60160	181248	2.997025
52	4	45777	254976	3.938681
52	5	37846	333312	4.764070
53	1	1472686	5405536	1.000000
53	2	723016	4631434	2.036865
53	3	495435	6090084	2.972511
53	4	394500	8679750	3.733044
53	5	357068	9823040	4.124385
54	1	815051	0	1.000000
54	2	465795	2620302	1.749806
54	3	359793	4112418	2.265333
54	4	274779	7279264	2.966206
54	5	257061	8197794	3.170652
55	1	118835	69632	1.000000
55	2	52737	16384	2.253352
55	3	35943	74240	3.306207
55	4	31281	115712	3.798951
55	5	24831	173568	4.785752
56	1	277285	0	1.000000
56	2	172836	3326338	1.604324
56	3	122295	3278908	2.267345

算例	核数	执行周期	额外搬运（字节）	加速比
56	4	115731	3685886	2.395944
56	5	117134	4124830	2.367246
57	1	59726	94216	1.000000
57	2	29703	22304	2.010773
57	3	20565	14112	2.904255
57	4	16345	28224	3.654084
57	5	14028	14400	4.257628
58	1	4539717	28696576	1.000000
58	2	2266495	29548544	2.002968
58	3	1538252	30400512	2.951218
58	4	1141935	31551488	3.975460
58	5	1000800	32104448	4.536088
59	1	829473	1691648	1.000000
59	2	369318	360448	2.245959
59	3	254073	1171456	3.264703
59	4	185216	1081344	4.478409
59	5	150782	2072576	5.501141
60	1	888789	2068520	1.000000
60	2	441525	2746508	2.012998
60	3	295983	3159296	3.002838
60	4	223107	3960200	3.983689
60	5	180081	3733888	4.935496
61	1	77191	0	1.000000
61	2	39298	34560	1.964248
61	3	28342	43008	2.723555
61	4	23892	62208	3.230830
61	5	18840	110592	4.097187
62	1	2854809	0	1.000000
62	2	1431735	2347008	1.993951
62	3	958567	2268672	2.978205
62	4	722454	2603520	3.951544
62	5	579973	2746368	4.922314
63	1	1070397	0	1.000000
63	2	536070	1049088	1.996749
63	3	358763	1262592	2.983577
63	4	270138	1420800	3.962408
63	5	218917	1539072	4.889511
64	1	19116	0	1.000000
64	2	14130	47620	1.352866
64	3	13074	41446	1.462139
64	4	10160	130956	1.881496
64	5	11391	159314	1.678167
65	1	53068	0	1.000000
65	2	25517	13968	2.079712
65	3	17964	27936	2.954130
65	4	13800	41904	3.845507
65	5	11212	55872	4.733143

算例	核数	执行周期	额外搬运（字节）	加速比
66	1	359070	0	1.000000
66	2	203504	1653418	1.764437
66	3	146357	2500738	2.453385
66	4	116226	3806464	3.089412
66	5	106067	3979388	3.385313
67	1	63834834	264449408	1.000000
67	2	32040130	215994240	1.992340
67	3	21255269	159238912	3.003248
67	4	16409509	226600576	3.890112
67	5	13461153	229484544	4.742152
68	1	362095	0	1.000000
68	2	196346	1624576	1.844168
68	3	180284	2355384	2.008470
68	4	156990	2646854	2.306484
68	5	158423	5510878	2.285621
69	1	25144	0	1.000000
69	2	17030	364198	1.476453
69	3	14179	325098	1.773327
69	4	15620	361896	1.609731
69	5	13147	433008	1.912528
70	1	117300	0	1.000000
70	2	54493	129536	2.152570
70	3	39333	259072	2.982229
70	4	30080	333312	3.899601
70	5	25374	449024	4.622842
71	1	18919	0	1.000000
71	2	12313	171622	1.536506
71	3	9867	250696	1.917401
71	4	10732	223406	1.762859
71	5	10542	307214	1.794631
72	1	23897828	108613680	1.000000
72	2	11836023	113724192	2.019076
72	3	7941294	120851232	3.009312
72	4	5965868	123071264	4.005759
72	5	4826156	77245216	4.951731
73	1	10081626	141570432	1.000000
73	2	5294770	86179712	1.904073
73	3	3805931	134138624	2.648925
73	4	3295138	137012864	3.059546
73	5	2848898	94477312	3.538781
74	1	1682239	6070280	1.000000
74	2	814840	3776512	2.064502
74	3	542996	5619712	3.098069
74	4	408908	6623232	4.113979
74	5	328499	7151616	5.120987
75	1	1441809	0	1.000000
75	2	754663	4745798	1.910534

算例	核数	执行周期	额外搬运（字节）	加速比
75	3	557783	12507404	2.584892
75	4	462730	11096688	3.115875
75	5	378054	12427608	3.813765
76	1	6185113	28280832	1.000000
76	2	3065214	31095808	2.017841
76	3	2046322	17912832	3.022551
76	4	1527039	19563520	4.050396
76	5	1227673	22311936	5.038079
77	1	495320	0	1.000000
77	2	248734	57344	1.991364
77	3	168976	114688	2.931304
77	4	127832	114688	3.874773
77	5	114837	172032	4.313244
78	1	2296143	12644568	1.000000
78	2	1050081	2782080	2.186634
78	3	718909	6965760	3.193927
78	4	540759	6950784	4.246148
78	5	470359	10766304	4.881682
79	1	31052251	32448512	1.000000
79	2	15118882	194232320	2.053872
79	3	10181692	39960576	3.049812
79	4	7620379	31801344	4.074896
79	5	6063012	34084864	5.121588
80	1	292138	548864	1.000000
80	2	145825	1675264	2.003346
80	3	100325	3047424	2.911916
80	4	90584	2666496	3.225051
80	5	89345	3047424	3.269774
81	1	151795	0	1.000000
81	2	76860	0	1.974954
81	3	51943	0	2.922338
81	4	39588	0	3.834369
81	5	30943	0	4.905633
82	1	721041	0	1.000000
82	2	407535	3389668	1.769274
82	3	288553	5338828	2.498817
82	4	235778	6420240	3.058135
82	5	206503	7708408	3.491673
83	1	1057993	13934688	1.000000
83	2	442847	5589600	2.389071
83	3	290664	5739712	3.639918
83	4	220741	6471456	4.792916
83	5	201107	7741824	5.260846
84	1	2507412	0	1.000000
84	2	1175864	32004864	2.132400
84	3	836220	0	2.998508
84	4	629028	0	3.986169

算例	核数	执行周期	额外搬运（字节）	加速比
84	5	503472	0	4.980241
85	1	3187899	0	1.000000
85	2	1843091	17607566	1.729648
85	3	1276351	31680716	2.497666
85	4	995687	27902578	3.201708
85	5	890475	38398484	3.579998
86	1	118403	0	1.000000
86	2	74609	1019206	1.586980
86	3	63229	2197292	1.872606
86	4	62299	2353614	1.900560
86	5	60402	2265748	1.960250
87	1	3974071	7902720	1.000000
87	2	1987606	10142208	1.999426
87	3	1374901	11036160	2.890442
87	4	1046846	12607488	3.796233
87	5	833349	12487680	4.768796
88	1	243623	0	1.000000
88	2	132639	1032234	1.836737
88	3	122079	1572244	1.995618
88	4	112565	2028164	2.164287
88	5	112833	2034888	2.159147
89	1	693092	0	1.000000
89	2	329513	1467520	2.103383
89	3	225452	2429184	3.074233
89	4	165365	1827200	4.191286
89	5	133148	2186320	5.205426
90	1	1178363	21233592	1.000000
90	2	470442	3196848	2.504800
90	3	342250	5523456	3.442989
90	4	294798	7360608	3.997188
90	5	267434	7981056	4.406182
91	1	9186769	53947008	1.000000
91	2	4532609	52032556	2.026817
91	3	3042680	60113508	3.019302
91	4	2314443	67112896	3.969322
91	5	1872567	71035764	4.905976
92	1	4815554	66000096	1.000000
92	2	2017778	10740320	2.386563
92	3	1348585	21856448	3.570820
92	4	1018849	18053920	4.726465
92	5	826308	19945856	5.827795
93	1	74205	0	1.000000
93	2	37827	8192	1.961694
93	3	25103	16384	2.956021
93	4	19779	24576	3.751706
93	5	15876	32768	4.674036
94	1	94892	0	1.000000

算例	核数	执行周期	额外搬运（字节）	加速比
94	2	47498	106272	1.997810
94	3	32728	212544	2.899413
94	4	24668	318528	3.846765
94	5	21804	424800	4.352045
95	1	2084853	0	1.000000
95	2	1042890	0	1.999111
95	3	695775	0	2.996447
95	4	524422	0	3.975525
95	5	420852	0	4.953886
96	1	84194	0	1.000000
96	2	42443	346520	1.983696
96	3	30876	123264	2.726843
96	4	24217	234168	3.476649
96	5	22137	448896	3.803316
97	1	11491509	10584064	1.000000
97	2	5153667	2490368	2.229773
97	3	3444086	82444288	3.336592
97	4	2622394	14221312	4.382068
97	5	2102348	21311488	5.466036
98	1	177430	0	1.000000
98	2	89620	0	1.979804
98	3	65476	784240	2.709848
98	4	53735	961274	3.301945
98	5	47355	1166870	3.746806
99	1	1622999	5484544	1.000000
99	2	807983	5906432	2.008704
99	3	540063	6479872	3.005203
99	4	405336	9838592	4.004083
99	5	332266	7084052	4.884638
100	1	151223	0	1.000000
100	2	76850	621700	1.967768
100	3	57811	72192	2.615817
100	4	49623	1207894	3.047438
100	5	53383	1599352	2.832793

D 问题三逐用例成对结果

下列长表依次列出无 L2、固定方案 L2 与优化方案 L2 的执行周期，以及同配置缓存统计；三种周期均由题目评价程序给出，缓存统计由保存的事件账本复核。

表 16 问题三逐用例执行周期

用例	核数	无 L2	同方案 L2	优化 L2
1	1	233110	233110	233110
1	2	116868	116868	116868
1	3	78545	78545	78545
1	4	58984	58984	58984
1	5	47502	47502	47502
2	1	261945	261945	261945
2	2	134224	134224	134068
2	3	90565	90565	90565
2	4	68574	68574	68574
2	5	55189	55189	55189
3	1	602212	602212	602212
3	2	341181	340455	340046
3	3	274348	265160	265160
3	4	217774	210167	210167
3	5	199233	187997	187997
4	1	392357	392357	392357
4	2	199074	199074	199074
4	3	134043	134043	134043
4	4	104188	104188	104188
4	5	88341	88341	88341
5	1	95618	95618	95618
5	2	61371	60812	60812
5	3	54267	52965	52658
5	4	52462	49060	49060
5	5	50673	48259	48259
6	1	82837	82837	82837
6	2	40815	40815	40815
6	3	30449	30449	30449
6	4	26843	26843	26843
6	5	27216	27216	27216
7	1	1012562	1012562	1012562
7	2	507494	507494	507494
7	3	338094	338094	338094
7	4	254398	254398	254398
7	5	203834	203834	203834
8	1	487605	487605	487605
8	2	244266	244266	244266
8	3	163359	163359	163359
8	4	123060	123060	123060
8	5	100603	100603	100603
9	1	153261	153261	153261
9	2	90801	90800	90800

用例	核数	无 L2	同方案 L2	优化 L2
9	3	72361	72333	72333
9	4	62725	60949	60949
9	5	56088	53324	53324
10	1	86969	86969	86969
10	2	42106	42069	41842
10	3	30856	30856	30856
10	4	26340	25769	25583
10	5	22006	21371	21371
11	1	184697	184697	184697
11	2	92646	92646	92646
11	3	64180	64180	63580
11	4	46880	46880	46880
11	5	46194	43952	43594
12	1	59088	59088	59088
12	2	25535	25535	25535
12	3	18128	18128	17926
12	4	14948	14948	13663
12	5	12519	12460	12460
13	1	263949	263949	263949
13	2	132198	132198	132198
13	3	88351	88351	88351
13	4	66982	66982	66982
13	5	54363	54363	54363
14	1	17698626	17689602	17689602
14	2	8909568	8904096	8904096
14	3	5951263	5936435	5929070
14	4	4526637	4510849	4476730
14	5	3663108	3647377	3607077
15	1	177362	177362	177362
15	2	87493	87493	87493
15	3	61018	61018	61018
15	4	48866	48792	48792
15	5	42569	42569	42569
16	1	7715523	7715523	7715523
16	2	4027025	4026193	4026193
16	3	2877849	2877209	2877209
16	4	2244367	2240364	2240364
16	5	2088965	2086755	2086755
17	1	162810	162810	162810
17	2	76658	76501	76501
17	3	49850	49801	49801
17	4	39668	39668	39668
17	5	33081	33081	33081
18	1	1004367	1003110	1003110
18	2	502491	500447	500447
18	3	337417	335889	335889
18	4	253676	253384	253384

用例	核数	无 L2	同方案 L2	优化 L2
18	5	205917	203609	203609
19	1	81144	81144	81144
19	2	38506	38506	37982
19	3	27716	27716	27716
19	4	22276	21470	21470
19	5	18267	18126	18009
20	1	532746	532746	532746
20	2	266580	266580	266580
20	3	177950	177950	177950
20	4	133704	133704	133704
20	5	107650	107650	107650
21	1	6567937	6558837	6558837
21	2	3174339	3174060	3174060
21	3	2090922	2090922	2090922
21	4	1550324	1550324	1550324
21	5	1250168	1249380	1249380
22	1	56780	56780	56780
22	2	27860	27860	27860
22	3	18978	18978	18978
22	4	14336	14336	14336
22	5	11479	11479	11479
23	1	122614	122614	122614
23	2	60965	60965	60965
23	3	40682	40682	40682
23	4	30516	30516	30516
23	5	24940	24940	24940
24	1	2555343	2555343	2555343
24	2	1335245	1334413	1334413
24	3	954537	953897	953897
24	4	748435	744432	744432
24	5	695849	693639	693639
25	1	4863026	4863026	4863026
25	2	2431840	2431840	2431840
25	3	1622164	1622164	1622164
25	4	1217444	1217444	1217444
25	5	975282	975282	975282
26	1	99203	99203	99203
26	2	51429	51429	51429
26	3	41688	41688	41688
26	4	42226	42208	42208
26	5	38198	37332	36689
27	1	2363952	2359515	2359515
27	2	1120962	1120962	1120962
27	3	744579	744579	744579
27	4	559196	559196	559196
27	5	499622	494470	494412
28	1	43836040	43836040	43836040

用例	核数	无 L2	同方案 L2	优化 L2
28	2	20517451	20505375	20505375
28	3	13687852	13687122	13686058
28	4	10347838	10168619	10168619
28	5	8286707	8179833	8179833
29	1	37555	37555	37555
29	2	18880	18880	18880
29	3	12743	12743	12743
29	4	10168	10168	10168
29	5	8787	8787	8787
30	1	2808034	2807891	2807891
30	2	1390496	1390292	1390292
30	3	933720	933018	933018
30	4	697351	697321	697321
30	5	559882	559862	559862
31	1	1414369	1414159	1414159
31	2	712262	711649	711649
31	3	473822	473052	473052
31	4	360134	359705	359705
31	5	284848	284820	284820
32	1	41443	41443	41443
32	2	17566	17566	17495
32	3	11910	11910	11910
32	4	10728	10728	10728
32	5	10414	10384	10384
33	1	1774196	1774080	1774080
33	2	889132	887756	887756
33	3	596821	593725	593725
33	4	443447	442667	442667
33	5	361176	360597	360597
34	1	842996	842733	842733
34	2	415251	415212	415212
34	3	270947	270017	266836
34	4	206454	203915	202625
34	5	167389	165377	162156
35	1	169571	169571	169571
35	2	93692	93692	93692
35	3	75717	75628	75176
35	4	63767	63250	63250
35	5	58308	57848	57848
36	1	931510	931510	931510
36	2	466068	466068	466068
36	3	311345	311345	311345
36	4	233584	233584	233584
36	5	187182	187182	187182
37	1	210001	210001	210001
37	2	105374	105374	105374
37	3	74038	74038	74038

用例	核数	无 L2	同方案 L2	优化 L2
37	4	53320	53320	53320
37	5	43013	43013	43013
38	1	1364555	1363312	1363312
38	2	674500	674382	674382
38	3	449590	449048	449048
38	4	338182	337550	337550
38	5	275278	272573	272573
39	1	1435262	1435262	1435262
39	2	708430	706188	706188
39	3	478013	477811	477811
39	4	368961	367059	355737
39	5	338925	317103	317103
40	1	253454	253454	253454
40	2	134642	134149	134149
40	3	105492	105156	105156
40	4	89539	87558	87558
40	5	78938	77963	77963
41	1	6003957	6001575	6001575
41	2	2990398	2988290	2988290
41	3	1991911	1989953	1989953
41	4	1490603	1487676	1487676
41	5	1196379	1193054	1193054
42	1	1686740	1675633	1675633
42	2	840111	839453	839453
42	3	570685	567481	567481
42	4	430510	425563	425563
42	5	350713	348878	348878
43	1	677656	677656	677656
43	2	349881	349668	349668
43	3	250900	250731	250728
43	4	207895	204194	204135
43	5	199041	184463	182238
44	1	154407	107814	107814
44	2	54510	52355	51266
44	3	44484	42915	40954
44	4	49989	44212	41362
44	5	43028	39788	39677
45	1	125614	125614	125614
45	2	63059	63059	63059
45	3	42601	42601	42601
45	4	31878	31878	31878
45	5	26164	26164	26164
46	1	276455	244994	244994
46	2	112410	112410	112410
46	3	96500	93498	92335
46	4	71871	69926	66914
46	5	77698	71867	69923

用例	核数	无 L2	同方案 L2	优化 L2
47	1	326508	326508	326508
47	2	197413	197408	197408
47	3	156690	156593	156593
47	4	143256	142699	142699
47	5	135565	134070	134070
48	1	222220	222220	222220
48	2	150049	150049	150049
48	3	114626	114531	114531
48	4	114384	114072	114072
48	5	101641	101570	101570
49	1	196276	196276	196276
49	2	126174	126174	126174
49	3	83711	83597	83597
49	4	75814	75141	75141
49	5	76846	74748	74748
50	1	149690	149690	149690
50	2	87435	87422	87422
50	3	60421	60008	60008
50	4	57912	57018	57012
50	5	57017	56411	56411
51	1	607628	607628	607628
51	2	319230	318398	318398
51	3	228581	227941	227941
51	4	183794	179791	179791
51	5	170016	167806	167806
52	1	180301	180301	180301
52	2	90903	90903	90903
52	3	60160	60160	60160
52	4	45777	45777	45777
52	5	37846	37846	37846
53	1	1472686	1428109	1428109
53	2	723016	723023	723023
53	3	495435	495119	495119
53	4	394500	388744	388744
53	5	357068	343404	343404
54	1	815051	815051	815051
54	2	465795	465795	465795
54	3	359793	359773	359773
54	4	274779	273124	272727
54	5	257061	252980	252980
55	1	118835	118835	118835
55	2	52737	52632	52632
55	3	35943	35943	35943
55	4	31281	31281	31281
55	5	24831	24831	24831
56	1	277285	277285	277285
56	2	172836	172843	172799

用例	核数	无 L2	同方案 L2	优化 L2
56	3	122295	122170	122170
56	4	115731	114771	114771
56	5	117134	115146	115146
57	1	59726	59726	59726
57	2	29703	29703	29703
57	3	20565	20565	20565
57	4	16345	16345	16345
57	5	14028	14028	14028
58	1	4539717	4539717	4539717
58	2	2266495	2264017	2264017
58	3	1538252	1538118	1538118
58	4	1141935	1129512	1128131
58	5	1000800	960493	960493
59	1	829473	815938	815938
59	2	369318	369318	369318
59	3	254073	254073	253986
59	4	185216	185216	185216
59	5	150782	150736	150736
60	1	888789	887801	887801
60	2	441525	441112	441112
60	3	295983	295334	295334
60	4	223107	222447	222447
60	5	180081	179744	179744
61	1	77191	77191	77191
61	2	39298	39272	39272
61	3	28342	28316	28316
61	4	23892	23892	23892
61	5	18840	18834	18497
62	1	2854809	2854809	2854809
62	2	1431735	1431735	1431735
62	3	958567	958567	955896
62	4	722454	722454	722454
62	5	579973	579973	579973
63	1	1070397	1070397	1070397
63	2	536070	536070	536070
63	3	358763	358763	358763
63	4	270138	270138	270138
63	5	218917	218917	218917
64	1	19116	19116	19116
64	2	14130	14130	14130
64	3	13074	13074	13074
64	4	10160	10130	10130
64	5	11391	11356	11356
65	1	53068	53068	53068
65	2	25517	25411	25411
65	3	17964	17964	17964
65	4	13800	13800	13800

用例	核数	无 L2	同方案 L2	优化 L2
65	5	11212	11212	11212
66	1	359070	359070	359070
66	2	203504	203504	203504
66	3	146357	145116	145116
66	4	116226	112359	112359
66	5	106067	100249	100249
67	1	63834834	63834834	63834834
67	2	32040130	32217720	32217720
67	3	21255269	20814755	20814755
67	4	16409509	16150583	16150583
67	5	13461153	13133609	13133609
68	1	362095	362095	362095
68	2	196346	196325	196325
68	3	180284	179131	179131
68	4	156990	153012	153012
68	5	158423	143687	143687
69	1	25144	25144	25144
69	2	17030	16941	16940
69	3	14179	14019	13997
69	4	15620	15156	15156
69	5	13147	12656	12584
70	1	117300	117300	117300
70	2	54493	54493	53981
70	3	39333	39333	39333
70	4	30080	30080	30080
70	5	25374	25264	25264
71	1	18919	18919	18919
71	2	12313	12271	12271
71	3	9867	9692	9534
71	4	10732	10334	10214
71	5	10542	10249	10249
72	1	23897828	23890756	23890756
72	2	11836023	11826643	11826643
72	3	7941294	7929713	7929713
72	4	5965868	5953384	5953384
72	5	4826156	4803888	4803888
73	1	10081626	10081626	10081626
73	2	5294770	5154759	5154759
73	3	3805931	3497998	3497998
73	4	3295138	2862419	2862419
73	5	2848898	2661730	2578536
74	1	1682239	1678120	1678120
74	2	814840	813116	813116
74	3	542996	542102	542102
74	4	408908	408908	408908
74	5	328499	328279	328279
75	1	1441809	1441809	1441809

用例	核数	无 L2	同方案 L2	优化 L2
75	2	754663	754663	754663
75	3	557783	554461	554461
75	4	462730	457083	457083
75	5	378054	364732	364732
76	1	6185113	6183098	6183098
76	2	3065214	3063843	3063843
76	3	2046322	2045754	2045044
76	4	1527039	1526713	1526713
76	5	1227673	1226487	1226487
77	1	495320	495320	495320
77	2	248734	248734	248734
77	3	168976	168976	168976
77	4	127832	127591	127591
77	5	114837	114837	114837
78	1	2296143	2296143	2296143
78	2	1050081	1049840	1049840
78	3	718909	715241	711932
78	4	540759	532165	532165
78	5	470359	435870	435870
79	1	31052251	31052251	31052251
79	2	15118882	15118882	15116703
79	3	10181692	10175315	10175315
79	4	7620379	7612680	7612680
79	5	6063012	6062122	6062082
80	1	292138	291826	291826
80	2	145825	145636	145636
80	3	100325	100325	100325
80	4	90584	89925	89925
80	5	89345	85328	78076
81	1	151795	151795	151795
81	2	76860	76860	76860
81	3	51943	51943	51943
81	4	39588	39588	39588
81	5	30943	30943	30943
82	1	721041	721041	721041
82	2	407535	407535	407508
82	3	288553	287410	287410
82	4	235778	232569	232569
82	5	206503	200705	200705
83	1	1057993	946145	946145
83	2	442847	434269	434269
83	3	290664	290640	290640
83	4	220741	220741	220741
83	5	201107	194079	194079
84	1	2507412	2507412	2507412
84	2	1175864	1176757	1176657
84	3	836220	836220	836220

用例	核数	无 L2	同方案 L2	优化 L2
84	4	629028	629028	629028
84	5	503472	503472	503472
85	1	3187899	3187899	3187899
85	2	1843091	1842465	1842465
85	3	1276351	1264859	1264508
85	4	995687	989911	989911
85	5	890475	846640	846640
86	1	118403	118403	118403
86	2	74609	74274	74274
86	3	63229	61856	61671
86	4	62299	59945	59945
86	5	60402	57399	57355
87	1	3974071	3969779	3969779
87	2	1987606	1984150	1984150
87	3	1374901	1372451	1372451
87	4	1046846	1043492	1043492
87	5	833349	829669	829669
88	1	243623	243623	243623
88	2	132639	132093	132093
88	3	122079	119274	119274
88	4	112565	111170	109290
88	5	112833	109702	109702
89	1	693092	693092	693092
89	2	329513	329513	329513
89	3	225452	225450	225450
89	4	165365	165365	165365
89	5	133148	133139	133139
90	1	1178363	1178363	1178363
90	2	470442	462677	456438
90	3	342250	322129	309173
90	4	294798	259575	258592
90	5	267434	239128	227333
91	1	9186769	9168820	9168820
91	2	4532609	4531465	4531465
91	3	3042680	3029129	3029129
91	4	2314443	2289607	2289607
91	5	1872567	1837577	1837577
92	1	4815554	4333631	4333631
92	2	2017778	2013086	2013056
92	3	1348585	1349050	1345074
92	4	1018849	1013827	1005380
92	5	826308	807785	801052
93	1	74205	74205	74205
93	2	37827	37827	37827
93	3	25103	25103	25103
93	4	19779	19779	19779
93	5	15876	15876	15876

用例	核数	无 L2	同方案 L2	优化 L2
94	1	94892	94892	94892
94	2	47498	47498	47498
94	3	32728	32728	32728
94	4	24668	24668	24668
94	5	21804	21282	21282
95	1	2084853	2084853	2084853
95	2	1042890	1042890	1042890
95	3	695775	695775	695775
95	4	524422	524422	524422
95	5	420852	420852	420852
96	1	84194	84194	84194
96	2	42443	42426	42426
96	3	30876	30876	30876
96	4	24217	24217	24217
96	5	22137	21973	21973
97	1	11491509	11491509	11491509
97	2	5153667	5153650	5153650
97	3	3444086	3444086	3444086
97	4	2622394	2622413	2622413
97	5	2102348	2102250	2102250
98	1	177430	177430	177430
98	2	89620	89620	89620
98	3	65476	65195	65195
98	4	53735	53405	53405
98	5	47355	45200	45200
99	1	1622999	1618902	1618902
99	2	807983	806886	806886
99	3	540063	539170	539170
99	4	405336	403555	403555
99	5	332266	332266	332266
100	1	151223	151223	151223
100	2	76850	76561	76561
100	3	57811	57811	57811
100	4	49623	48347	48102
100	5	53383	48631	48041

同方案的逻辑额外搬运量与无 L2 相同；无 L2 命中率不适用。

表 17 问题三逐用例额外搬运字节与字节命中率

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
1	1	0	0	0.0000	0.0000
1	2	1152	1152	0.0000	0.0000
1	3	2304	2304	0.0000	0.0000
1	4	3456	3456	0.0000	0.0000
1	5	4608	4608	0.0000	0.0000
2	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
2	2	216576	245760	0.0372	0.0369
2	3	423936	423936	0.0000	0.0000
2	4	377856	377856	0.0000	0.0000
2	5	514560	514560	0.0031	0.0031
3	1	0	0	0.0000	0.0000
3	2	6564470	6564470	0.0743	0.0777
3	3	9497892	9497892	0.2174	0.2174
3	4	7333194	7333194	0.2996	0.2996
3	5	7908308	7908308	0.3395	0.3395
4	1	0	0	0.0000	0.0000
4	2	110592	110592	0.0000	0.0000
4	3	221184	221184	0.0000	0.0000
4	4	331776	331776	0.0000	0.0000
4	5	442368	442368	0.0000	0.0000
5	1	0	0	0.0000	0.0000
5	2	882310	882310	0.1366	0.1366
5	3	1461452	1465868	0.2530	0.2527
5	4	1495922	1495922	0.2631	0.2631
5	5	1688564	1688564	0.2921	0.2921
6	1	16400	16400	0.0293	0.0293
6	2	17280	17280	0.0308	0.0308
6	3	118176	118176	0.0837	0.0837
6	4	203696	203696	0.0979	0.0979
6	5	219928	219928	0.1174	0.1174
7	1	0	0	0.0000	0.0000
7	2	27648	27648	0.0000	0.0000
7	3	55296	55296	0.0000	0.0000
7	4	82944	82944	0.0000	0.0000
7	5	110592	110592	0.0000	0.0000
8	1	0	0	0.0000	0.0000
8	2	0	0	0.0000	0.0000
8	3	0	0	0.0000	0.0000
8	4	0	0	0.0000	0.0000
8	5	0	0	0.0000	0.0000
9	1	0	0	0.0000	0.0000
9	2	547342	547342	0.0663	0.0663
9	3	737384	737384	0.0851	0.0851
9	4	838982	838982	0.1867	0.1867
9	5	898504	898504	0.2349	0.2349
10	1	0	0	0.0000	0.0000
10	2	311944	300424	0.1418	0.1275
10	3	48384	48384	0.1119	0.1119
10	4	483232	485920	0.2698	0.2772
10	5	562432	562432	0.2929	0.2929
11	1	0	0	0.0000	0.0000
11	2	180224	180224	0.0000	0.0000
11	3	1660928	1660928	0.1861	0.1953

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
11	4	540672	540672	0.0000	0.0000
11	5	2033664	2033664	0.2430	0.2576
12	1	0	0	0.0000	0.0000
12	2	94960	94960	0.0204	0.0204
12	3	189892	189892	0.0673	0.0673
12	4	39168	39168	0.0396	0.0727
12	5	25344	25344	0.0497	0.0497
13	1	0	0	0.0000	0.0000
13	2	8192	8192	0.0000	0.0000
13	3	16384	16384	0.0000	0.0000
13	4	24576	24576	0.0000	0.0000
13	5	32768	32768	0.0000	0.0000
14	1	71010720	71010720	0.1229	0.1229
14	2	69998520	69998520	0.2013	0.2013
14	3	56526504	57041304	0.2471	0.2363
14	4	59878296	59878296	0.2981	0.2963
14	5	62443416	62443416	0.2880	0.3113
15	1	180256	180256	0.1693	0.1693
15	2	109824	109824	0.1105	0.1105
15	3	298252	298252	0.2522	0.2522
15	4	365824	365824	0.2140	0.2140
15	5	326144	326144	0.2694	0.2694
16	1	0	0	0.0000	0.0000
16	2	404200	404200	0.4965	0.4965
16	3	341100	341100	0.4512	0.4512
16	4	605676	605676	0.5964	0.5964
16	5	444880	444880	0.5163	0.5163
17	1	22528	22528	0.0404	0.0404
17	2	87552	87552	0.0617	0.0617
17	3	175104	175104	0.0735	0.0735
17	4	259200	259200	0.0464	0.0464
17	5	331776	331776	0.1183	0.1183
18	1	1970176	1970176	0.3438	0.3438
18	2	2638848	2638848	0.4835	0.4835
18	3	3971072	3971072	0.5357	0.5357
18	4	4938752	4938752	0.5624	0.5624
18	5	5320704	5320704	0.5712	0.5712
19	1	0	0	0.0000	0.0000
19	2	354360	370744	0.1209	0.1183
19	3	54336	54336	0.2326	0.2326
19	4	762456	762456	0.1934	0.1934
19	5	255068	279644	0.0784	0.3117
20	1	0	0	0.0000	0.0000
20	2	0	0	0.0000	0.0000
20	3	0	0	0.0000	0.0000
20	4	0	0	0.0000	0.0000
20	5	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
21	1	4501504	4501504	0.5006	0.5006
21	2	33017856	33017856	0.0150	0.0150
21	3	34854912	34854912	0.0165	0.0165
21	4	4608000	4608000	0.0000	0.0000
21	5	11431936	11431936	0.5700	0.5700
22	1	0	0	0.0000	0.0000
22	2	145236	145236	0.0749	0.0749
22	3	13824	13824	0.0608	0.0608
22	4	20736	20736	0.0885	0.0885
22	5	20736	20736	0.0885	0.0885
23	1	0	0	0.0000	0.0000
23	2	74016	74016	0.0006	0.0006
23	3	207264	207264	0.3232	0.3232
23	4	281280	281280	0.3932	0.3932
23	5	280992	280992	0.2555	0.2555
24	1	0	0	0.0000	0.0000
24	2	396856	396856	0.4988	0.4988
24	3	332124	332124	0.4534	0.4534
24	4	595068	595068	0.5988	0.5988
24	5	432232	432232	0.5188	0.5188
25	1	0	0	0.0000	0.0000
25	2	27648	27648	0.0000	0.0000
25	3	55296	55296	0.0000	0.0000
25	4	82944	82944	0.0000	0.0000
25	5	110592	110592	0.0000	0.0000
26	1	0	0	0.0000	0.0000
26	2	21936	21936	0.0429	0.0429
26	3	95664	95664	0.1637	0.1637
26	4	172128	172128	0.2232	0.2232
26	5	835212	835212	0.1869	0.2337
27	1	1400832	1400832	0.2689	0.2689
27	2	1744896	1744896	0.0890	0.0890
27	3	18481152	18481152	0.0599	0.0599
27	4	3317760	3317760	0.0000	0.0000
27	5	21954560	21948416	0.1943	0.1969
28	1	277412256	277412256	0.0000	0.0000
28	2	75738432	75738432	0.0383	0.0383
28	3	44350128	44353200	0.1475	0.1489
28	4	74883456	74883456	0.5881	0.5881
28	5	73953792	73953792	0.6104	0.6104
29	1	0	0	0.0000	0.0000
29	2	16388	16388	0.0000	0.0000
29	3	0	0	0.0000	0.0000
29	4	49164	49164	0.0000	0.0000
29	5	65552	65552	0.0000	0.0000
30	1	5519360	5519360	0.2599	0.2599
30	2	1292288	1292288	0.3451	0.3451

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
30	3	8985600	8985600	0.5365	0.5365
30	4	6002688	6002688	0.7014	0.7014
30	5	7484416	7484416	0.7449	0.7449
31	1	3491328	3491328	0.2606	0.2606
31	2	5250048	5250048	0.4305	0.4305
31	3	7544832	7544832	0.4998	0.4998
31	4	7808000	7808000	0.4349	0.4349
31	5	7931904	7931904	0.5328	0.5328
32	1	0	0	0.0000	0.0000
32	2	6216	6216	0.0218	0.0218
32	3	12360	12360	0.0424	0.0424
32	4	74980	74980	0.0385	0.0385
32	5	12360	12360	0.0424	0.0424
33	1	4592640	4592640	0.0963	0.0963
33	2	6452256	6452256	0.1705	0.1705
33	3	7053888	7053888	0.3033	0.3033
33	4	7214688	7214688	0.4267	0.4267
33	5	7894656	7894656	0.3110	0.3110
34	1	1079832	1079832	0.3090	0.3090
34	2	943616	943616	0.2927	0.2927
34	3	2288640	2288640	0.4821	0.4915
34	4	3059200	3083776	0.5684	0.5625
34	5	3463168	3487744	0.5879	0.5832
35	1	0	0	0.0000	0.0000
35	2	801138	801138	0.1547	0.1547
35	3	1276568	1254808	0.1948	0.1962
35	4	1505978	1505978	0.2872	0.2872
35	5	1058710	1058710	0.2730	0.2730
36	1	0	0	0.0000	0.0000
36	2	1152	1152	0.0000	0.0000
36	3	2304	2304	0.0000	0.0000
36	4	3456	3456	0.0000	0.0000
36	5	4608	4608	0.0000	0.0000
37	1	0	0	0.0000	0.0000
37	2	204800	204800	0.0000	0.0000
37	3	573440	573440	0.3537	0.3537
37	4	614400	614400	0.0000	0.0000
37	5	942080	942080	0.4222	0.4222
38	1	5627904	5627904	0.1736	0.1736
38	2	6790656	6790656	0.2788	0.2788
38	3	8107008	8107008	0.3442	0.3442
38	4	8954880	8954880	0.3324	0.3324
38	5	9592320	9592320	0.4060	0.4060
39	1	7929856	7929856	0.0000	0.0000
39	2	9166848	9166848	0.4136	0.4136
39	3	11911168	11911168	0.4837	0.4837
39	4	14536704	14848000	0.5318	0.5736

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
39	5	14536704	14536704	0.4645	0.4645
40	1	0	0	0.0000	0.0000
40	2	1227916	1227916	0.1273	0.1273
40	3	1985768	1985768	0.2179	0.2179
40	4	1836692	1836692	0.2695	0.2695
40	5	1915296	1915296	0.3204	0.3204
41	1	30898176	30898176	0.1016	0.1016
41	2	33669120	33669120	0.2225	0.2225
41	3	34851840	34851840	0.2605	0.2605
41	4	39164928	39164928	0.3438	0.3438
41	5	40630272	40630272	0.3177	0.3177
42	1	6629376	6629376	0.3187	0.3187
42	2	7262496	7262496	0.4130	0.4130
42	3	8474176	8474176	0.3819	0.3819
42	4	8605536	8605536	0.4043	0.4043
42	5	9936736	9936736	0.5569	0.5569
43	1	49160	49160	0.0332	0.0332
43	2	3210754	3210754	0.2306	0.2306
43	3	4852554	4852554	0.3110	0.3164
43	4	5894554	5894554	0.3500	0.3694
43	5	6609206	6592822	0.3748	0.3820
44	1	4438112	4438112	0.8200	0.8200
44	2	886880	885856	0.1451	0.1879
44	3	891584	1149632	0.2131	0.3724
44	4	1389344	1468192	0.2893	0.5055
44	5	655488	655488	0.2112	0.2112
45	1	0	0	0.0000	0.0000
45	2	0	0	0.0000	0.0000
45	3	0	0	0.0000	0.0000
45	4	0	0	0.0000	0.0000
45	5	0	0	0.0000	0.0000
46	1	3176288	3176288	0.7530	0.7530
46	2	942688	942688	0.0755	0.0755
46	3	2958528	2958528	0.1543	0.4851
46	4	2098976	2316064	0.1966	0.2580
46	5	2660736	2920832	0.2171	0.4623
47	1	0	0	0.0000	0.0000
47	2	1264006	1264006	0.1064	0.1064
47	3	1857324	1857324	0.2092	0.2092
47	4	2256146	2256146	0.2820	0.2820
47	5	2575320	2575320	0.3312	0.3312
48	1	0	0	0.0000	0.0000
48	2	467846	467846	0.1452	0.1452
48	3	1177612	1177612	0.1576	0.1576
48	4	1270930	1270930	0.1970	0.1970
48	5	1273560	1273560	0.1722	0.1722
49	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
49	2	1027062	1027062	0.0341	0.0341
49	3	2408560	2408560	0.0989	0.0989
49	4	2607184	2607184	0.1189	0.1189
49	5	2907980	2907980	0.1535	0.1535
50	1	0	0	0.0000	0.0000
50	2	1022694	1022694	0.0460	0.0460
50	3	1831624	1831624	0.1256	0.1256
50	4	2053712	2043408	0.1550	0.1536
50	5	1337182	1337182	0.1440	0.1440
51	1	0	0	0.0000	0.0000
51	2	394084	394084	0.4997	0.4997
51	3	328736	328736	0.4543	0.4543
51	4	591064	591064	0.5997	0.5997
51	5	427456	427456	0.5197	0.5197
52	1	0	0	0.0000	0.0000
52	2	102912	102912	0.1127	0.1127
52	3	181248	181248	0.1828	0.1828
52	4	254976	254976	0.2264	0.2264
52	5	333312	333312	0.2369	0.2369
53	1	5405536	5405536	0.6249	0.6249
53	2	4631434	4631434	0.2397	0.2397
53	3	6090084	6090084	0.2527	0.2527
53	4	8679750	8679750	0.2221	0.2221
53	5	9823040	9823040	0.2537	0.2537
54	1	0	0	0.0000	0.0000
54	2	2620302	2620302	0.0559	0.0559
54	3	4112418	4112418	0.1204	0.1204
54	4	7279264	7284480	0.1450	0.1450
54	5	8197794	8197794	0.1659	0.1659
55	1	69632	69632	0.1001	0.1001
55	2	16384	16384	0.0255	0.0255
55	3	74240	74240	0.1061	0.1061
55	4	115712	115712	0.1188	0.1188
55	5	173568	173568	0.1653	0.1653
56	1	0	0	0.0000	0.0000
56	2	3326338	3326338	0.0445	0.0451
56	3	3278908	3278908	0.0939	0.0939
56	4	3685886	3685886	0.1385	0.1385
56	5	4124830	4124830	0.1673	0.1673
57	1	94216	94216	0.2187	0.2187
57	2	22304	22304	0.0621	0.0621
57	3	14112	14112	0.0008	0.0008
57	4	28224	28224	0.0774	0.0774
57	5	14400	14400	0.0016	0.0016
58	1	28696576	28696576	0.0000	0.0000
58	2	29548544	29548544	0.1157	0.1157
58	3	30400512	30400512	0.0538	0.0538

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
58	4	31551488	31821824	0.4236	0.4247
58	5	32104448	32104448	0.3095	0.3095
59	1	1691648	1691648	0.6100	0.6100
59	2	360448	360448	0.0000	0.0000
59	3	1171456	1171456	0.4000	0.5191
59	4	1081344	1081344	0.0000	0.0000
59	5	2072576	2072576	0.5442	0.5442
60	1	2068520	2068520	0.4231	0.4231
60	2	2746508	2746508	0.4789	0.4789
60	3	3159296	3159296	0.4754	0.4754
60	4	3960200	3960200	0.4592	0.4592
60	5	3733888	3733888	0.5229	0.5229
61	1	0	0	0.0000	0.0000
61	2	34560	34560	0.0597	0.0597
61	3	43008	43008	0.0733	0.0733
61	4	62208	62208	0.0709	0.0709
61	5	110592	110592	0.1583	0.1583
62	1	0	0	0.0000	0.0000
62	2	2347008	2325504	0.0000	0.0009
62	3	2268672	2268672	0.0000	0.0000
62	4	2603520	2582016	0.0012	0.0012
62	5	2746368	2746368	0.0000	0.0000
63	1	0	0	0.0000	0.0000
63	2	1049088	1049088	0.0057	0.0057
63	3	1262592	1262592	0.0063	0.0063
63	4	1420800	1420800	0.0086	0.0086
63	5	1539072	1539072	0.0100	0.0100
64	1	0	0	0.0000	0.0000
64	2	47620	47620	0.0256	0.0256
64	3	41446	41446	0.1366	0.1366
64	4	130956	130956	0.2073	0.2073
64	5	159314	159314	0.2249	0.2249
65	1	0	0	0.0000	0.0000
65	2	13968	13968	0.0445	0.0445
65	3	27936	27936	0.0431	0.0431
65	4	41904	41904	0.0822	0.0822
65	5	55872	55872	0.1312	0.1312
66	1	0	0	0.0000	0.0000
66	2	1653418	1653418	0.0916	0.0916
66	3	2500738	2500738	0.2189	0.2189
66	4	3806464	3806464	0.2231	0.2231
66	5	3979388	3979388	0.2264	0.2264
67	1	264449408	264449408	0.0000	0.0000
67	2	215994240	215994240	0.2151	0.2151
67	3	159238912	159238912	0.6616	0.6616
67	4	226600576	226600576	0.3433	0.3433
67	5	229484544	229484544	0.3472	0.3472

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
68	1	0	0	0.0000	0.0000
68	2	1624576	1624576	0.1013	0.1013
68	3	2355384	2355384	0.1847	0.1847
68	4	2646854	2646854	0.0877	0.0877
68	5	5510878	5510878	0.1944	0.1944
69	1	0	0	0.0000	0.0000
69	2	364198	364006	0.1287	0.1291
69	3	325098	325098	0.2892	0.2964
69	4	361896	361896	0.2733	0.2733
69	5	433008	437616	0.2238	0.2516
70	1	0	0	0.0000	0.0000
70	2	129536	129536	0.0513	0.1463
70	3	259072	259072	0.2730	0.2730
70	4	333312	333312	0.3419	0.3419
70	5	449024	449024	0.3041	0.3041
71	1	0	0	0.0000	0.0000
71	2	171622	171622	0.1003	0.1003
71	3	250696	246088	0.3087	0.3114
71	4	223406	223406	0.1782	0.2041
71	5	307214	307214	0.2663	0.2663
72	1	108613680	108613680	0.0262	0.0262
72	2	113724192	113724192	0.1242	0.1242
72	3	120851232	120851232	0.1151	0.1151
72	4	123071264	123071264	0.1695	0.1695
72	5	77245216	77245216	0.2340	0.2340
73	1	141570432	141570432	0.0000	0.0000
73	2	86179712	86179712	0.2207	0.2207
73	3	134138624	134138624	0.3086	0.3086
73	4	137012864	137012864	0.3213	0.3213
73	5	94477312	96247040	0.2336	0.3236
74	1	6070280	6070280	0.1668	0.1668
74	2	3776512	3776512	0.3465	0.3465
74	3	5619712	5619712	0.4595	0.4595
74	4	6623232	6623232	0.4171	0.4171
74	5	7151616	7151616	0.4715	0.4715
75	1	0	0	0.0000	0.0000
75	2	4745798	4745798	0.1045	0.1045
75	3	12507404	12507404	0.2335	0.2335
75	4	11096688	11096688	0.2466	0.2466
75	5	12427608	12427608	0.2615	0.2615
76	1	28280832	28280832	0.1336	0.1336
76	2	31095808	31095808	0.2788	0.2788
76	3	17912832	17794048	0.5245	0.5505
76	4	19563520	19563520	0.5526	0.5526
76	5	22311936	22311936	0.6242	0.6242
77	1	0	0	0.0000	0.0000
77	2	57344	57344	0.0261	0.0261

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
77	3	114688	114688	0.0254	0.0254
77	4	114688	114688	0.0508	0.0508
77	5	172032	172032	0.0744	0.0744
78	1	12644568	12644568	0.0000	0.0000
78	2	2782080	2782080	0.3321	0.3321
78	3	6965760	6965760	0.3449	0.4940
78	4	6950784	6950784	0.6988	0.6988
78	5	10766304	10766304	0.6982	0.6982
79	1	32448512	32448512	0.0000	0.0000
79	2	194232320	193798144	0.0009	0.0011
79	3	39960576	39960576	0.1373	0.1373
79	4	31801344	31801344	0.2438	0.2438
79	5	34084864	34283520	0.4735	0.4624
80	1	548864	548864	0.1686	0.1686
80	2	1675264	1675264	0.3902	0.3902
80	3	3047424	3047424	0.0000	0.0000
80	4	2666496	2666496	0.4597	0.4597
80	5	3047424	3047424	0.2051	0.4359
81	1	0	0	0.0000	0.0000
81	2	0	0	0.0000	0.0000
81	3	0	0	0.0000	0.0000
81	4	0	0	0.0000	0.0000
81	5	0	0	0.0000	0.0000
82	1	0	0	0.0000	0.0000
82	2	3389668	3389668	0.0594	0.0733
82	3	5338828	5338828	0.1725	0.1725
82	4	6420240	6420240	0.2422	0.2422
82	5	7708408	7708408	0.3140	0.3140
83	1	13934688	13934688	0.7959	0.7959
83	2	5589600	5589600	0.1595	0.1595
83	3	5739712	5739712	0.1416	0.1416
83	4	6471456	6471456	0.3448	0.3448
83	5	7741824	7741824	0.1860	0.1860
84	1	0	0	0.0000	0.0000
84	2	32004864	32000640	0.0052	0.0052
84	3	0	0	0.0000	0.0000
84	4	0	0	0.0000	0.0000
84	5	0	0	0.0000	0.0000
85	1	0	0	0.0000	0.0000
85	2	17607566	17607566	0.0753	0.0753
85	3	31680716	31680716	0.1996	0.2009
85	4	27902578	27902578	0.1863	0.1863
85	5	38398484	38398484	0.2762	0.2762
86	1	0	0	0.0000	0.0000
86	2	1019206	1019206	0.1717	0.1717
86	3	2197292	2183372	0.1747	0.1816
86	4	2353614	2353614	0.2334	0.2334

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
86	5	2265748	2265748	0.2584	0.2632
87	1	7902720	7902720	0.1834	0.1834
87	2	10142208	10142208	0.2955	0.2955
87	3	11036160	11036160	0.2615	0.2615
87	4	12607488	12607488	0.2995	0.2995
87	5	12487680	12487680	0.3022	0.3022
88	1	0	0	0.0000	0.0000
88	2	1032234	1032234	0.0924	0.0924
88	3	1572244	1572244	0.2117	0.2117
88	4	2028164	1954436	0.0682	0.0808
88	5	2034888	2034888	0.0803	0.0803
89	1	0	0	0.0000	0.0000
89	2	1467520	1467520	0.0194	0.0194
89	3	2429184	2429184	0.1780	0.1780
89	4	1827200	1827200	0.1027	0.1027
89	5	2186320	2186320	0.1429	0.1429
90	1	21233592	21233592	0.0000	0.0000
90	2	3196848	3196848	0.2160	0.2933
90	3	5523456	5524224	0.2970	0.4317
90	4	7360608	7550112	0.3184	0.3847
90	5	7981056	8312448	0.3274	0.4516
91	1	53947008	53947008	0.1054	0.1054
91	2	52032556	52032556	0.3523	0.3523
91	3	60113508	60113508	0.4441	0.4441
91	4	67112896	67112896	0.4855	0.4855
91	5	71035764	71035764	0.5336	0.5336
92	1	66000096	66000096	0.7935	0.7935
92	2	10740320	10740320	0.1172	0.1173
92	3	21856448	21848256	0.1120	0.1331
92	4	18053920	18267936	0.1479	0.2199
92	5	19945856	20375936	0.2219	0.2708
93	1	0	0	0.0000	0.0000
93	2	8192	8192	0.0000	0.0000
93	3	16384	16384	0.0000	0.0000
93	4	24576	24576	0.0000	0.0000
93	5	32768	32768	0.0000	0.0000
94	1	0	0	0.0000	0.0000
94	2	106272	106272	0.0563	0.0563
94	3	212544	212544	0.3108	0.3108
94	4	318528	318528	0.3624	0.3624
94	5	424800	424800	0.2478	0.2478
95	1	0	0	0.0000	0.0000
95	2	0	0	0.0000	0.0000
95	3	0	0	0.0000	0.0000
95	4	0	0	0.0000	0.0000
95	5	0	0	0.0000	0.0000
96	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
96	2	346520	346520	0.2773	0.2773
96	3	123264	123264	0.2038	0.2038
96	4	234168	234168	0.1969	0.1969
96	5	448896	448896	0.3090	0.3090
97	1	10584064	10584064	0.0000	0.0000
97	2	2490368	2490368	0.2629	0.2629
97	3	82444288	82444288	0.0535	0.0535
97	4	14221312	14221312	0.0903	0.0903
97	5	21311488	21311488	0.2452	0.2452
98	1	0	0	0.0000	0.0000
98	2	0	0	0.0000	0.0000
98	3	784240	784240	0.2601	0.2601
98	4	961274	961274	0.3211	0.3211
98	5	1166870	1166870	0.3037	0.3037
99	1	5484544	5484544	0.2524	0.2524
99	2	5906432	5906432	0.4562	0.4562
99	3	6479872	6479872	0.4862	0.4862
99	4	9838592	9838592	0.5568	0.5568
99	5	7084052	7084052	0.6682	0.6682
100	1	0	0	0.0000	0.0000
100	2	621700	621700	0.2955	0.2955
100	3	72192	72192	0.0171	0.0171
100	4	1207894	1228630	0.4199	0.4249
100	5	1599352	1580920	0.4154	0.4202